
INGENIC®

Halley6 Linux Kernel Development Manual

——Kernel4.4.94&kernel5.10 General
purpose Edition
Author: System Software Department
Version: 1.0
Date: 2022.11.07



北京君正集成电路股份有限公司
Ingenic Semiconductor Co., Ltd.

君正®

Halley6 Linux Kernel Development Manual

Copyright © Ingenic Semiconductor Co. Ltd 2020. All rights reserved.

Release history

Date	Revision	Change
2022.11.07	2.0	First release

Disclaimer

This documentation is provided for use with Ingenic products. No license to Ingenic property rights is granted. Ingenic assumes no liability, provides no warranty either expressed or implied relating to the usage, or intellectual property right infringement except as provided for by Ingenic Terms and Conditions of Sale.

Ingenic products are not designed for and should not be used in any medical or life sustaining or supporting equipment.

All information in this document should be treated as preliminary. Ingenic may make changes to this document without notice. Anyone relying on this documentation should contact Ingenic for the current documentation and errata.

Ingenic Semiconductor Co., Ltd.

Ingenic Headquarters, East Bldg. 14, Courtyard #10
Xibeiwang East Road, Haidian District, Beijing, China,
Tel: 86-10-56345000
Fax: 86-10-56345001
Http: //www.ingenic.com

北京君正集成电路股份有限公司

地址:北京市海淀区东北西路中关村软件园二期君正总部大楼
电话: 86-10-56345000
传真: 86-10-56345001
http: //www.ingenic.com

目 录

1	Introduction to kernel development	15
1.1	Basic kernel structure	15
1.2	Kernel Development process	15
1.2.1	Junzheng SDK Kernel Development Process (recommended)	16
1.2.2	Development process based on kernel source code	17
1.3	Kernel default configurations.....	17
1.4	Device Tree Configuration.....	18
1.4.1	Device Tree File Description.....	18
1.4.2	Kernel Built-in device tree (default usage).....	19
1.4.3	Compile the device tree separately	19
1.4.3.1	Modify uboot configurations	19
1.4.3.2	Compile dtb files.....	20
1.4.3.3	Burn dtb files	20
1.4.4	Passing kernel parameters through the device tree.....	22
1.4.4.1	Modify the device tree	22
1.4.4.2	Modify the kernel configurations	23
2	CIM camera interface module	1
2.1	Module Function Description.....	1
2.2	Driver source code location.....	1
2.3	Device tree configurations.....	1
2.3.1	Default configurations of device tree	1
2.3.2	Device tree custom configurations	2
2.3.2.1	CIM controller configuration	2
2.3.2.2	camera sensor configurations	3
2.4	Kernel compilation configurations	4
2.4.1	Kernel default compilation configurations.....	4
2.4.2	Kernel custom compilation configurations	4
2.4.2.1	CIM Controller configurations.....	4
2.4.2.2	camera sensor configurations	5
2.5	Kernel difference	5
2.6	Device node generation	5
2.7	Application instructions.....	6
2.7.1	cimutils.....	6
2.7.2	v4l2-ctl.....	6
3	Display Controller.....	1
3.1	Module Function Description.....	1
3.2	Driver source location.....	1

3.3	Add Screen.....	1
3.4	Device Tree Configuration.....	2
3.4.1	Device Tree Default Configuration.....	2
3.4.1.1	DPU Controller Configuration.....	2
3.4.1.2	Display Configuration	2
3.4.2	Device tree User-defined configuration	3
3.4.2.1	X1600 DPU Supported screen types	3
3.4.2.2	Match other signal display.....	3
3.5	Module kernel differences	3
3.6	Screen Configuration.....	3
3.6.1	smart lcd configuration.....	4
3.6.2	tft lcd configuration.....	5
3.6.3	PWM Backlight configuration.....	5
3.7	Kernel compilation Configuration	6
3.8	Device Node Generation	6
4	Audio subsystem.....	2
4.1	Module function introduction	2
4.2	Hardware Description.....	2
4.3	Driver source code location.....	3
4.4	Device Tree Configuration.....	3
4.4.1	Device tree default configuration	3
4.4.2	Device Tree Custom Configuration.....	4
4.5	Kernel compilation configuration	4
4.5.1	Kernel default compilation configuration.....	4
4.5.2	Kernel Custom Compilation Configuration	5
4.6	Device Node Generation	5
4.7	Add new codec driver	5
4.8	Application Instructions	6
4.8.1	Recording and playback method	6
4.8.2	Introduction of asound.conf	6
4.8.2.1	hw plugin	6
4.8.2.2	slave plugin	7
4.8.2.3	rate plugin.....	8
4.8.2.4	plug plugin	8
4.8.2.5	Soft Volume	8
4.8.3	Usage of tloop.....	9
4.8.4	FAQ.....	9
4.8.4.1	i2s rx related clocks are available, tx related clocks are not	9
4.8.4.2	Mono sound is small	9
4.8.4.3	Audio too fast or too slow	10
4.8.4.4	underrun/overrun questions	10

4.8.4.5	pcm_read:2145: read error: Input/output error.....	10
4.8.4.6	Unable to install hw params	10
5	NEMC EXTERNAL STORAGE CONTROLLER INTERFACE.....	2
5.1	Module function introductions	2
5.2	Driver source code location	2
5.3	Device Tree Configurations	2
5.3.1	Device tree default configurations	3
5.3.2	Device Tree Custom Configurations	3
5.4	Kernel compilation configurations	3
5.4.1	Kernel default compilation configurations.....	3
5.5	Module kernel differences	3
5.6	Device node generation	3
6	SFC Controller	2
6.1	Introduction to module functions	2
6.2	Location of driver source code	2
6.3	Configuration of the device tree	2
6.3.1	Default configuration of the device tree	3
6.3.2	Customized configuration of the device tree	4
6.4	Configuration of kernel compilation.....	4
6.4.1	Default kernel compilation configuration.....	4
6.4.2	Kernel customized compilation configuration	4
6.5	Generation of device nodes	5
6.6	Application instructions.....	5
6.6.1	Flash read and write test	5
6.7	Add a new flash parameter	6
6.7.1	Add NOR flash parameter	6
6.7.2	Add NAND flash parameter	7
6.8	Note	9
7	CPM Clock Power Reset Interface	2
7.1	Module function introduction	2
7.2	Drive position.....	2
7.3	Device Tree Configuration.....	2
7.3.1	Device tree default configuration	3
7.4	Kernel compilation configuration	3
7.4.1	Kernel default compilation configuration.....	3
7.4.2	Kernel custom compilation configuration	3
7.5	Device Node Generation	3
7.6	Application Instructions	6
7.6.1	Use of the clock	6

8	TCU timer unit	2
8.1	Module function introduction	2
8.2	Drive position	2
8.3	Device Tree Configuration	2
8.3.1	Device Tree Default Configuration	3
8.3.2	Device Tree Custom Configuration	3
8.4	Kernel compilation configuration	3
8.4.1	Kernel default compilation configuration	3
8.4.2	Kernel Custom Compilation Configuration	3
8.5	Device Node Generation	3
8.6	Application Instructions	4
9	INTC Interrupt Controller	2
9.1	Module function introduction	2
9.2	Drive position	2
9.3	Device Tree Configuration	2
9.3.1	Device Tree Default Configuration	2
9.3.2	The default compilation produces intc devices	2
9.4	Kernel compilation configuration	2
9.4.1	Kernel default compilation configuration	3
9.4.2	Kernel Custom Compilation Configuration	3
9.5	Device Node Generation	3
9.6	Application Instructions	3
10	BLUETOOTH	2
10.1	Module Introduction	2
10.1.1	AP6212 chip	2
10.1.2	GPIO Function Description	2
10.2	Driver source location	2
10.3	Device Tree Configuration	2
10.3.1	Device Tree Default Configuration	2
10.3.2	Device Tree User-defined Configuration	3
10.4	Kernel compilation configuration	3
10.4.1	Kernel Default compilation Configuration	3
10.4.2	Kernel Custom Compilation Configuration	3
10.5	Kernel version differences	3
10.6	Device Node Generation	3
10.7	Application Instructions	3
10.7.1	Module Power Supply	3
10.7.2	Obtaining the Bluetooth Module ID Through the Serial Port	4
10.7.3	Bluetooth uart channel test	4

10.7.4	Bluetooth pcm channel test	4
10.7.5	Develop reference content based on bsa server.....	4
10.8	FAQ	5
10.8.1	bsa_server does not support -lpm parameter.....	5
10.8.2	Common normal error print, do not affect the normal use.....	5
11	THE WATCHDOG MODULE	6
11.1	Module function introductions	6
11.2	Driver source code location.....	6
11.3	Device Tree Configurations	6
11.3.1	Default Configurations of Device Tree.....	6
11.3.2	Device Tree Custom Configurations.....	6
11.4	Kernel compilation configurations	2
11.4.1	Kernel default compilation configurations.....	2
11.4.2	Kernel custom compilation configurations.....	2
11.5	Module kernel differences	2
11.6	Device node generation	2
11.7	Application Instructions	2
12	PDMA controller.....	2
12.1	Module function introduction	2
12.2	Drive position.....	2
12.3	Device Tree Configuration.....	2
12.3.1	Device Tree Default Configuration.....	2
12.3.2	Device Tree Custom Configuration.....	2
12.4	Kernel compilation configuration	3
12.4.1	Kernel default compilation configuration.....	3
12.4.2	Kernel Custom Compilation Configuration	3
12.5	Version Differences	4
12.6	Device Node Generation	4
12.6.1	Controller Node.....	4
12.6.2	Test program node.....	4
12.7	How to use mcu.....	4
12.7.1	mcu firmware introduction.....	5
12.7.2	Introduction of mcu loader	5
12.7.3	How to use mcu loader	5
12.8	Application Instructions	6
12.8.1	Dmatest test program	6
12.8.2	SSI driver test	6
12.8.2.1	Device Tree Configuration.....	6
12.8.2.2	Driver usage flow.....	7
13	THE SADC CONTROLLER.....	2

13.1	Module function introductions	2
13.2	Driver location	2
13.3	Device Tree Configurations	2
13.3.1	Device tree default configurations	2
13.3.2	Device Tree Custom Configurations	2
13.4	Kernel compilation configurations	3
13.4.1	Kernel default compilation configurations	3
13.5	Module kernel differences	3
13.6	Device node generation	4
13.7	Application Instructions	4
13.7.1	Application source location	4
13.7.2	Commands and Parameters	4
14	THE RTC CONTROLLER	2
14.1	Module function introductions	2
14.2	Driver location	2
14.3	Device Tree Configurations	2
14.3.1	Device tree default configurations	2
14.3.2	Device Tree Custom Configurations	2
14.4	Kernel compilation configurations	3
14.4.1	Kernel default compilation configurations	3
14.4.2	Kernel custom compilation configurations	3
14.5	Module kernel differences	4
14.6	Device node generation	4
14.7	Application Instructions	4
14.7.1	Timed wake-up test	4
15	EFUSE INTERFACE	2
15.1	Module Functions	2
15.2	Driver Position	2
15.3	Device Tree Configuration	2
15.3.1	Device Tree Default Configuration	2
15.3.2	Device Tree User-defined Configuration	2
15.4	Kernel compilation configuration	2
15.4.1	Kernel Default compilation Configuration	3
15.4.2	Kernel Custom Compilation Configuration	3
15.5	Kernel version differences	3
15.6	Device Node Generation	3
15.7	Application Instructions	3
15.7.1	Reading Data in efuse	3
15.8	Precautions	3
16	GPIO General IO interface	2

16.1	Module function introduction	2
16.2	Drive position.....	2
16.3	Device Tree Configuration.....	2
16.3.1	Device Tree Default Configuration.....	3
16.3.2	Device Tree Custom Configuration.....	3
16.3.2.1	Configuring pincfg properties	3
16.3.2.2	Configuring gpio properties	4
16.3.2.3	System sleep GPIO configuration	5
16.4	Kernel compilation configuration	7
16.4.1	Kernel default build configuration	8
16.4.2	Kernel Custom Compilation Configuration	9
16.5	Version Differences	9
16.6	Device Node Generation	9
16.6.1	GPIO number calculation instructions	9
16.6.2	GPIO import and export instructions	10
16.6.3	GPIO property.....	10
16.7	Application Instructions	11
16.7.1	Source code location	11
16.7.2	Test method	11
16.7.3	dump_gpio file	12
17	THE SMB I2C INTERFACE	2
17.1	Module function introductions	2
17.2	Driver location	2
17.3	Device Tree Configurations	2
17.3.1	Device tree default configurations	3
17.3.2	Device Tree Custom Configurations	3
17.4	Kernel compilation configurations	4
17.4.1	Kernel default compilation configurations.....	4
17.4.2	Kernel custom compilation configurations	4
17.5	Module kernel differences	5
17.6	GPIO simulates I2C.....	5
17.7	Device node generation	6
17.8	Application Instructions	6
17.8.1	Application source location	6
17.8.2	Commands and Parameters.....	6
18	SSI SPI controller	7
18.1	Module function introduction	7
18.2	Driver source code location.....	2
18.3	Device Tree Configuration.....	2
18.3.1	Device Tree Default Configuration.....	2

18.3.2	Device Tree Custom Configuration	2
18.3.3	Emulate the spi protocol with gpio	3
18.4	Kernel compilation configuration	4
18.4.1	Kernel default build configuration	5
18.4.2	Kernel Custom Compilation Configuration	5
18.4.3	Emulate the spi protocol with gpio	5
18.5	Device Node Generation	5
18.6	Application Instructions	6
18.6.1	Test spi read and write	6
18.6.1.1	Source code location	6
18.6.1.2	Test method	6
18.6.1.3	Test Results	6
18.6.2	Test spi nor flash	7
18.6.2.1	kernel version 4.4.94 operation	7
18.6.2.2	kernel version 5.10 operation	10
19	SSI SLV SPI controller	2
19.1	Module function introduction	2
19.2	Driver source code location	2
19.3	Device Tree Configuration	2
19.3.1	Device Tree Default Configuration	3
19.3.2	Device Tree Custom Configuration	3
19.4	Kernel compilation configuration	3
19.4.1	Kernel default build configuration	3
19.4.2	Kernel Custom Compilation Configuration	3
19.5	Device Node Generation	3
19.6	Application Instructions	4
19.6.1	Test spi read and write	4
19.6.1.1	Source code location	4
19.6.1.2	Test method	4
19.6.1.3	Test Results	5
20	THE UART MODULE	2
20.1	Module function introductions	2
20.2	Driver location	2
20.3	Device Tree Configurations	2
20.3.1	Default configurations of device tree	2
20.3.2	Device Tree Custom Configurations	3
20.4	Kernel compilation configurations	3
20.4.1	Kernel default compilation configurations	4
20.4.2	Kernel custom compilation configurations	4
20.5	Module kernel differences	4

20.6	Device node generation	4
20.7	Application Instructions	4
20.7.1	Application source location	4
21	SD/SDIO INTERFACE.....	2
21.1	Module Function Description.....	2
21.1.1	GPIO Function Description:	2
21.1.1.1	SDIO	2
21.1.1.2	SD	2
21.1.1.3	EMMC	2
21.1.2	Mapping between MSC controller names:.....	2
21.2	Driver source location.....	3
21.3	Device Tree Configuration.....	3
21.3.1	Device Tree Default Configuration.....	3
21.3.1.1	SDIO	3
21.3.1.2	SD	4
21.3.1.3	EMMC	4
21.3.2	Device Tree User-defined Configuration	5
21.3.2.1	SDIO	5
21.3.2.2	SD	5
21.3.2.3	EMMC	6
21.4	Kernel compilation configuration	6
21.4.1	Kernel Block & wireless device Basic compilation Configuration	6
21.4.1.1	Block device Basic Compilation Configuration.....	6
21.4.1.2	Wireless device Basic Compilation Configuration.....	7
21.4.2	EMMC/SD/SDIO Controller Driver Configuration	7
21.4.2.1	Configuring INGENIC_MMC_MMC0 using EMMC/SD Drivers	7
21.4.2.2	Configuring INGENIC_MMC_MMC1 Using the sdio Driver	7
21.4.3	wifi Driver Configuration.....	7
21.4.4	Kernel Customized Compilation Configuration.....	8
21.5	Kernel version differences.....	8
21.6	Device node generation	8
21.6.1	SDIO	8
21.6.2	SD	8
21.7	Application Instructions	9
21.7.1	EMMC/SD test method	9
21.7.1.1	Write Tests	9
21.7.1.2	Reading test	9
21.7.2	SDIO test method	9
21.7.2.1	Configuring the Network.....	9
21.7.2.2	airkiss distribution network test	10
21.7.2.3	How to use AP mode.....	12

22	USB OTG Controller Interface	2
22.1	Module Function Introduction	2
22.2	Driver source code location	2
22.3	Device Tree Configuration	2
22.3.1	Device Tree Default Configuration	3
22.3.2	Device Tree Custom Configuration	3
22.4	Kernel compilation configuration	4
22.4.1	Kernel default compilation configuration	4
22.4.2	Kernel custom compile configuration	4
22.4.2.1	USB controller mode configuration	4
22.4.2.2	USB host kernel configuration	5
22.4.2.3	USB gadget kernel configuration	8
22.4.2.4	USB Legacy Kernel Configuration	12
22.4.2.5	USB gadget descriptor configuration	13
22.4.2.6	USB Power saving mode	13
22.5	Application Instructions	15
22.5.1	USB host	15
22.5.1.1	USB host Mass Storage	15
22.5.1.2	USB host camera	15
22.5.1.3	USB host hid mouse	16
22.5.1.4	USB host rndis	17
22.5.2	USB device	18
22.5.2.1	USB device mass storage	18
22.5.2.2	USB device adb	18
22.5.2.3	USB device hid	18
22.5.2.4	USB device uvc	19
22.5.2.5	USB device remote NDIS	20
22.5.2.6	USB device serial	21
22.5.2.7	USB device printer	22
22.5.2.8	USB device uac1.0	22
22.5.3	USB legacy	22
22.5.3.1	USB device serial	22
22.5.3.2	USB device uvc	23
22.5.4	USB software switch	23
22.5.5	USB eye map adjustment	25
22.6	matters needing attention	26
22.6.1	Modify the USB gadget configuration	26
22.6.2	When Huawei mobile phones use UVC, the OTG cannot identify the problem Legacy configuration can solve the problem	26
23	GMAC GIGABIT ETHERNET CONTROLLER INTERFACE	2

23.1	Module function introductions	2
23.2	Driver source code location	2
23.3	Device Tree Configurations	2
23.3.1	Device tree default configurations	2
23.4	Kernel compilation configurations	3
23.4.1	Kernel default compilation configurations	3
23.4.2	Kernel custom compilation configurations	3
23.5	Module kernel differences	4
23.6	Device node generation	4
23.7	Application Instructions	4
23.7.1	Network performance test	4
24	AES Encryption and decryption driver interface	2
24.1	Module Function Introduction	2
24.2	Driver source code location	2
24.3	Device Tree Configuration	2
24.3.1	Device Tree Default Configuration	2
24.3.2	Device Tree Custom Configuration	2
24.4	Kernel compile configuration	3
24.4.1	Kernel default compile configuration	3
24.4.2	Kernel custom compile configuration	3
24.5	Device node generation	3
24.6	Application Instructions	3
25	Hash Module drive interface	2
25.1	Module Function Introduction	2
25.2	Driver source code location	2
25.3	Device Tree Configuration	2
25.3.1	Device Tree Default Configuration	2
25.3.2	Device Tree Custom Configuration	2
25.4	Kernel compile configuration	2
25.4.1	Kernel default compile configuration	3
25.4.2	Kernel custom compile configuration	3
25.5	Device node generation	3
25.6	Application Instructions	4
26	PWM Module driver interface	2
26.1	Module function introduction	2
26.2	Driver source code location	2
26.3	Device Tree Configuration	2
26.3.1	Device Tree Default Configuration	3
26.3.2	Device Tree Custom Configuration	3

26.4	Kernel compilation configuration	3
26.4.1	Kernel default compilation configuration.....	4
26.4.2	Kernel Custom Compilation Configuration	4
26.5	Device Node Generation	4
26.5.1	Export of the kernel pwm standard framework	4
26.6	pwm playback.....	5
26.6.1	Kernel compilation configuration	5
26.6.2	Device Tree Configuration	5
26.6.3	Application Compilation	5
26.7	Application Instructions	6
26.7.1	Test method	6
26.7.2	Source code location	7
27	DTRNG Module drive interface.....	2
27.1	Module Function Introduction.....	2
27.2	Driver source code location.....	2
27.3	Device Tree Configuration.....	2
27.3.1	Device Tree Default Configuration.....	2
27.3.2	Device Tree Custom Configuration.....	2
27.4	Kernel compile configuration	3
27.4.1	Kernel default compilation configuration.....	3
27.4.2	Kernel custom compile configuration.....	3
27.5	Device node generation	3
27.6	Application Instructions	3
28	THE CAN MODULE`	2
28.1	Module function introductions	2
28.2	Driver source code location.....	2
28.3	Device Tree Configurations	2
28.3.1	Default Configurations of Device Tree.....	3
28.3.2	Device Tree Custom Configurations.....	3
28.4	Kernel compilation configurations	3
28.4.1	Kernel default compilation configurations.....	3
28.4.2	Kernel custom compilation configurations	4
28.5	Module kernel differences	4
28.6	Device node generation	4
28.7	Application Instructions	5
29	THE WATCHDOG MODULE	2
29.1	Module function introductions	2
29.2	Driver source code location.....	2
29.3	Device Tree Configurations	2

29.3.1	Default Configurations of Device Tree	2
29.3.2	Device Tree Custom Configurations	2
29.4	Kernel compilation configurations	3
29.4.1	Kernel default compilation configurations	3
29.4.2	Kernel custom compilation configurations	3
29.5	Module kernel differences	3
29.6	Device node generation	3
29.7	Application Instructions	3

Key words	Document Format
Pick to	This document describes the basic document format.
Abbreviations	
Reference Materials	

1 Introduction to kernel development

This document focuses on the research and development of Junzheng series chips and Kernelx.x standard kernel platform, and mainly describes the basic structure of the kernel, kernel compilation, device tree and driver configuration and tailoring.

1.1 Basic kernel structure

The controller of Junzheng series chip is adapted to the driver layer in Kernelx.x, and the driver interface used also uses the standard driver interface of the kernel. Users can conduct secondary development based on the standard application interface, without caring about the specific implementation of the bottom layer.

The following figure shows the basic kernel adaptation structure:

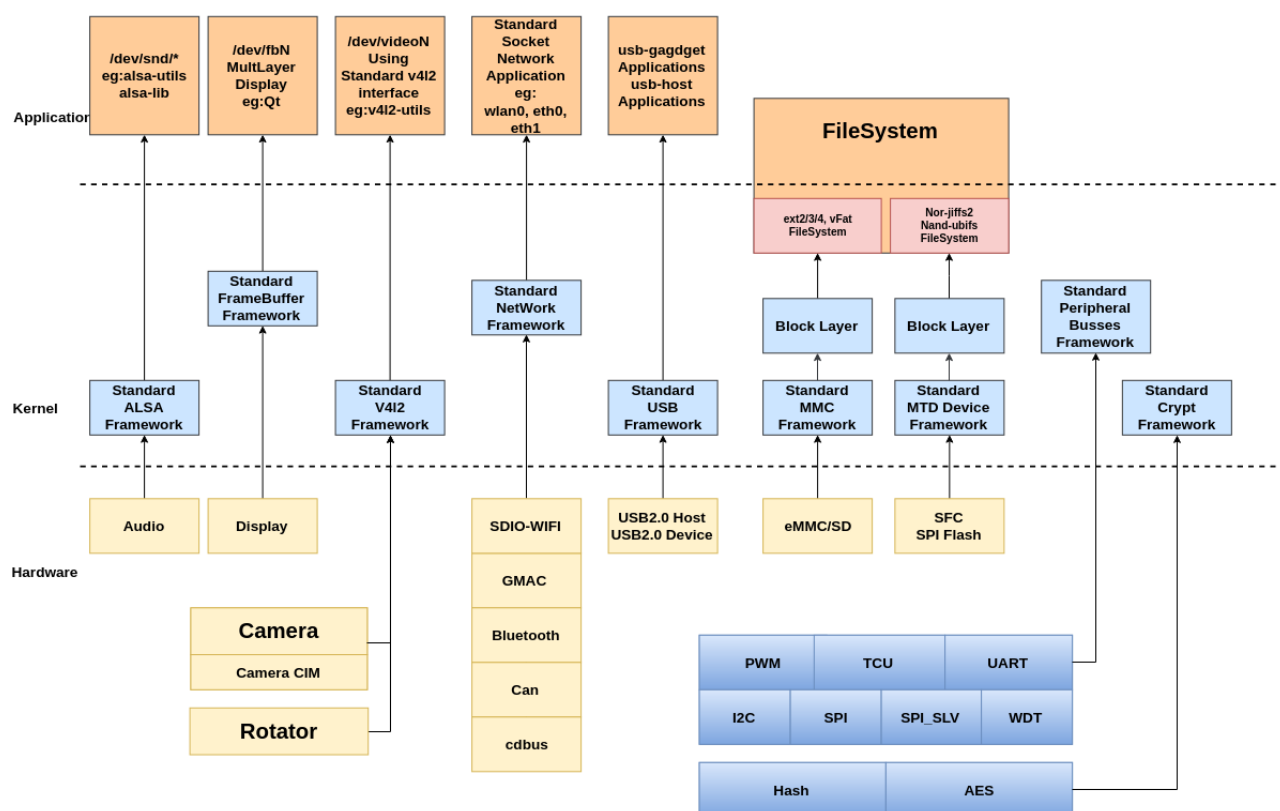


Chart 1-1 Kernel driver adaptation structure diagram

1.2 Kernel Development process

The development of the kernel can be divided into two types of development, one is based on the SDK development environment provided by Junzheng, the other is based on the source code directory development process.

1.2.1 Junzheng SDK Kernel Development Process (recommended)

Typically, you download the full SDK package, build the basic development environment first, and compile the whole thing once (note: this is important). This process performs the basic configuration of the kernel. The flow chart is as follows:

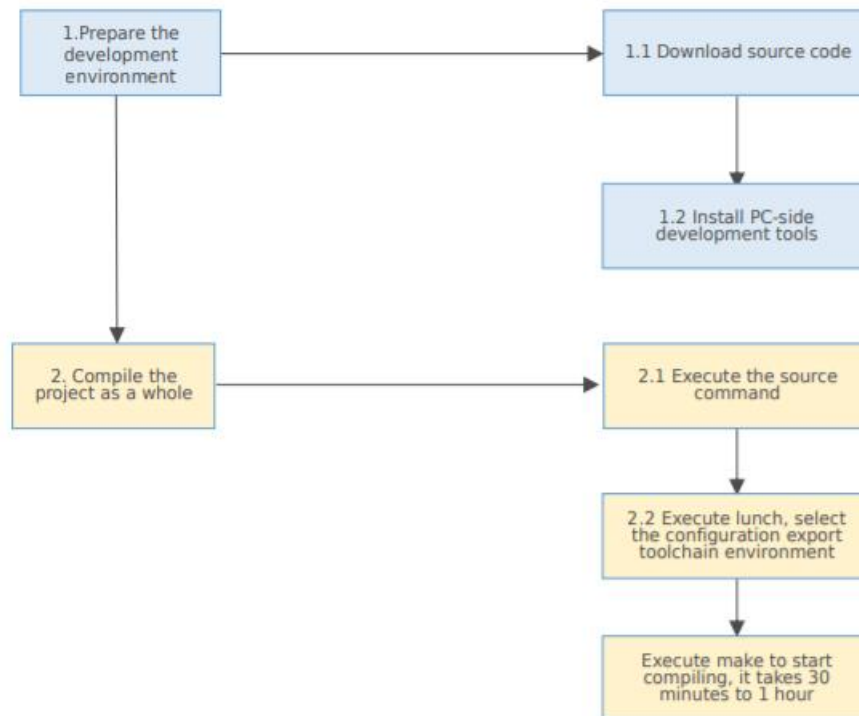


Chart 1-2 Kernel development flow chart

After ensuring that the above tasks have been performed, if the kernel needs to be reconfigured, or the kernel needs to be recompiled if the kernel is modified, the following commands can be executed at the top level of the SDK to configure.

```
$ make kernel-menuconfig # Configure the kernel
$ make kernel             # Compile the kernel
```

After the command is executed, a file is generated:

out/product/halley6.v10_nand_x.x.x-eng/obj/kernel-intermediate/arch/mips/boot/ulmage, This file is the final kernel image that needs to be burned.

Based on the development process of Ingenic SDK, source code and compilation generated intermediate files are separated, the intermediate files will be generated in out/product/halley6.v10_nand_x.x.x-eng/obj/kernel-intermediate/ directory, including the kernel .config file.

Precautions:

1. To use the development process of Ingenic SDK, you need to ensure that the

source code directory is not contaminated, that is, you cannot use traditional make xx_defconfig in the source code directory to configure and compile the kernel, otherwise an error will be reported. At this time, it can be executed in the kernel source directory.

```
$ make distclean
$ make mrproper
```

2. All commands that use make kernel-xxx actually execute:

```
$ make -C out/product/productName/obj/kernel-intermediate xxx
```

In the actual development process, you can try to replace xxx in make kernel-xxx with the make command supported by the kernel.

1.2.2 Development process based on kernel source code

The development process based on the kernel source code is that the modified and compiled object files are generated in the kernel directory. All commands of the development process are executed in the kernel directory.

```
$ cd kernel
$ make board_cfg_defconfig
$ make uImage

#When you need to modify the kernel reconfiguration, use:
$ make menuconfig
$ make uImage
# Is the generated target file
$ ls arch/mips/boot/uImage
```

After executing the above command, the arch/mips/boot/uImage file will be generated in the kernel source directory, which is the final burned image. The board_cfg can be found in the arch/mips/configs/ directory.

1.3 Kernel default configurations

All kernel configuration default configuration files for the development board are stored in the arch/mips/configs/ directory.

The configuration file included in Halley6 is described as follows:

Configuration files	Descriptions
halley6_v10_linux_defconfig	Configuration for primary storage to start for NAND/NOR

Table 1 Halley6 kernel-4.4.94 configuration table

Configuration file	Instructions
halley6_v10_linux_defconfig	Used for configuration where the primary storage is nand/nor startup and supports halley6_v10 board level configuration
halley6_v20_linux_defconfig	It is used for configuration where the primary storage is nand/nor startup and supports halley6_v20 board level configuration

Table 1 Halley6 kernel-5.10 configuration table

1.4 Device Tree Configuration

Device tree is used to describe chip and development board-level resources. There are two main ways to use it. One way is directly compiled with the kernel, and the other way is compiled separately and stored separately, and u-boot passes parameters to inform the location of dtb storage.

By default, the kernel is directly compiled, and you can refer to the following for specific differences.

1.4.1 Device Tree File Description

Regarding the configuration file of the device tree, the main storage path is the arch/mips/boot/dts/ingenic directory, and the composition of dts is generally the definition of soc and board.

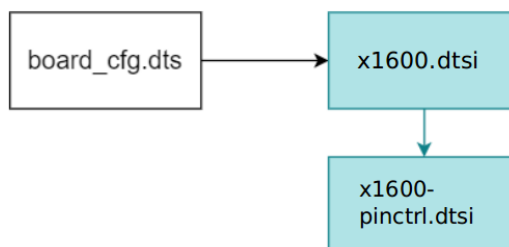


Chart 1-3 Schematic diagram of dts structure

The device tree files involved in the Halley6 platform are as follows:

dts files	File descriptions
halley6_v10.dts	Development board level configuration for Halley V1.x series. You need to modify this file to customize the development board configuration.
Halley6_cameras	The contents in this folder are for the Camera sub-board configurations supported by Halley6. If you need to add a new Camera configuration, you can refer to the contents in this folder to add it. If the official configuration is used, no modification is required.
—— RD_X1600_HALLEY6_MIPI_CAMERA.dtsi	
—— RD_X1600_HALLEY6_DVP_CAMERA.dtsi	

x1600.dtsi	Customers generally do not need to modify the basic configuration of the X1600 SOC chip.
x1600-pinctrl.dtsi	If X1600 GPIO function is configured, customers generally do not need to modify it..

1.4.2 Kernel Built-in device tree (default usage)

Refer to the default method used by the platform.

1.4.3 Compile the device tree separately

Compiling and using the device tree separately requires multiple support

- 1) Modify u-boot to support loading device tree.
- 2) Modify the kernel configuration to support separate compilation of device tree binary files.
- 3) Modify the burning tool to support burning device tree binary files.

1.4.3.1 Modify uboot configurations

1. Need to modify the configuration file, such as: **u-boot/include/configs/halley6.h**,

a) Add device tree support

```
/* Device Tree Configuration*/
#define CONFIG_OF_LIBFDT 1
#define IMAGE_ENABLE_OF_LIBFDT 1
#define CONFIG_LMB
```

b) Modify kernel startup parameters and startup commands

● Nand boot:

```
#define CONFIG_BOOTARGS BOOTARGS_COMMON "ip=off init=/linuxrc ubi.mtd=3 root=ubi0:rootfs
ubi.mtd=4 rootfstype=ubifs rw
#define CONFIG_BOOTCOMMAND "set uImage 0x80600000; set dtb 0x83000000; sfc nand read
0x100000 0x400000 ${uImage}; sfc nand read 0x900000 0x20000 ${dtb}; bootm ${uImage} -
${dtb}"
```

● Emmc/SD card boot:

```
#define CONFIG_BOOTARGS BOOTARGS_COMMON " rootfstype=ext4 root=/dev/mmcblk0p7
rootdelay=3 rw"
#define CONFIG_BOOTCOMMAND "set dtb 0x83000000; set uImage 0x80f00000; mmc dev 0; mmc
read ${uImage} 0x1800 0x2000; mmc read ${dtb} 0x5800 0x100; bootm ${uImage} - ${dtb}"
```

Note: Need to be modified according to the actual partition situation

Modify the kernel configuration:

At the top level of the SDK, execute `make kernel-menuconfig` to remove the **INGNEIC_BUILTIN_DTB** configuration:

The configuration description is as follows:

```
There is no help available for this option.
Symbol: INGENIC_BUILTIN_DTB [=n]
Type : boolean
Prompt: Ingenic Device Tree build into Kernel.
Location:
    -> Machine selection
    -> SOC Type Selection
Defined at arch/mips/xburst2/Kconfig:74
Depends on: MACH_XBURST2 [=n]
Selects: BUILTIN_DTB [=y]
```

After removing the option, the interface is as follows:

```
-- SOC type
SOC types (x1600 socs) --->
device tree select (x1600 halley6 v10) --->
(24) extal clock in MHz
[*] The physical space is larger than the virtual space
[ ] auto suspend test
```

Chart 1 4 Remove INGENIC_BUILTIN_DTB

1.4.3.2 Compile dtb files

Execute at the top level of SDK

```
$make kernel-dtbs
```

The file will be generated in `out/product/halley6/obj/kernel-intermediate/:`

```
arch/mips/boot/dts/ingenic/halley6_v10.dtb
```

This file is the final device tree binary file that needs to be burned.

1.4.3.3 Burn dtb files

For Nand primary storage and eMMC/SD primary storage, the burning tool configuration is different.

1. Burn Nand DTB files

Note: Burn the dtb file to the reserved partition of the device tree, and keep it consistent with the configuration of `bootcmd` and `bootargs`.

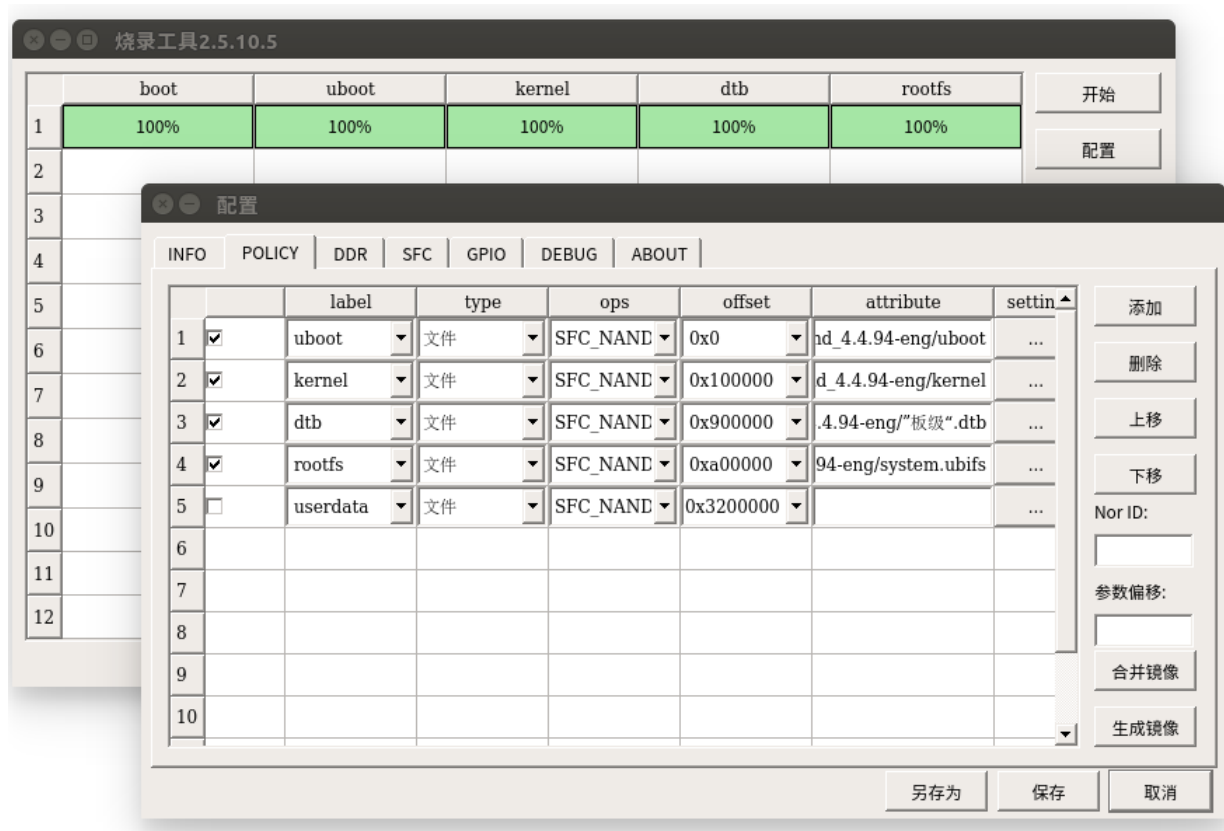


Chart 1 5 Nand DTB burning

2. Burn eMMC/SD DTB files

- dtb storage location: It is recommended to use the default address 0xb00000 (11MB), no other modification is needed, just burn it directly.
- If you need to specify the dtb partition yourself, you need to make the following modifications:
 - Halley6 is used as an example , you need to refer to the file u-boot/board/ingenic/"boardlevel" /partitions.tab.
 - The kernel needs to modify the number of partitions, which is the same as the partitions.tab configuration above.
 - Modify the location of the burning tool dtb

Modify u-boot/board/ingenic/"board name"/partitions.tab

```
property:
    disk_size = 4096m
    gpt_header_lba = 512
    custom_signature = 0

partition:
    #name      = start,      size, fstype
    xboot      = 0m,        3m,
    boot       = 3m,        8m, EMPTY
```

```

recovery = 12m, 16m, EMPTY
pretest  = 28m, 16m, EMPTY
reserved = 44m, 52m, EMPTY
misc     = 96m, 4m,  EMPTY
cache    = 100m, 100m, LINUX_FS
system   = 200m, 1800m, LINUX_FS
data     = 2000m, 2048m, LINUX_FS

```

#fstype could be: LINUX_FS, FAT_FS, EMPTY

Note: dtb is recommended to use address 0xb00000, which is the 11MB location of MMC

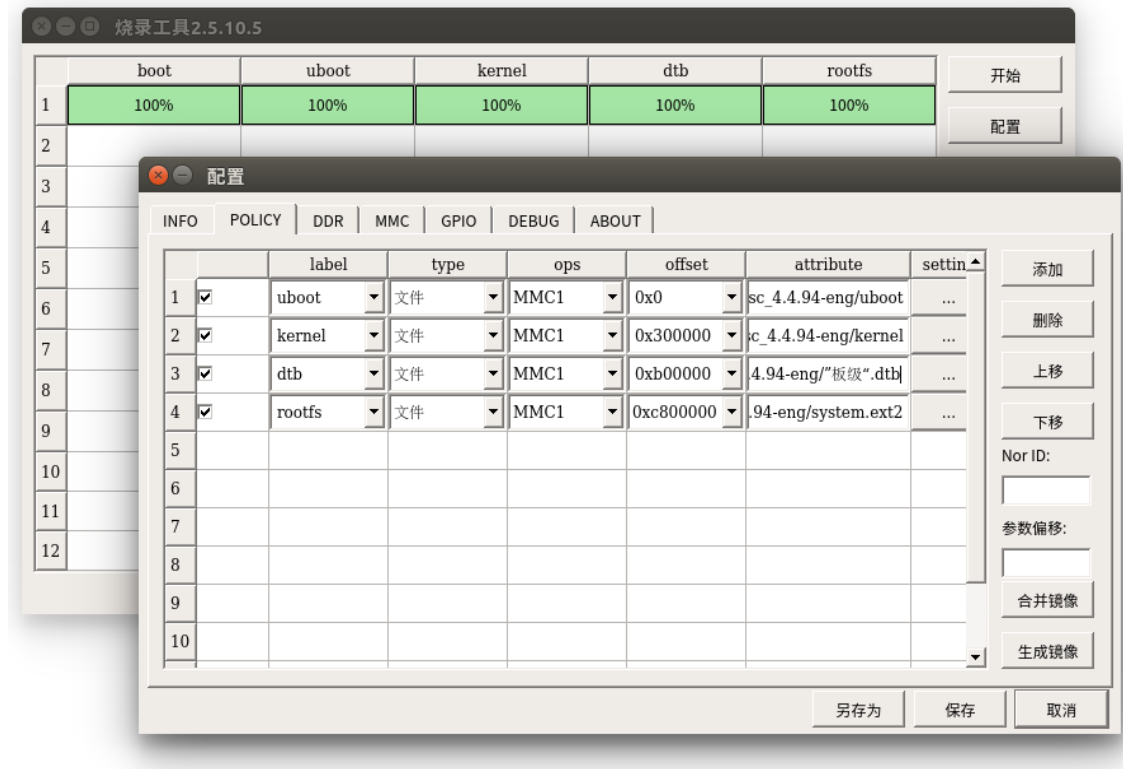


Chart 1-6 eMMC/SD DTB burning

1.4.4 Passing kernel parameters through the device tree

There are many ways to pass the kernel bootargs. Here we only introduce how to use the dts file to pass kernel parameters.

1.4.4.1 Modify the device tree

Define the chosen node, and the bootargs in the node are used as the cmdline passed to the kernel.

Example: arch/mips/boot/dts/ingenic/halley6_v10.dts

```

/ {
    compatible = "ingenic, halley6_v10", "ingenic, halley6_v10";

```

```

/*chosen {
    bootargs = "console=ttyS1,115200 mem=128M@0x0ip=off init=/linuxrc ubi.mtd=3
root=ubi0:rootfs ubi.mtd=4 rootfstype=ubifs
rw";
};*/

reserved-memory {
    #address-cells = <1>;
    #size-cells = <1>;
    ranges = <>;

    reserved_memory: reserved_mem@0x2000000{
        compatible = "shared-dma-pool";
        reg = <0x02000000 0x2000000>;
    };
};

};

```

1.4.4.2 Modify the kernel configurations

After the kernel selects the MIPS_CMDLINE_FROM_DTB configuration, it will parse the bootargs in the dts file and ignore the bootargs passed from u-boot.

Note: During the boot process, u-boot parses the chosen node in dtb. If it is not defined, u-boot will create the default chosen node.

```

Symbol: MIPS_CMDLINE_FROM_DTB [=y]
Type : boolean
Prompt: Dtb kernel arguments if available
Location:
-> Kernel type
-> Kernel command line type (<choice> [=y])
Defined at arch/mips/Kconfig:2858
Depends on: <choice> && USE_OF [=y]

```

Refer to the image below for the selected image

```

[ ] Enable seccomp to safely compute untrusted bytecode
-* Support for 032 binaries using 64-bit FP
Kernel appended dtb support (None) --->
Kernel command line type (Dtb kernel arguments if available) --->

```

Chart 1-7

2 CIM camera interface module

2.1 Module Function Description

CIM (camera interface module) accepts image signals sent by the front camera sensor, supports 8bit DVP, MIPI, BT656 data input, and supports YUV422, RGB888, RGB565, MONO(8) formats. Support the Snapshot function.

2.2 Driver source code location

Driver source code path:

```
Module_drivers/drivers/media/platform/ingenic-cim
├──ingenic_camera.c
├──ingenic_camera.h
├──Kconfig
├──Makefile
├──mipi_csi.c
└──mipi_csi.h
```

2.3 Device tree configurations

Location of the device tree:

```
Module_drivers/dts/x1600.dtsi
```

CIM controller description:

```
cim: cim@0x13060000 {
    compatible = "ingenic,x1600-cim";
    reg = <0x13060000 0x10000>;
    interrupt-parent = <&core_intc>;
    interrupts = <IRQ_CIM>;
    clocks = <&clock CLK_DIV_CIM>, <&clock CLK_GATE_CIM>, <&clock CLK_GATE_MIPI_CSI>;
    clock-names = "div_cim", "gate_cim", "gate_mipi";
    status = "disable";
};
```

2.3.1 Default configurations of device tree

```
reserved-memory {
    reserved_memory: reserved_mem@0x2000000 {
        compatible = "shared-dma-pool";
        reg = <0x02000000 0x2000000>;
    };
};

&cim{
    status = "okay";
    memory-region=<&reserved_memory>;
};
```

```
};
```

2.3.2 Device tree custom configurations

```
&cim{  
  
    status = "disable";  
    memory-region=<&reserved_memory>;  
  
};
```

In halley6_v10.dts, configure the reserved_memory as follows:

Note: The total memory of the x1600 is 32M, the total memory of the x1600e is 64M, and the reserved_memory can be set to 8M. You can also customize the reserved_memory size based on the camera sensor. The calculation method is as follows:

Bit width × output image width × output image height × number of descriptors.

```
reserved-memory {  
    reserved_memory: reserved_mem@0x18000000 {  
        compatible = "shared-dma-pool";  
        reg = <0x01800000 0x800000>;  
    };  
};
```

2.3.2.1 CIM controller configuration

In halley6_cameras/RD_X1600_HALLEY6_MIPI_CAMERA.dtsi, the data-lanes field is filled according to the number of data lanes of the connected mipi sensor, for example: to connect 2 lane mipi sensor and data-lanes, two parameters should be filled, data-lanes = <0 1>; to connect 4 lane mipi sensor and data-lanes, four parameters should be filled, data-lanes = <0 1 3 4>; the clk-lanes field is filled according to the number of clock lanes of the connected mipi sensor. For example, if a mipi sensor is connected to a clk lane, clk-lanes needs to be filled with one parameter. clk-lanes = <2>. configure the CIM controller as follows:

```
&cim {  
    status = "okay";  
    port {  
        cim_0: endpoint@0 {  
            remote-endpoint = <&sc031gs_ep0>;  
            data-lanes = <0 1>;  
            clk-lanes = <2>;  
        };  
    };  
};
```

In halley6_cameras/RD_X1600_HALLEY6_DVP_CAMERA.dtsi, configure the CIM controller as follows:

```
&cim {  
    status = "okay";  
    port {
```

```

        cim_0: endpoint@0 {
            remote-endpoint = <&sc031gs_ep0>;

bus-width = <8>;

            data-shift = <0>;
            bus-type = <5>;

            hsync-active = <1>;
            vsync-active = <0>;
            data-active = <1>;
            pclk-sample = <1>;

        };
    };
};

```

2.3.2.2 camera sensor configurations

In halley6_cameras/RD_X1600_HALLEY6_MIPI_CAMERA.dtsi, customize the camera sensor as follows:

```

&i2c0 {
    status = "okay";
    clock-frequency = <100000>;
    timeout = <1000>;
    pinctrl-names = "default";
    pinctrl-0 = <&i2c0_pa>;

    sc031gs_0:sc031gs@0x30 {
        status = "okay";
        compatible = "sc031gs";
        reg = <0x30>;
        pinctrl-names = "default";
        pinctrl-0 = <&cim_mipi_mclk_pc>;

        resetb-gpios = <&gpa 30 GPIO_ACTIVE_LOW INGENIC_GPIO_NOBIAS>;
        vcc-en-gpios = <&gpb 12 GPIO_ACTIVE_HIGH INGENIC_GPIO_NOBIAS>;

        port {
            sc031gs_ep0:endpoint {
                remote-endpoint = <&cim_0>;
            };
        };
    };
};

```

In halley6_cameras/RD_X1600_HALLEY6_DVP_CAMERA.dtsi, customize the camera sensor as follows:

```

&i2c0 {
    status = "okay";
    clock-frequency = <100000>;
    timeout = <1000>;
    pinctrl-names = "default";
    pinctrl-0 = <&i2c0_pa>;

    sc031gs_0:sc031gs@0x30 {
        status = "okay";
        compatible = "sc031gs";
        reg = <0x30>;
        pinctrl-names = "default";
        pinctrl-0 = <&cim_mipi_mclk_pc>, <&cim_pa>;

        resetb-gpios = <&gpa 30 GPIO_ACTIVE_HIGH INGENIC_GPIO_NOBIAS>;
        vcc-en-gpios = <&gpb 12 GPIO_ACTIVE_LOW INGENIC_GPIO_NOBIAS>;
        port {
            sc031gs_ep0:endpoint {
                remote-endpoint = <&cim_0>;
            };
        };
    };
};
};

```

2.4 Kernel compilation configurations

The kernel configuration VIDEO_INGENIC_CIM, the configuration description is as follows:

```

Symbol: VIDEO_INGENIC_CIM [=y]
Type : tristate
Prompt: Ingenic Soc Camera Driver for X2000 && M300 && X1600
Location:
    -> Ingenic device-drivers Configurations
        -> [CIM] Drivers
Defined at module_drivers/drivers/media/platform/ingenic-cim/Kconfig

```

2.4.1 Kernel default compilation configurations

By default, device tree compilation generates CIM controller devices.

2.4.2 Kernel custom compilation configurations

2.4.2.1 CIM Controller configurations

Start the CIM controller driver. The configuration screen is as follows:

```

Ingenic device-drivers Configurations  —>
    [CIM] Drivers  —>

```

```

<*>   Ingenic Soc Camera Driver for X2000 && M300 && X1600
[ ]    Sensor support snapshot function
[ ]    panda_camera_board  ——
[*]    halley6_camera_board  ——>
        [ ]    halley6 camera driver for RD_X1600_HALLEY6_MIPI_CAMERA
        [*]    halley6 camera driver for RD_X1600_HALLEY6_DVP_CAMERA
[Sensors] Camera Sensors  ——>
        *** ingenic-isp camera sensor drivers ***
        *** ingenic-isp-v2 camera sensor drivers ***
        *** ingenic-cim sensor drivers ***
< > ar0144 support
< > ov2735b support
< > ov5640 camera DVP interface support
- *- sc031gs dvp interface support
< > sc031gs mipi interface support

```

When using the snapshot function, enable the Sensor support snapshot function and set the pulse width and delay time.

2.4.2.2 camera sensor configurations

If halley6_CAMERA_board is selected on the controller, sensor is selected by default. The following uses SC031GS MIPI as an example:

```

Ingenic device-drivers Configurations  ——>
[Sensors] Camera Sensors  ——>
        *** ingenic-isp camera sensor drivers ***
        *** ingenic-isp-v2 camera sensor drivers ***
        *** soc_camera sensor drivers ***
< > ar0144 support
< > ov2735b support
< > ov5640 camera DVP interface support
< > sc031gs dvp interface support
- *- sc031gs mipi interface support

```

When using the Snapshot function, set the sensor to sanpshot mode in the sensor driver.

2.5 Kernel difference

2.6 Device node generation

After the driver is loaded successfully, the following nodes are generated:

```
/dev/video11
```

Output node of the CIM.

2.7 Application instructions

2.7.1 cimutils

The source code location:

```
packages/example/App/cimutils
```

Command line and parameter indication:

```
cimutils -C -I dev/video11 -x 640 -y 480 -t grey -f test.raw
```

- -x specify image width;
- -y specify image height;
- -v specifying CIM controller;
- -t specify the image format;
- -f specify the image file name.

2.7.2 v4l2-ctl

The source code location

```
buildroot/dl/libv4l/v4l-utils-1.18.0.tar.bz2
```

Command line and parameter indication

```
v4l2-ctl -v width=640,height=480,pixelformat="GREY" --stream-mmap=3 --stream-to="cim-test.yuv" -d /dev/video11
```

- width specifies the width of the output image;
- heigh specifies the height of the output image;
- pixelformat specifies the output image format;
- --stream-mmap specifies the number of rotating buffers;
- --stream-to specifies the name of the stored image file;
- -d specifies the video node of the image output.

3 Display Controller

3.1 Module Function Description

- Characteristic of layer
Input format support RGB888, RGB565, RGB555;
- Display Features:
Support TFT, SLCD;

3.2 Driver source location

Driver source location:

```
module_drivers/drivers/video/fbdev/ingenic/fb_stage
```

3.3 Add Screen

(1) Create a file related to the screen in module_drivers/drivers/video/fbdev/ingenic/fb_stage/displays/

Take FW050 as an example

```
touch panel-fw050.c
```

(2) Add screen flags and dependencies in Kconfig , The path is as follows:

```
module_drivers/drivers/video/fbdev/ingenic/fb_stage/displays/Kconfig
```

Take FW050 as an example

```
config STAGE_FW050
    tristate lcd panel FW050"
    depends on FB_INGENIC_DISPLAYS_STAGE
    help
        lcd panel FW050, for ingenicfb drivers.
```

(3) Add a new compile file in Makefile, The path is as follows:

```
module_drivers/drivers/video/fbdev/ingenic/fb_stage/displays/Makefile
```

Take FW050 as an example

```
obj-$(CONFIG_STAGE_FW050) += panel-fw050.o
```

(4) After configuring the screen file according to the screen manual, select the newly added screen in the kernel compilation configuration.

(5) Precautions

- Ensure that all voltages of the lcd are normal
- Ensure the reset pin timing is normal
- Ensure that the clk Settings of the lcd are normal. Otherwise, the screen may not be bright
- Ensure the lcd hbp, fbp, vfb, vbp and other parameters are normal
- If there are initialization parameters, ensure that the initialization parameters are correct
- On the MIPI screen, ensure that the parameters on the MIPI screen and the parameters on the

MIPI interface are correct

3.4 Device Tree Configuration

Location of the device tree:

```
module_driver/dts/halley6_v10.dts
```

The corresponding screen must be included in the device tree

```
module_drivers/dts/halley6_lcd/RD_X1600_HALLEY6_RGB_SLCD_1V0.dtsi
```

DPU Controller Description:

```
dpu: dpu@0x13050000 {
    compatible = "ingenic,x1600-dpu";
    reg = <0x13050000 0x10000>;
    interrupt-parent = <&core_intc>;
    interrupts = <IRQ_LCD>;
    status = "disabled";
};
```

3.4.1 Device Tree Default Configuration

By default, the device tree compilation generates DPU controller devices.

3.4.1.1 DPU Controller Configuration

The configuration is as follows in RD_X1600_HALLEY6_RGB_SLCD_1V0.dtsi:

```
&dpu {
    status = "okay";
    port {
        dpu_out_ep: endpoint {
            remote-endpoint = <&panel_fw035_ep>;
        };
    };
};
```

3.4.1.2 Display Configuration

```
display-dbi {
    compatible = "simple-bus";
    #interrupt-cells = <1>;
    #address-cells = <1>;
    #size-cells = <1>;
    ranges = <>;
    panel_fw035 {
        compatible = "ingenic,fw035";
        status = "okay";
        pinctrl-names = "default";
    };
};
```



```
pinctrl-0 = <&smart_lcd_pa_8bit>;
ingenic,pwm-gpio = <&gpc 0 GPIO_ACTIVE_HIGH INGENIC_GPIO_NOBIAS>;*/
ingenic,cs-gpio = <&gpa 23 GPIO_ACTIVE_LOW INGENIC_GPIO_NOBIAS>;
ingenic,rd-gpio = <&gpa 16 GPIO_ACTIVE_HIGH INGENIC_GPIO_NOBIAS>;
ingenic,vdd-en-gpio = <&gpa 31 GPIO_ACTIVE_HIGH INGENIC_GPIO_NOBIAS>;
ingenic,rst-gpio = <&gpb 13 GPIO_ACTIVE_LOW INGENIC_GPIO_NOBIAS>;

port {
    panel_fw035_ep: endpoint {
        remote-endpoint = <&dpu_out_ep>;
    };
};

};

};
```

3.4.2 Device tree User-defined configuration

You can disable the dpu as required and disable the above nodes.

3.4.2.1 X1600 DPU Supported screen types

- smart lcd: dbi hardware interface, The LCD has its own ram, and the crack screen needs to have te capability
- tft lcd: dpi hardware interface, Without ram, you need to refresh the screen at a certain frame rate

3.4.2.2 Match other signal display

- The endpoint (port) in the DPU controller corresponds to the endpoint in the display configuration
- Configure compatible screens (*compatible*)
- Select the control pin (pinctrl-0) according to different types of screen, Different pin Settings in module_drivers/dts/ x1600-pinctrl.dtsi
- Configure other screen-related pins according to different screens, such as power supply pins and reset pins

3.5 Module kernel differences

3.6 Screen Configuration

```
struct lcd_panel {
    const char *name;

    unsigned int num_modes; /* Number of modes supported for display, fixed value 1 */
    struct fb_videomode *modes; /* Display mode */
    struct jzdsi_data *dsi_pdata; /* If the screen is mipi the dsi interface needs to be implemented */
    enum ingenic_lcd_type lcd_type; /* Screen type.If it is mipi tft lcd, LCD_TYPE_TFT needs to be configured. Other configurations are based on functions */

    unsigned int bpp; /* You don't need to fill */
    unsigned int width; /* The actual physical width of the screen (unit: mm) */
};
```

```

    unsigned int height; /* The actual physical height of the screen (unit: mm) */

    struct smart_config *smart_config; /* If the screen is smart lcd needs to be implemented */
    struct tft_config *tft_config; /* If the screen is tft lcd needs to be implemented */

    unsigned dither_enable:1; /* Turn on the dither function */
    struct {
        unsigned dither_red;
        unsigned dither_green;
        unsigned dither_blue;
    } dither;
};

struct fb_videomode {
    const char *name; /* optional */
    u32 refresh; /* Configure the frame rate. The driver calculates the pixclock based on the configuration parameters
and frame rate */
    u32 xres; /* Display valid width, fill in according to the screen manual */
    u32 yres; /* Display effective height, fill in according to the screen manual */
    u32 pixclock;
    u32 left_margin; /* For the tft screen, follow the manual. For the smart lcd screen, set the value to 0 */
    u32 right_margin; /* For the tft screen, follow the manual. For the smart lcd screen, set the value to 0 */
    u32 upper_margin; /* For the tft screen, follow the manual. For the smart lcd screen, set the value to 0 */
    u32 lower_margin; /* For the tft screen, follow the manual. For the smart lcd screen, set the value to 0 */
    u32 hsync_len; /* For the tft screen, follow the manual. For the smart lcd screen, set the value to 0 */
    u32 vsync_len; /* For the tft screen, follow the manual. For the smart lcd screen, set the value to 0 */
    u32 sync; /* When mipi dsi tft lcd needs to be assigned (FB_SYNC_HOR_HIGH_ACT &
FB_SYNC_VERT_HIGH_ACT), The other screens don't pay attention */
    u32 vmode; /* Default Value: FB_VMODE_NONINTERLACED */
    u32 flag;
};

```

3.6.1 smart lcd configuration

```

struct smart_config {
    unsigned int te_switch; /* smart lcd te function control */
    unsigned int te_mipi_switch; /* Set the mipi dsi smart lcd te function control */
    unsigned int te_md; /* 0: the te front edge is valid, 1: the TE back edge is valid */
    unsigned int te_dp; /* 0: indicates that the te low level is valid. 1: indicates that the TE high level is valid */
    unsigned int te_anti_jit; /* 0: The te signal is valid when maintaining 1 pixclk, 1: the te signal is valid when
maintaining 3 pixclk */
    unsigned int dc_md; /* 0: DC high level data, low level command, 1: DC high level command, low level data */
};

```

```

    unsigned int wr_md; /*0: sampling along the falling edge, 1: sampling along the rising edge */
    enum smart_lcd_type smart_type; /* smart slcd type, which supports 6800/8080, spi-3, and spi-4*/
    enum smart_lcd_format pix_fmt; /* Bus data format, support 565,666, etc */
    enum smart_lcd_dwidth dwidth; /* Data bus width */
    enum smart_lcd_cwidth cwidth; /* Command bus width */
    unsigned int bus_width;

    unsigned long write_gram_cmd; /* Command to be sent before data is sent. Default 0x2c*/
    struct smart_lcd_data_table *data_table; /* Configure the screen command table */
    unsigned int length_data_table; /* Configure the number of on-screen commands */
};

```

3.6.2 tft lcd configuration

```

struct tft_config {
    unsigned int pix_clk_inv; /*0:pixclk default output, 1: inverts pixelk */
    unsigned int de_dl; /*0:DE pin high output valid data, 1: low output valid data */
    unsigned int sync_dl; /*0:vsync and hsync pins output valid data at high level, 1: output valid data at low level */
    enum tft_lcd_color_even color_even; /* Even number of lines bus RGB order */
    enum tft_lcd_color_odd color_odd; /* Odd number of lines when the bus RGB order */
    enum tft_lcd_mode mode; /* Bus data format, support 888/666/565*/
};

```

3.6.3 PWM Backlight configuration

If pwm backlight is required, configure pwm in halley6_v10.dts:

```

&pwm {
    pinctrl-names = <default>;
    pinctrl-0 = <&pwm0_pc>;
    status = <okay>;
};

```

The currently supported screen pwm backlight device tree location:

```

module_drivers/dts/halley6_lcd

```

This section uses the fw035 screen as an example:

```

backlight {
    compatible = <fbwm-backlight>;
    pwms = <&pwm 0 1000000>;
    brightness-levels = <0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15>;
    default-brightness-level = <4>;
};

```

3.7 Kernel compilation Configuration

Configure the controller, display, and backlight in the kernel. The configuration is described as follows:

Controller Driver Configuration:

Symbol: FB_INGENIC_STAGE [=y]
Type : tristate
Prompt: Ingenic Framebuffer Driver for stage
Location:
 -> Ingenic device-drivers Configurations
 -> [LCD] Panel/Touchscreen Drivers
 -> Ingenic Framebuffer Driver (FB_INGENIC [=y])
Defined at module_drivers/drivers/video/fbdev/ingenic/fb_stage/Kconfig

Display Configuration

Symbol: STAGE_FW035 [=y]
Type : tristate
Prompt: lcd panel FW035
Location:
 -> Ingenic device-drivers Configurations
 -> [LCD] Panel/Touchscreen Drivers
 -> Ingenic Framebuffer Driver (FB_INGENIC [=y])
 -> Ingenic Framebuffer Driver for stage (FB_INGENIC_STAGE [=y])
 -> Supported lcd panels (FB_INGENIC_DISPLAYS_STAGE [=y])
Defined at module_drivers/drivers/video/fbdev/ingenic/fb_stage/displays/Kconfig

PWM Backlight driver configuration:

Symbol: PWM_INGENIC_V3 [=y]
Type : boolean
Prompt: Ingenic V3 PWM support
Location:
 -> Ingenic device-drivers Configurations
 -> [PWM] drivers
Defined at module_drivers/drivers/pwm/Kconfig

3.8 Device Node Generation

The following nodes are generated after the driver is successfully loaded:

/dev/fb0

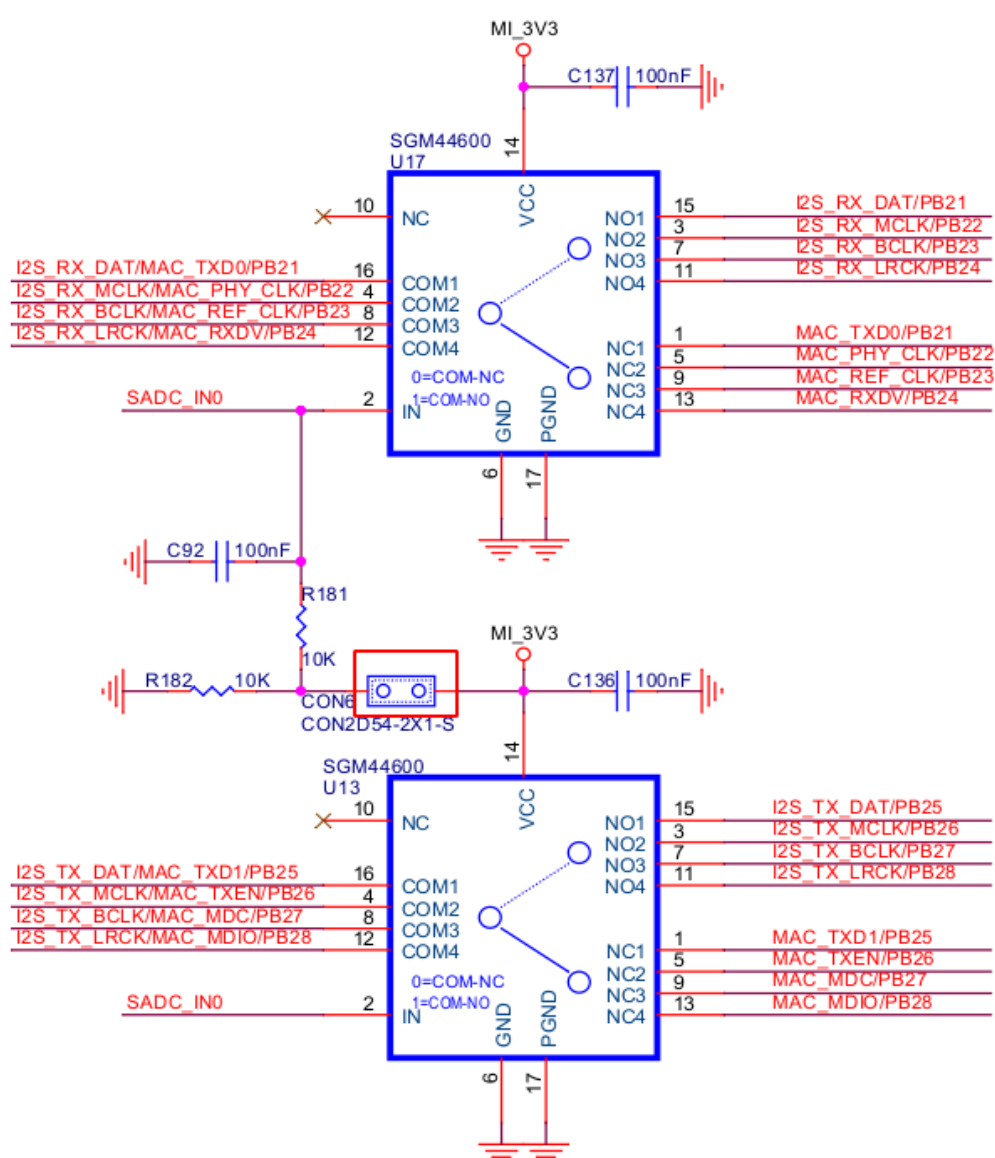
4 Audio subsystem

4.1 Module function introduction

AIC supports the I2S or IIS protocol for transferring digitized audio between the processor's system memory (external I2S codec).

- Supports an independent clock and a shared clock
- Support i2s, left justified protocol
- Support master and slave mode

4.2 Hardware Description



Since the x1600's i2s and mac share a common set of GPIOs, you can only choose whether to use the record and playback function or the Ethernet function when using it. The

hardware design also requires shorting CON6 to select which function to use, if you use the record and playback function you need to short CON6, if you use the Ethernet function you don't need to short CON6.

4.3 Driver source code location

Audio controller source code location:

sound/soc/ingenic/as-v1

Board-level source code location:

sound/soc/ingenic/board

External codec source location:

sound/soc/ingenic/ecodec/

4.4 Device Tree Configuration

The location of the device tree

module_drivers/dts/x1600.dtsi

Device tree description:

```
aic: aic@0x10020000 {
    compatible = "ingenic,aic";
    reg = <0x10079000 0x100>;
    interrupt-parent = <&core_intc>;
    interrupts = <IRQ_AUDIO>;
    dmas = <&pdma INGENIC_DMA_TYPE(INGENIC_DMA_REQ_ASOC_AIC_TX)>,
          <&pdma INGENIC_DMA_TYPE(INGENIC_DMA_REQ_ASOC_AIC_RX)>;
    dma-names = "tx", "rx";
    i2s: i2s {
        compatible = "ingenic,i2s";
        status = "ok";
    };
    i2s_tloop: i2s_tloop {
        compatible = "ingenic,i2s-tloop";
        dmas = <&pdma INGENIC_DMA_TYPE(INGENIC_DMA_REQ_ASOC_AIC_LOOP)>;
        dma-names = "rx";
        status = "ok";
    };
};
```

4.4.1 Device tree default configuration

The device tree is compiled by default without generating audio devices and is configured under module_drivers/dts/halley6_v10.dts.

```
&aic {
    status = "disable";
    pinctrl-names = "default";
    pinctrl-0 = <&aic_pb>;
};

dump_pcm_codec:dump_pcm_codec{
    compatible = "ingenic,pcm-dump-codec";
    status = "disable";
};

sound_halley6_ecdc {
    status = "disable";
    compatible = "ingenic,x1600-sound";
    ingenic,model = "halley6";
    ingenic,dai-link = "i2s-ecodec", "i2s-tloop";
    ingenic,stream = "i2s-ecodec", "i2s-tloop";
    ingenic,cpu-dai = <&i2s>, <&i2s_tloop>;
    ingenic,platform = <&aic>, <&i2s_tloop>;
    ingenic,codec = <&dump_pcm_codec>, <&dump_pcm_codec>;
    ingenic,codec-dai = "pcm-dump", "pcm-dump";
    ingenic,en3v3-gpios = <&gpb 30 GPIO_ACTIVE_HIGH INGENIC_GPIO_NOBIAS>;
};
```

4.4.2 Device Tree Custom Configuration

Users can turn on the audio device according to their actual needs.

4.5 Kernel compilation configuration

The kernel configures SND_ASOC_INGENIC_AIC, the configuration description is as follows:

```
Symbol: SND_ASOC_INGENIC [=y]
Type : tristate
Prompt: [AUDIO] ASoC support for Ingenic
Location:
    -> Ingenic device-drivers Configurations
Defined at module_drivers/sound/soc/ingenic/Kconfig:1
Depends on: MACH_XBURST [=y] || MACH_XBURST2 [=n]
Selects: SND_SOC [=y] && SND [=y] && SOUND [=y]
```

4.5.1 Kernel default compilation configuration

The kernel configures the audio driver by default, and the configuration interface is as

follows:

```
Ingenic device-drivers Configurations --->
  <*> [AUDIO] ASoC support for Ingenic --->
    [ ] enable ingenic debug message
    [ ] enable ingenic verbose debug message
    [ ] enable dmec and amec sync
    Audio Version: (AudioSystem Version 1 For Ingenic SOC) --->
    select aic clock mode (aic use independent clock mode) --->
    select audio mono channel (select audio mono channel right) ---
>

    select aic clock direction (aic as master mode) --->
    select aic protocol mode (aic use i2s protocol) --->
    Ingenic Board Type Select --->
    ingenic external codec Type select --->
```

4.5.2 Kernel Custom Compilation Configuration

Users can remove the configuration of this driver according to actual needs.

4.6 Device Node Generation

After the driver is loaded successfully, the following nodes are generated:

```
/dev/snd/controlC0
/dev/snd/pcmC0D0p
/dev/snd/pcmC0D0c
/dev/snd/pcmC0D1c
/dev/snd/timer
```

pcmC0D0p indicates the playback device and pcmC0D*c indicates the recording device.

4.7 Add new codec driver

When you need to add a new codec driver, you can write your own driver code by referring to the driver under "module_drivers/sound/soc/ingenic/ecodec". When using the new codec driver code, you need to modify the following contents of the dts.

```
sound_halley6_ecdc {
    status = "disable";
    compatible = "ingenic,x1600-sound";
    ingenic,model = "halley6";
    ingenic,dai-link = "i2s-ecodec", "i2s-tloop";
    ingenic,stream = "i2s-ecodec", "i2s-tloop";
    ingenic,cpu-dai = <&i2s>, <&i2s_tloop>;
    ingenic,platform = <&aic>, <&i2s_tloop>;
    ingenic,codec = <&dump_pcm_codec>, <&dump_pcm_codec>;
```



```
ingenic,codec-dai = "pcm-dump", "pcm-dump";
ingenic,en3v3-gpios = <&gpb 30 GPIO_ACTIVE_HIGH
INGENIC_GPIO_NOBIAS>;
};
```

The user needs to modify the contents of "ingenic, codec" and "ingenic, codec-dai".

4.8 Application Instructions

4.8.1 Recording and playback method

arecord command to record, aplay command to play

```
arecord -D hw:0,0 -c 2 -f S16_LE -r 16000 -d 5 baic0_16000-32-1.wav
aplay -D hw:0,0 baic0_16000-32-1.wav
```

4.8.2 Introduction of asound.conf

The asound.conf configuration file, which is the default configuration file for alsa-lib, is located in /etc/ and can be used to configure some additional features of the alsa library. This file is not necessary for the alsa library to run, but the alsa library can run normally without it. asound.conf allows more advanced control over sound cards or devices, providing access to the pcm plug-in method in alsa-lib, allowing you to do more complex control, such as combining sound cards into one or more sound cards to access multiple I/Os.

In ALSA, the PCM plugin extends the functionality and features of PCM devices. The plug-in can automatically handle tasks such as: naming devices, sample rate conversion, sample copying between channels, writing to files, connecting sound cards/devices for multiple inputs/outputs (without simultaneous sampling), using multi-channel sound cards/devices, etc.

4.8.2.1 hw plugin

This plugin communicates directly with the ALSA kernel driver, it is a raw communication without any conversion.

This plugin allows to perform renaming operations on pcm devices.

For example:

```
arecord -l
**** List of CAPTURE Hardware Devices ****
card 0: halley6 [halley6], device 0: i2s-ecodec dump_pcm_codec-0 []
  Subdevices: 1/1
  Subdevice #0: subdevice #0
card 0: halley6 [halley6], device 1: i2s-tloop dump_pcm_codec-1 []
  Subdevices: 1/1
  Subdevice #0: subdevice #0
```

```
# aplay -l
**** List of PLAYBACK Hardware Devices ****
card 0: halley6 [halley6], device 0: i2s-ecodec dump_pcm_codec-0 []
  Subdevices: 1/1
  Subdevice #0: subdevice #0
```

In the use of arecord or aplay for recording and playback, when specifying the device, generally use the "hw: 0, 0" method to specify the device used for recording and playback, which is not intuitive enough to use, you can define an alias for the device through the hw plug-in. For example.

```
pcm.amic {
    type hw
    card 0
    device 0
}
```

This example renames "hw: 0, 0" and can be used in the following way when recording and playing back audio.

```
arecord -D amic-c 2 -f S16_LE -r 16000 -d 5 baic0_16000-32-1.wav
aplay -D amic baic0_16000-32-1.wav
```

4.8.2.2 slave plugin

Slave plugin can be specified directly with a string or can be defined by entering them within a composite configuration node. Some limits (such as static rate or number of channels) can also be specified.

For example:

```
pcm_slave.slave_rate48000Hz {
    pcm "hw:0,0"
    rate 48000
}

pcm.rate48000Hz {
    type plug
    slave slave_rate48000Hz
}
```

or

```
pcm.rate48000Hz {
    type plug
    slave {
        pcm "hw:0,0"
        rate 48000
    }
}
```

```
}  
}
```

The above example can be understood as follows: a virtual pcm device named rate48000Hz is defined, the device actually uses the "hw:0,0" pcm device and is set to support only 48000Hz sample rate. In the use of rate48000Hz the pcm device recording and playback, hardware only support 48000 sample rate, you can also set a fixed number of format bits, wide channels, etc..

4.8.2.3 rate plugin

The plugin converts the rate. The input and output formats must be linear and are commonly used to play audio files where the hardware does not support sample rate data bit widths.

Example:

```
pcm.rate_16000 {  
    type rate  
    slave {  
        pcm "hw:0,0"  
        rate 16000  
        format S8  
    }  
}
```

The plugin supports conversion of sample rate and sample format bit width, not channel conversion.

4.8.2.4 plug plugin

The plugin can convert channels, rates and formats on request, and has more channel conversions than the rate plugin.

Example:

```
pcm.rate_48000 {  
    type plug  
    slave {  
        pcm "hw:0,0"  
        rate 16000  
        format S8  
        channels 1  
    }  
}
```

4.8.2.5 Soft Volume

This plugin is applied to software volume adjustment.

Example:

```
pcm.amic_record {  
    type softvol  
  
    slave {  
        pcm "hw:0,0"  
    }  
  
    control {  
        name "amic volume"  
    }  
  
    min_dB -51.0  
    max_dB 30.0  
  
}
```

The corresponding control will be generated only when the softvol plugin is played for the first time, i.e. you need to execute "arecord -D amic_record" (as defined above) once before you can adjust the volume through the amixer settings.

4.8.3 Usage of tloop

The tloop is the playback function, which is a function to directly pick up the playback data, the specific operation is as follows.

First, you need to play and record normally, and then play back the data.

```
aplay -D i2s_play 16000-16-1.wav &  
arecord -D i2s_record -f S16_LE -r 16000 -c 1 xxxxx.wav &  
arecord -D i2s_tloop -f S16_LE -r 16000 -c 1 xxxxx.wav &
```

Cautions.

1. Normal recording and playback recording must use mono.
2. Normal recording and playback recording must use the same sampling rate and sampling bits.

4.8.4 FAQ

4.8.4.1 i2s rx related clocks are available, tx related clocks are not

Debugging ideas:

Check if all the aic clock registers are set correctly?

4.8.4.2 Mono sound is small

Debugging ideas:

1. checked the software problem, recorded the left and right channel audio data

separately.

2. Checked the hardware problem, measuring the audio signal picked up from the microphone or pickup.

4.8.4.3 Audio too fast or too slow

1. When the clock frequency of lrcclk is greater than the sample rate of the audio file to be played, the sound will become faster
2. When the clock frequency of lrcclk is less than the sample rate of the audio file, the sound will be slowed down.

4.8.4.4 underrun/overflow questions

underrun: generally appear in the playback, when the upper layer of data is not sent to the driver buffer in a timely manner, resulting in no data to read the driver, that is, there will be an underrun error

overflow: generally occurs in the recording, when the upper layer does not take the data in the driver buffer in time, resulting in the driver buffer no space to write the situation, that is, overflow error

4.8.4.5 pcm_read:2145: read error: Input/output error

The root cause of this problem is that the data written by aplay to the ring buffer is not taken away, or arecord can't get the data when it is taken from the buffer, debugging ideas.

1. check whether the corresponding gpio function is right
2. Check whether there is a relevant clock
3. Check whether there is a problem with dma-related operations

4.8.4.6 Unable to install hw params

The hardware supports this audio parameter, but the lack of judgment of this parameter when the driver configures the audio parameter causes an error to be returned.

5 NEMC EXTERNAL STORAGE CONTROLLER INTERFACE

5.1 Module function introductions

It divides the off-chip memory space and outputs control signals conforming to various static memory and bus interface specifications. Static memory, such as devices such as NOR flash, can be connected to the processor.

5.2 Driver source code location

Driver source code location

```
module_drivers/drivers/char/ingenic_nemc.c
```

5.3 Device Tree Configurations

The path of the device tree:

```
module_drivers/dts/x1600dtsi
```

nemc controller definition:

```
nemc: nemc@0x13410000{
    compatible = "ingenic,x1600-nemc";
    reg = <0x13410000 0x1000>;
    status = "disable";
    ranges = <1 0 0x1b000000 0x1000000
              2 0 0x1a000000 0x1000000>;

    nand: nand@1{
        compatible = "ingenic,x1600-sfc";
        reg = <1 0 0x1000000>;
        ingenic,nemc-bus-width = <16>;
        ingenic,nemc-tCH = <10>;
        ingenic,nemc-tAW = <5>;
        ingenic,nemc-tBP = <10>;
        ingenic,nemc-tRAS = <15>;
        ingenic,nemc-tSTRV = <100>;
    };
};
```

5.3.1 Device tree default configurations

The default compilation of the device tree will generate NEMC devices.

5.3.2 Device Tree Custom Configurations

The user can turn off the nemc device according to actual needs, and configure the node as disabled in the board.dts.

```
&nemc {
    status = "disable";
    pinctrl-names = "default";
    pinctrl-0 = <&nemc_pa>, <&nemc_pb>;
};
```

5.4 Kernel compilation configurations

The kernel configuration nemc, the configuration description is as follows:

```
Symbol: INGENIC_NEMC [=n]
Type : boolean
Prompt: [NEMC] Ingenic Nemc Driver
Location:
-> Ingenic device-drivers Configurations
Defined at module_drivers/drivers/char/Kconfig
```

5.4.1 Kernel default compilation configurations

The kernel is configured with NEMC driver by default, and the configuration interface is as follows:

```
[*] [NEMC] Ingenic Nemc Driver
[ ] [PWM-AUDIO] Ingenic Pwm Audio Driver
```

5.5 Module kernel differences

no difference

5.6 Device node generation

After the driver is loaded successfully, the following nodes are generated:

```
# ls /dev/nemc
/dev/nemc
```

NOTE: At present, the nemc driver only simulates the nemc timing, and the specific read and write should be carried out according to the timing of the external storage device. The

driver should also be changed accordingly.



6 SFC Controller

6.1 Introduction to module functions

SFC is a SPI master device used to control SPI flash. Support CPU mode and DMA mode transmission.

- Support CDT command transmission mode.
- Support DMA descriptor chain data transmission mode.
- Support the status of hardware polling flash.
- Support standard, dual, Quad, Octal SPI and other transmission protocols.

6.2 Location of driver source code

Location of driver source code:

```
module_drivers/drivers/mtd/devices/ingenic_sfc_v2/
├── fmw.h
├── ingenic_sfc_common.c
├── ingenic_sfc_common.h
├── ingenic_sfc_drv.c
├── ingenic_sfc_drv.h
├── ingenic_sfc_nand.c
├── ingenic_sfc_nor.c
├── ingenic_sfc_ops.c
├── Makefile
├── nand_device
│   ├── ato_nand.c
│   ├── dosilicon_nand.c
│   ├── fm_nand.c
│   ├── foresee_nand.c
│   ├── gd_nand.c
│   ├── issi_nand.c
│   ├── Makefile
│   ├── mxic_nand.c
│   ├── nand_common.c
│   ├── nand_common.h
│   ├── toshiba_nand.c
│   ├── winbond_nand.c
│   ├── xcsp_nand.c
│   ├── xtx_mid0b_nand.c
│   ├── xtx_mid2c_nand.c
│   ├── xtx_nand.c
│   ├── yhy_nand.c
│   └── zetta_nand.c
├── nor_device
│   ├── Makefile
│   ├── nor_device.c
│   └── nor_device.h
├── sfc_flash.h
├── sfc.h
├── spinand_cmd.h
├── spinand.h
├── spinor_cmd.h
└── spinor.h
```

6.3 Configuration of the device tree

Location of device tree file:

```
module_drivers/dts/x1600.dtsi
```

Description of the module in the device tree:

```
sfc: sfc@0x13440000 {
    compatible = "ingenic,x1600-sfc";
```

```
reg = <0x13440000 0x10000>;
interrupt-parent = <&core_intc>;
interrupts = <IRQ_SFC>;
};
```

6.3.1 Default configuration of the device tree

The default configuration compilation of the device tree will generate sfc devices

```
&sfc {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&sfc_pc>;
    ingenic,sfc-init-frequency = <200000000>;
    ingenic,sfc-max-frequency = <400000000>;
    ingenic,use_ofpart_info = /bits/ 8 <0>;
    ingenic,spiflash_param_offset = <0>;

    norflash@0 {
        partitions {
            compatible = "fixed-partitions";
            #address-cells = <1>;
            #size-cells = <1>;

            /* spi nor flash partition */
            uboot@0 {
                label = "uboot";
                reg = <0x00000000 0x40000>;
                /*read-only;*/
            };

            kernel@40000 {
                label = "kernel";
                reg = <0x40000 0x300000>;
            };

            rootfs@360000 {
                label = "rootfs";
                reg = <0x360000 0xca0000>;
            };
        };
    };

    nandflash@1 {
        partitions {
            compatible = "fixed-partitions";
            #address-cells = <1>;
            #size-cells = <1>;

            /* spi nand flash partition */
            partition@0 {
                label = "uboot";
                reg = <0x00000000 0x100000>;
                /*read-only;*/
            };

            partition@100000 {
                label = "kernel";
                reg = <0x100000 0x800000>;
            };

            partition@900000 {
                label = "rootfs";
                reg = <0x900000 0xf700000>;
            };
        };
    };
};
```

```
};
```

6.3.2 Customized configuration of the device tree

User can close the node according to actual requirements, or perform the following configurations:

Attribute Name	Explanation
ingenic, sfc-init-frequency	The clock frequency set by the sfc controller in the initialization phase is low because the timing parameters are not configured.
ingenic, sfc-max-frequency	To configure the maximum sfc frequency, 4 frequency divisions are required. The actual online clock is (sfc-max-frequency/4)MHz.
ingenic, use_ofpart_info	By default, the MTD partition is transferred from the burning tool. When the node is configured as 1, the device tree partition information is used.
ingenic, spiflash_param_offset	When using spi flash to store flash parameters and partition information, it is used to modify the offset address for storing parameters in spi flash. When the value is 0, the default offset is 0x5800.

6.4 Configuration of kernel compilation

Kernel Configuration INGENIC_SFC, configuration instructions are as follows:

```
Symbol: INGENIC_SFC [=y]
Type : tristate
Prompt: Ingenic series SFC driver
Location:
-> Ingenic device-drivers Configurations
(1) -> [SFC] (SPI Nand/Nor Flash) Drivers
Defined at module_drivers/drivers/mtd/devices/Kconfig:1
Depends on: MACH_XBURST [=y] || MACH_XBURST2 [=n]
```

6.4.1 Default kernel compilation configuration

By default, the kernel configures the SFC driver as follows:

```
Ingenic device-drivers Configurations -->
[SFC] (SPI Nand/Nor Flash) Drivers -->
```

NAND:

```
<*> Ingenic series SFC driver
Select Ingenic series SFC driver version (Use ingenic sfc driver version 2) -->
the SFC external memory (nor or nand) (Support ingenic sfc-nand) -->
[ ] ingenic SN and MAC read write support.
```

NOR:

```
<*> Ingenic series SFC driver
Select Ingenic series SFC driver version (Use ingenic sfc driver version 2) -->
the SFC external memory (nor or nand) (Support ingenic sfc-nor) -->
[ ] Use SPI Nor Flash params built in kernel (NEW) -->
```

6.4.2 Kernel customized compilation configuration

一、NAND

Reserve space for NAND for storing SN number, MAC address, License, etc.
Configurable reserve area is large or small, default single area size is 1MB.
This feature is not enabled by default.

```
Symbol: INGENIC_SFCNAND_FMW [=y]
Type : bool
Defined at module_drivers/drivers/mtd/devices/Kconfig:44
Prompt: ingenic SN and MAC read write support.
Depends on: INGENIC_SFC [=y] && MTD_INGENIC_SFC_NANDFLASH [=y]
Location:
    -> Ingenic device-drivers Configurations
        -> [SFC] (SPI Nand/Nor Flash) Drivers
            -> Ingenic series SFC driver (INGENIC_SFC [=y])
```

```
<*> Ingenic series SFC driver
    Select Ingenic series SFC driver version (Use ingenic sfc driver version 2)  -->
    the SFC external memory (nor or nand) (Support ingenic sfc-nand)  -->
[*] ingenic          SN          and          MAC          read          write
support.
(1) SN space size (MB) (NEW)
(1) MAC space size (MB) (NEW)
(1) LICENSE space size (MB) (NEW)
```

二、NOR:

Enable the NOR flash parameter included in the kernel, which takes precedence over the NOR flash parameter in the kernel.

(Selection is supported, but only one flash with the same ID)

This feature is not enabled by default.

```
Symbol: INGENIC_BUILTIN_PARAMS [=y]
Type : bool
Defined at module_drivers/drivers/mtd/devices/Kconfig:70
Prompt: Use SPI Nor Flash params built in kernel
Depends on: INGENIC_SFC [=y] && MTD_INGENIC_SFC_NORFLASH [=y]
Location:
    -> Ingenic device-drivers Configurations
        -> [SFC] (SPI Nand/Nor Flash) Drivers
            -> Ingenic series SFC driver (INGENIC_SFC [=y])
```

```
-- Use SPI Nor Flash params built in kernel
[ ] GD25Q127C 0xc84018 (NEW)
[ ] GD25Q256C 0xc84019 (NEW)
[ ] GD25S512MD 0xc84019 (NEW)
```

6.5 Generation of device nodes

The following nodes are generated after successful drive loading:

```
/dev/mtd*
```

MTD character device node

```
/dev/mtdblock*
```

MTD block device node

6.6 Application instructions

6.6.1 Flash read and write test

1. Execute script: StorageMedia-Test.sh

```
#!/StorageMedia-Test.sh
*****
* * Please input which module you want test .
*****
* Enter 1 : Read/Write test in the nand/nor .
* Enter 2 : Read/Write test in the sdcard .
* Enter 3 : Read/Write test in the ddr .
* Enter 4 : Read/Write test between nand/nor and sdcard .
* Enter 5 : Read/Write test between nand/nor and ddr .
* Enter 6 : Read/Write test between ddr and sdcard .
```

2. Input 1:

```
You select 1, nand/nor <-> nand/nor
*****
* * Please choice which Size you want copy .
*****
* Enter 1 : auto random Size
* Enter 2 : setting a fix Size
```

3. Input 1 or 2:

Option 1: Test data of random size (calculate the size by yourself according to the remaining space of flash)

Option 2: Test data of specified size (Kbyte)

4. The flash read/write test starts. The test lasts for a period of time. If there is no error printing and abnormal exit, it is normal.

6.7 Add a new flash parameter

6.7.1 Add NOR flash parameter

1、Add spi nor flash parameters through the cloner burning tool (recommended)

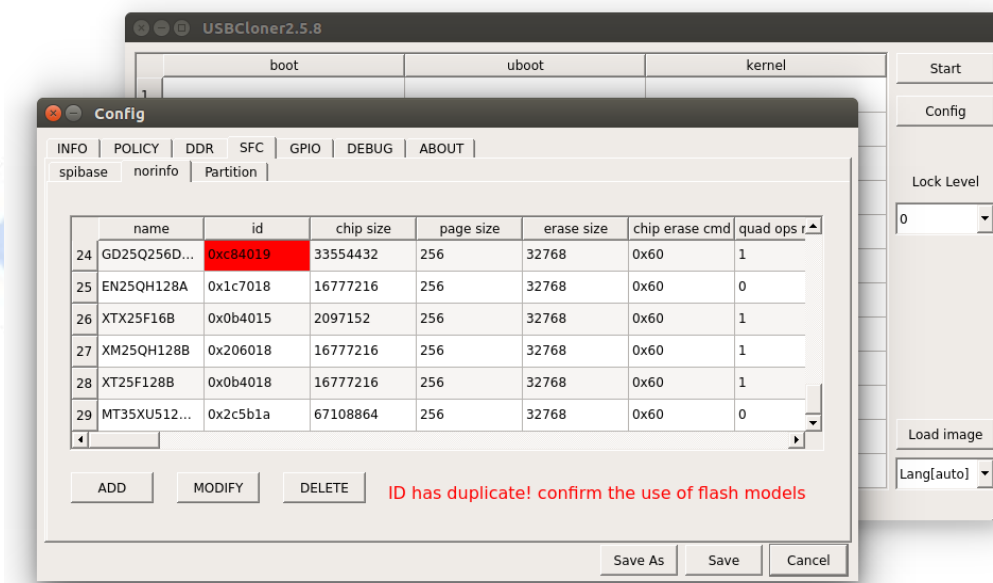
1. Open **Config** in the cloner, and under the INFO menu, select

“x2000_sfc_nor_lpddr3_linux.cfg” as the Board.

2. Under the SFC menu, select the secondary menu norinfo, and select ADD.

3. In the pop-up dialog box, fill in and save according to the parameters of spi nor flash.

Note: When the id number in the burning tool is red, it means that there are multiple flash parameters with the same ID, and the conflicting flash parameters need to be deleted manually



2、Add the NOR flash parameter to the kernel

If you need to add a customized flash, the process is as follows:

1. Add the base params, cdt params, and private params parameters according to the flash model. Reference is as follows:

module_drivers/drivers/mtd/devices/ingenic_sfc_v2/nor_device/nor_device.c*

2. Add config configuration as follows:

module_drivers/drivers/mtd/devices/Kconfig

6.7.2 Add NAND flash parameter

1、Add NAND flash vendor support

Location of NAND flash parameter configuration files of various manufacturers:

```
module_drivers/drivers/mtd/devices/ingenic_sfc_v2/nand_device/
```

If the configuration file of the vendor exists, skip this step:

If it does not exist, you need to refer to the parameter configuration file of other manufacturers to write your own configuration file. For example:

```
cp gd_nand.c myflash_nand.c && vi myflash_nand.c
```

The flash parameter configuration file mainly consists of (for example, gd_nand.c)

```
/* 1. Number of flash supported models */
define GD_DEVICES_NUM 7

/* 2. Basic parameters of flash */
static struct ingenic_sfc_nand_base_param gd_param[GD_DEVICES_NUM] = {
[0] = { ... },
}

/* 3. The flash ID is associated with the parameter */
static struct device_id_struct device_id[GD_DEVICES_NUM] = {
DEVICE_ID_STRUCT(0xD1, "GD5F1GQ4UB", &gd_param[0]),
...
}

/* 4. Flash gets the default cdt parameters and updates them according to the ID */
static cdt_params_t *gd_get_cdt_params(struct sfc_flash *flash, uint8_t device_id) {
...
}

/* 5. To use the general get feature interface, you need to define the interface for processing ecc */
static inline int deal_ecc_status(struct sfc_flash *flash, uint8_t device_id, uint8_t ecc_status) {
...
}

/* 6. Register flash parameters */
static int __init gd_nand_init(void) {
...
}
```

2、Add nand flash model support

Complete the following steps according to the flash manual

1. Add flash basic parameters

```
[6] = { .pagesize = 2 * 1024, ... },
```

2. Establish the association between flash and parameters

```
DEVICE_ID_STRUCT(0xA1, "GD5F1GQ4RF", &gd_param[6]),
```

3. Update the default parameters according to the ID

```
case 0xA1:
    gd_nand->cdt_params.standard_r.addr_nbyte = 3;
    gd_nand->cdt_params.quad_r.addr_nbyte = 3;
    break;
```

4. Add the processing of the corresponding model in the interface for processing ecc

```
switch(device_id) {
case 0xA1:
    ...
}
```

5. Add one to the total number of flash supported

```
#define GD_DEVICES_NUM 7
```

3、Compare the default parameters with the flash manual and update the parameters

Default command parameter file:

```
module_drivers/drivers/mtd/devices/ingenic_sfc_v2/nand_device/nand_common.h
```

Write parameter interface description:

Control flash related:

```
/*CMD_INFO(CMD, flash command, dummy bits, Address Length, transport protocol)*/
Read parameters of flash status
/*ST_INFO(ST, flash command, status bit offset, status bit mask, expected value of status bit,
transmission length, dummy bits)*/
```

For example:

Command	Byte	Byte 2	Byte 3	Byte 4	Byte 5	Byte N
Name	1					
Read From	6BH	dummy (2)	A15-A8	A7-A0	dummy (2)	(D7-D0)x4
Cache x 4						

```
/*dummy 8bit, address length 3, SFC manual for transmission protocol:TM_QI_QO_SPI*/
CMD_INFO(cdt_params.quad_r, SPINAND_CMD_RDCH_X4, 8, 2, TM_QI_QO_SPI);
```

Register	Addr	7	6	5	4	3	2	1	0
Status	COH	Reserved	ECCS2	ECCS1	ECCS0	P_FAIL	E_FAIL	WEL	OIP

```
/*OIP offset bit is 0, status bit mask is 0x1, expected value 0x0, transmission length 1 byte,
get feature dummy 0bit*/
ST_INFO(cdt_params.oip, SPINAND_CMD_GET_FEATURE, 0, 0x1, 0x0, 1, 0);
```

The default command parameters are as follows:

```
/*
 * cdt params
 */
#define CDT_PARAMS_INIT(cdt_params) { \
/* read to cache */ \
CMD_INFO(cdt_params.r_to_cache, SPINAND_CMD_PARD, 0, 3, TM_STD_SPI); \
/* standard read from cache */ \
CMD_INFO(cdt_params.standard_r, SPINAND_CMD_FRCH, 8, 2, TM_STD_SPI); \
/* quad read from cache */ \
CMD_INFO(cdt_params.quad_r, SPINAND_CMD_RDCH_X4, 8, 2, TM_QI_QO_SPI); \
/* standard write to cache */ \
CMD_INFO(cdt_params.standard_w_cache, SPINAND_CMD_PRO_LOAD, 0, 2, TM_STD_SPI); \
/* quad write to cache */ \
CMD_INFO(cdt_params.quad_w_cache, SPINAND_CMD_PRO_LOAD_X4, 0, 2, TM_QI_QO_SPI); \
/* write exec */ \
CMD_INFO(cdt_params.w_exec, SPINAND_CMD_PRO_EN, 0, 3, TM_STD_SPI); \
/* block erase */ \
CMD_INFO(cdt_params.b_erase, SPINAND_CMD_ERASE_128K, 0, 3, TM_STD_SPI); \
/* write enable */ \
CMD_INFO(cdt_params.w_en, SPINAND_CMD_WREN, 0, 0, TM_STD_SPI); \
\
/* get frature wait oip not busy */ \
ST_INFO(cdt_params.oip, SPINAND_CMD_GET_FEATURE, 0, 0x1, 0x0, 1, 0); \
}
```

Parameter update (such as gd_nand.c)

```
static cdt_params_t *gd_get_cdt_params(struct sfc_flash *flash, uint8_t device_id) {
switch(device_id) {
...
case 0xA1:
gd_nand->cdt_params.standard_r.addr_nbyte = 3; /* Update the address length to 3 byte */
gd_nand->cdt_params.quad_r.addr_nbyte = 3; /* Update the address length to 3 byte */
break;
...
}
```

4、Configure get_Feature interface

Default get_feature interface location:

```
module_drivers/drivers/mtd/devices/ingenic_sfc_v2/nand_device/nand_common.c
```

You can use the default get_feature interface. In special cases, you need to customize

8

the processing interface of ECC

```
static inline int deal_ecc_status(struct sfc_flash *flash, uint8_t device_id, uint8_t ecc_status)
{
    switch(device_id) {
        case 0xA1:
        case 0xB1 ... 0xB4:
            switch((ecc_status >> 4) & 0x7) {
                case 0x7:
                    ret = -EBADMSG;
                    break;
                case 0x6:
                    ret = 0x8;
                    break;
                case 0x5:
                    ret = 0x7;
                    break;
                default:
                    ret = 0;
            }
        }
        ...
        break;
    }
}
```

Custom get_feature interface:

1. Define your get_feature interface in the configuration file of the flash parameter
2. Register the self defined interface to the parameter

```
int32_t my_get_feature(struct sfc_flash *flash, uint8_t flag) {
    ...
}

static int __init gd_nand_init(void) {
    ...
    /* use private get feature interface, please define it in this document */
    //gd_nand->ops.get_feature = NULL;
    gd_nand->ops.get_feature = my_get_feature();
    ...
}
```

6.8 Note

1.Nand Flash ECC scheme

Use the hardware ECC of nand flash to establish BBT in memory based on MTD framework.

2.Current Supported Parameter Locations

NAND: Under Catalog

module_drivers/drivers/mtd/devices/ingenic_sfc_v2/nand_device/

NOR: See the interface of the burning tool

3.Kernel can use the u-boot built-in parameters. Please refer to the uboot documentation for details

You need to rely on u-boot. The kernel can use the default configuration.

4.SPI NOR Flash supporting multiple Dies

SPL: Multi Die switching is not supported temporarily.

Parameters: When flash parameters are burned, they must be stored in Die0.

Start: support reset flash switch Die0 in case of abnormal restart.

7 CPM Clock Power Reset Interface

7.1 Module function introduction

The CPM module is used to manage clocks and power. Including: clock control, pll control, power control, reset control.

The CPM driver is mainly responsible for the clock power management of the system and modules, including frequency multiplication, frequency division, clock switching, and clock source selection.

7.2 Drive position

The location of the driver source code:

```
module_drivers/drivers/clk/ingenic-v2
├── clk-bus.c
├── clk-bus.h
├── clk.c
├── clk-div.c
├── clk-div.h
├── clk.h
├── clk-m300.c
├── clk-pll.c(The file is not available in kernel 4.4.94)
├── clk-pll.h
├── clk-pll-v1.c
├── clk-pll-v1.h
├── clk-pll-v2.c
├── clk-pll-v2.h
├── clk-x1600.c
├── clk-x2000.c
├── clk-x2500.c
├── Kconfig
├── Makefile
├── power-gate.c
└── power-gate.h
```

7.3 Device Tree Configuration

The location of the device tree:

```
module_drivers/dts/x1600.dtsi
```

Device tree description:

```
clock: clock-controller@0x10000000 {
    compatible = "ingenic,x1600-clocks";
```

```

    reg = <0x10000000 0x100>;
    clocks = <&extclk>, <&rtcclock>;
    clock-names = "ext", "rtc_ext";
    #clock-cells = <1>;
    little-endian;
};

extclk: extclk {
    compatible = "ingenic, fixed-clock";
    clock-output-names = "ext";
    #clock-cells = <0>;
    clock-frequency = <24000000>;
};

rtcclock: rtcclock {
    compatible = "ingenic, fixed-clock";
    clock-output-names = "rtc_ext";
    #clock-cells = <0>;
    clock-frequency = <32768>;
};

```

7.3.1 Device tree default configuration

The default compilation of the device tree will generate clock, extclk, rtcclock devices, and configure the extclk frequency to 24Mhz:

```

extclk: extclk {
    clock-frequency = <24000000>;
};

```

7.4 Kernel compilation configuration

7.4.1 Kernel default compilation configuration

Using cpm to drive the kernel does not need to add special configuration, only depends on the configuration option CONFIG_SOC_X1600 of soc, the relevant code is in: module_drivers/drivers/clock/ingenic-v2/

7.4.2 Kernel custom compilation configuration

7.5 Device Node Generation

1. The generated device node is in:

```
/sys/devices/platform/apb/10000000.clock-controller
```

2. Mount debugfs to /mnt, the nodes under /mnt/clk can view the frequency and status of each clock.
 - a) cat clk_summary can view the parent-child relationship of all clocks, enable status, frequency, etc.:

```
# cat clk_summary
```

clock	enable_cnt	prepare_cnt	rate	accuracy	phase
ext_div_512	1	1	46875	0 0	
mux_wdt	1	1	46875	0 0	
rtc_ext	1	1	32768	0 0	
gate_rtc	1	1	32768	0 0	
ext	3	3	24000000	0 0	
gate_ost	1	1	24000000	0 0	
epll	2	2	100000000	0 0	
mux_cdbus	0	0	100000000	0 0	
div_cdbus	0	0	100000000	0 0	
mux_i2s0r	1	1	100000000	0 0	
div_i2s0r	1	1	50000000	0 0	
ce_i2s0r	1	1	50000000	0 0	
gate_i2s0r	0	0	50000000	0 0	
mux_i2s0t	1	1	100000000	0 0	
div_i2s0t	1	1	50000000	0 0	
ce_i2s0t	1	1	50000000	0 0	
gate_i2s0t	0	0	50000000	0 0	
mpll	4	4	1400000000	0 0	
mux_pwm	1	1	1400000000	0 0	
div_pwm	2	2	87500000	0 0	
gate_pwm	1	1	87500000	0 0	
mux_cim	0	0	1400000000	0 0	
div_cim	0	0	87500000	0 0	
gate_cim	0	0	87500000	0 0	
pd_mem_cim	0	0	87500000	0 0	
mux_ssi	0	0	1400000000	0 0	
div_ssi	0	0	87500000	0 0	
gate_ssi0	0	0	87500000	0 0	
pd_mem_ssi0	0	0	87500000	0 0	
mux_sfc	1	1	1400000000	0 0	
div_sfc	2	2	350000000	0 0	
gate_sfc	1	1	350000000	0 0	
pd_mem_sfc	0	0	350000000	0 0	

- b) Under /mnt/clk, each folder represents a clock, where mux_xxx controls clock source selection, div_xxx controls clock frequency division, and gate_xxx controls clock switch. pd_mem_xxx controls the power switch state of each module memory. The power switch status of the power_xxx control module.

```
# ls
apll                                gate_cim                            mux_ddr
ce_i2s0r                           gate_csi                            mux_i2s0r
ce_i2s0t                           gate_ddr                            mux_i2s0t
clk_dump                           gate_dtrng                          mux_lcd
clk_orphan_dump                   gate_efuse                          mux_macphy
clk_orphan_summary               gate_gmac0                          mux_msc0
clk_summary                       gate_hash                           mux_msc1
div_ahb0                          gate_i2c0                           mux_pwm
div_ahb2                          gate_i2c1                           mux_sfc
div_apb                           gate_i2s0r                          mux_ssi
div_can0                          gate_i2s0t                          mux_wdt
div_can1                          gate_intc                           pd_mem_aes
div_cdbus                         gate_lcd                            pd_mem_aic
div_cim                           gate_msc0                           pd_mem_cdbus
div_cpu                           gate_msc1                           pd_mem_cim
div_cpu_l2c_x1                   gate_nemc                           pd_mem_ddr_ch3
div_cpu_l2c_x2                   gate_ost                            pd_mem_ddr_ch5
div_ddr                           gate_otg                            pd_mem_ddr_ch6
div_i2s0r                         gate_pdma                           pd_mem_ddr_top
div_i2s0t                         gate_pwm                             pd_mem_dpu
div_l2c                           gate_rtc                            pd_mem_gmac
div_lcd                           gate_sadc                           pd_mem_msc0
div_macphy                       gate_sfc                            pd_mem_msc1
div_msc0                         gate_ssi0                           pd_mem_nemc
div_msc1                         gate_ssislv                         pd_mem_pdma
div_pwm                           gate_tcu                            pd_mem_pdma_sec
div_sfc                           gate_uart0                          pd_mem_pdma_slp
div_ssi                           gate_uart1                          pd_mem_pwm
epll                              gate_uart2                          pd_mem_sfc
ext                               gate_uart3                          pd_mem_ssi0
ext_div_512                       gate_usbphy                         pd_mem_ssislv
gate_aes                          mppll                               pd_mem_uart0
gate_ahb0                         mux_ahb0                           pd_mem_uart1
gate_apb0                         mux_ahb2                           pd_mem_uart2
gate_arb                          mux_can0                           pd_mem_uart3
gate_audio                        mux_can1                           pd_mem_usb
gate_can0                        mux_cdbus                          rtc_ext
gate_can1                        mux_cim                            sclka
gate_cdbus                       mux_cpu_l2c
```

c) Enter into a certain clock folder, cat clk_rate can view the clock frequency.

```
# cd div_msc0/
# ls
clk_accuracy                       clk_max_rate                       clk_phase
clk_duty_cycle                    clk_min_rate                       clk_prepare_count
clk_enable_count                  clk_notifier_count                 clk_protect_count
clk_flags                         clk_parent                         clk_rate
# cat clk_rate
700000000
```

7.6 Application Instructions

7.6.1 Use of the clock

The clock information is defined in the "module_drivers/drivers/clk/ingenic-v2" path of the SDK, and will be associated with the clock information in the form of a tree after the kernel starts. The user can use it in the following way (with cim as an example)

The cim node in "module_drivers/dts/x1600.dtsi" is described as follows:

```
cim: cim@0x13060000 {
    compatible = "ingenic,x1600-cim";
    reg = <0x13060000 0x10000>;
    interrupt-parent = <&core_intc>;
    interrupts = <IRQ_CIM>;
    clocks = <&clock CLK_DIV_CIM>, <&clock CLK_GATE_CIM>, <&clock
CLK_GATE_MIPI_CSI>;
    clock-names = "div_cim", "gate_cim", "gate_mipi";
    status = "disable";
};
```

The kernel keywords "clocks" and "clock-name" are used to get the clock information.

The probe function of cim will get the clock information of cim, which is used to control the clock of cim device.

```
/*get cim clk*/
pcdev->clk = of_clk_get(pdev->dev.of_node, 1);
if (pcdev->clk == NULL) {
    err = PTR_ERR(pcdev->clk);
    dev_err(&pdev->dev, "%s:can't get clk %s\n", __func__, "cim");
    goto err_clk_get_cim;
}
```

To modify the device clock frequency of cim you need to do the following.

1. Get the clock information of div_cim.

```
clk = clk_get(dev, "div_cim");
```

2. Set cim frequency to 24MHz (need to turn off cim's gate clock first).

```
clk_set_rate(clk, 24000000);
```

8 TCU timer unit

8.1 Module function introduction

The Timing Counting Unit (TCU) chip design includes 8 tcu channels with labels 0~7, each channel can have three clock input sources pwm (gpio0, gpio1), extal, any combination of which can be used for function use.

According to the spec description, it can be divided into 7 functional modes::

Normal mode: The counter counts on the rising or falling edge of the clock, or it can rise and fall while collecting the count.

Gate mode: When the gate is 0, the counter counts.

Quadrature Mode: The counter counts due to the quadrature input.

Direction mode: The counter is incremented or decremented by the input signal.

pos mode: due to rising or falling edge, the counter starts counting from 0.

Capture mode: counter counts cycles and outputs high time and cycle time.

Filter mode: This mode is used to remove undesired gpio inputs (e.g. burrs) from the user.

gpio store mode: This mode stores the count value when a change in gpio level is detected.

TCU CHANNELS	GPIO0	GPIO1
CH0	input0	input0
CH1	input1	input1
CH2	input2	input2
CH3	input3	input3
CH4	input4	input4
CH5	input5	input5
CH6	input6	input6
CH7	input7	input7

8.2 Drive position

The location of the driver source code

```
module_drivers/drivers/mfd/ingenic-tcu.c
```

8.3 Device Tree Configuration

The location of the device tree:

```
module_drivers/dts/x1600.dtsi
```

TCU controller description:

```
tcu: tcu@0x10002000 {
    compatible = "ingenic,tcu";
    reg = <0x10002000 0x1000>;
```

```
interrupt-parent = <&core_intc>;
interrupt-names = "default";
interrupts = <IRQ_TCU0 IRQ_TCU1>;
status = "ok";
};
```

8.3.1 Device Tree Default Configuration

The device tree is compiled by default without generating tcu devices

8.3.2 Device Tree Custom Configuration

Users can turn on the tcu device and configure this controller as ok according to their actual needs.

8.4 Kernel compilation configuration

The kernel configures MFD_INGENIC_TCU, the configuration instructions are as follows:

```
Symbol: MFD_INGENIC_TCU [=y]
Type : boolean
Prompt: Ingenic tcu driver
Location:
    -> Device Drivers
        -> Multifunction device drivers
Defined at drivers/mfd/Kconfig:417
Depends on: HAS_IOMEM [=y] && (MACH_XBURST [=y] || MACH_XBURST2 [=n])
Selects: MFD_CORE [=y] && GENERIC_IRQ_CHIP [=y]
```

8.4.1 Kernel default compilation configuration

The kernel configures the TCU driver by default, and the configuration interface is as follows:

```
[ ] Linux pmem allocator
[ ] [Efuse] Ingenic Efuse v2 Driver
< > Bluetooth power control driver for BCM-4345C5 module
[RANDOM] drivers --->
*** [MFD] Multi Function Device Drivers ***
< > [SADC] Support for the Ingenic SADC core
< > [SADC] Support for the Ingenic SADC AUX
[*] [TCU] Ingenic tcu driver
```

8.4.2 Kernel Custom Compilation Configuration

Users can remove the configuration of the driver according to actual needs.

8.5 Device Node Generation

After the driver is loaded successfully, the following nodes are generated:

```
cd sys/devices/platform/apb10002000.tcu/
```



```
disable enable power
driver modalias subsystem
driver_override of_node uevent
```

8.6 Application Instructions

- 1、The main use is disable and enable
- 2、There is a configured function in the driver that defaults to drivers/mfd/ingenic-tcu.c

```
/*This is a simple configuration test demo*/
static void ingenic_tcu_config_attr(int id,enum tcu_mode_sel mode_sel)
{
    int i = 0;
    g_tcu_chn[id].mode_sel = mode_sel;
    g_tcu_chn[id].irq_type = FULL_IRQ_MODE;
    .....
    for(i = 0; i < 20 ;i++)
        sws_pr_debug(&g_tcu_chn[id]);
}
```

Note: There are no specific application scenarios, and there are not too many system interfaces. The configuration of this function combined with spec can perform fast verification functions, and more system configuration interfaces will be reserved in the future.

Before using DIRECTION_MODE, POS_MODE, CAPTURE_MODE, and FILTER_MODE, please configure the pwm0 (gpio0) of the corresponding tcu channel.

When using the QUADRATURE_MODE mode, please configure the pwm0 (gpio0) and pwm1 (gpio1) of the corresponding tcu channel as quadrature signals.

- 3、The use process is to write the corresponding channel to enable the channel to start working.

```
#

mode_sel :
0:GENERAL_MODE 1:GATE_MODE 2:DIRECTION_MODE 3:QUADRATURE_MODE
4:POS_MODE 5:CAPTURE_MODE 6:FILTER_MODE

#####example#####
## echo channel_id mode_sel > enable
## echo channel_id > disable

channel: 00 disable
channel: 01 disable
channel: 02 disable
channel: 03 disable
```



```
channel: 04 disable
channel: 05 disable
channel: 06 disable
channel: 07 disable

# echo 0 0 > enable
[ 164.485676] mode_id = 0
...
...
[ 165.896941] -----count N0.20-----
[ 165.896941]
[ 165.912304] -stop----addr-b000201c-value-00000000-----
[ 165.918236] -mask----addr-b0002030-value-00ff80fe-----
[ 165.924181] -enable---addr-b0002010-value-00000001-----
[ 165.930120] -flag----addr-b0002020-value-00010000-----
[ 165.936048] -Control--addr-b000204c-value-00010004-----
[ 165.941990] -full----addr-b0002040-value-0000ffff-----
[ 165.947925] -half- -addr-b0002044-value-00005000-----
[ 165.953874] -TCNT----addr-b0002048-value-0000b5bf-----
[ 165.959811] -CAP reg_base-----value-00000000-----
[ 165.965399] -CAP_VAL register---value-00000000-----
```

4、Write the corresponding channel to disable to stop the counting work.

```
# echo 0 > disable
# cat disable
channel: 00 disable
channel: 01 disable
channel: 02 disable
channel: 03 disable
channel: 04 disable
channel: 05 disable
channel: 06 disable
channel: 07 disable
```

Description: tcu uses so many functional modes that there are not too many interfaces, but the internal logic has been written, just need to assign values to the `ingenic_tcu_config_attr` function according to the different modes in the spec. The macros of the class are also defined, which can be easily configured and used. Please modify the corresponding configuration for different usage scenarios.

9 INTC Interrupt Controller

9.1 Module function introduction

The intc interrupt controller controls the processor's interrupt sources. 64bit interrupt source can independently control the opening and masking of each interrupt.

9.2 Drive position

The location of the driver source code:

```
module_drivers/drivers/irqchip/irq-ingenic.c
```

9.3 Device Tree Configuration

The location of the device tree:

```
module_drivers/dts/x1600.dtsi
```

INTC Interrupt Controller Description:

```
cpuintc: interrupt-controller {
    .....
};

core_intc: core-intc@0x12300000 {
    .....
};
```

9.3.1 Device Tree Default Configuration

The default compilation produces intc devices.

9.3.2 The default compilation produces intc devices.

9.4 Kernel compilation configuration

The kernel configures IRQ_INGENIC_CPU, the configuration instructions are as follows:

```
Symbol: IRQ_MIPS_CPU [=y]
Type : boolean
Defined at drivers/irqchip/Kconfig:93
Selects: GENERIC_IRQ_CHIP [=y] && IRQ_DOMAIN [=y]
Selected by: SOC_EMMA2RH [=n] || SOC_PNX833X [=n] || SIBYTE_SB1250 [=n] ||
SIBYTE_BCM1120 [=n] ||
BYTE_BCM1125 [=n] || SIBYTE_BCM1125H [=n] || SIBYTE_BCM112X [=n] ||
SIBYTE_BCM1x80 [=n]
```

```
Symbol: INGENIC_INTC [=y]
Type : boolean
Defined at drivers/irqchip/Kconfig:197
Selects: IRQ_DOMAIN [=y]
Selected by: SOC_X1000 [=n] && <choice> || SOC_X1800 [=n] && <choice> || SOC_X1021
[=n] && <choice>
|| SOC_X1520 [=n] && <choice> || SOC_X1630 [=n] && <choice> || SOC_X1600 [=y] &&
<choice>
```

9.4.1 Kernel default compilation configuration

The kernel configures the intc driver by default.

9.4.2 Kernel Custom Compilation Configuration

9.5 Device Node Generation

Generate a device node after the driver is successfully registered.

```
/sys/devices/platform/apb/10000000.clock-
controller/subsystem/devices/10001000.core-intc
```

9.6 Application Instructions

View outages:

```
CPU0
2: 28773 MIPS 2 ingenic cascade interrupt
4: 28014 MIPS 4 ingenic-timerost
9: 63 INTC 1 13500000.otg, 13500000.otg, dwc2_hstotg:usb1
11: 540 INTC 3 pdma
12: 5 INTC 4 pdmad
15: 27990 INTC 7 13440000.sfc
38: 0 INTC 30 13060000.cim
44: 39 INTC 36 13460000.msc
53: 112 INTC 45 uart2
69: 1 INTC 61 i2c0
72: 0 GPIO 31 WAKEUP
73: 0 GPIO 27 bootsel0
74: 0 GPIO 28 bootsel1
ERR: 0
```

10 BLUETOOTH

10.1 Module Introduction

10.1.1 AP6212 chip

The x1600 uses the AP6212 chip, which is a functional module integrating wifi and bluetooth based on the BCM4340A1 scheme.

- support bluetooth module HCI UART interface, audio data of PCM interface.
- bluetooth standard specification 5.0.

10.1.2 GPIO Function Description

Bluetooth module

Name	I/O	Description
BT_REG_ON	I	Powerup/down internal regulators used by BT section
BT_WAKE	I	HOST wake-up Bluetooth device
BT_HOST_WAKE	O	Bluetooth device to wake-up HOST
UART_RTS_N	O	Bluetooth UART interface
UART_TXD	O	Bluetooth UART interface
UART_RXD	I	Bluetooth UART interface
UART_CTS_N	I	Bluetooth UART interface

10.2 Driver source location

Driver source location:

```
module_drivers/drivers/misc/  
|——bt_power_bluesleep.c
```

10.3 Device Tree Configuration

10.3.1 Device Tree Default Configuration

In the board device tree halley6_v10.dts, perform the following default configurations:

1. Configure uart for Bluetooth

```
&uart0 {  
    status = "disable";  
    /*dma-mode*/  
    pinctrl-names = "default";  
    pinctrl-0 = <&uart0_pd>;  
};
```

2. Configure a gpio related to power management

```
bt_power {  
    compatible = "ingenic, bt_power";
```

```
/*ingenic,reg-on-gpio = <&gpb 5 GPIO_ACTIVE_LOW INGENIC_GPIO_NOBIAS>*/  
/*ingenic,wake-gpio = <&gpb 2 GPIO_ACTIVE_LOW INGENIC_GPIO_NOBIAS>*/  
};
```

10.3.2 Device Tree User-defined Configuration

You can enable the configuration as required.

10.4 Kernel compilation configuration

Kernel configuration BCMDHD is described as follows:

```
Symbol: BCMDHD [=y]  
Type : tristate  
Prompt: Broadcom FullMAC wireless cards support  
Location:  
-> Ingenic device-drivers Configurations  
(1) -> SDIO-WIFI drivers  
Defined at module_drivers/drivers/net/wireless/bcmdhd/Kconfig
```

10.4.1 Kernel Default compilation Configuration

The kernel is configured to turn this module on by default.

10.4.2 Kernel Custom Compilation Configuration

You can disable the module as required

10.5 Kernel version differences

10.6 Device Node Generation

The following nodes are generated after the driver is successfully loaded:

1. UART node

```
# ls dev/ttyS*  
dev/ttyS0 dev/ttyS1 dev/ttyS3
```

2. Power management nodes

```
# cat sys/class/rfkill/rfkill0/name  
bluetooth
```

10.7 Application Instructions

10.7.1 Module Power Supply

Note: for the first time on the system power must echo 1 / sys/class/rfkill rfkill0 / state, or may be on electricity is not successful **

```
# echo 1 > sys/class/rfkill/rfkill0/state
```

```
[ 1402.277128] restore_pin_status is not defined
```

10.7.2 Obtaining the Bluetooth Module ID Through the Serial Port

- test program: bcm_test
- ensure bluetooth power supply
- ensure UART interface and the bluetooth communication module is normal

```
# ./pretest/bcm_test -d /dev/ttyS0  
  
=====Read Successfully! Chip Version : BCM4340A1
```

10.7.3 Bluetooth uart channel test

Set up the bsa_server

Test program: bsa_server

Steps:

1. Enter adb shell

```
ser@user-HP-Compaq-8200:~$ adb shell
```

2. Run the bsa_server command to start the service

```
# bsa_server -r 15 -p /lib/firmware/bluetooth/BCM43430A1.hcd -u /var/run/ -d /dev/ttyS0
```

- -r Specifies baudrate as 3M * (UART supports a maximum of 3M).
- -p bluetooth firmware specified path
- -u specifies generate bt node location
- -d equipment

Note: bsa_server cannot be executed directly on the serial port. Because the serial port is too slow, bsa_server cannot be executed properly. The test requires executing bsa_server through adb shell.

3. After the bsa_server service succeeds, a daemon process is created, producing two bt nodes

```
# ls var/run/bt*  
var/run/bt-avk-fifo      var/run/bt-daemon-socket
```

10.7.4 Bluetooth pcm channel test

The Bluetooth voice call environment needs to be set up.

10.7.5 Develop reference content based on bsa server

1. Location of the bsa_server development guide

```
packages/example/App/bluetooth_demo/BSA_GATT_Guide-v03.pdf  
packages/example/App/bluetooth_demo/BSA_Simple_Guideline-v01.pdf
```

2. demo source directory

```
packages/example/App/bluetooth_demo/3rdparty/
```

3. Compile app quickly (refer to bsa_server development Guide for details)

a) Configure the cross-compilation tool

```
# export PATH=prebuilts/toolchains/mips-gcc720-glibc226/bin:$PATH
```

b) Enter the build of the APP to be compiled. Take app_manager as an example */

```
# cd packages/example/App/bluetooth_demo
# cd 3rdparty/embedded/bsa_examples/linux/app_manager/build
```

c) Configure the environment

```
# export MIPS GCC=mips-linux-gnu-gcc
```

d) Compiling

```
# make CPU=mips clean
# make CPU=mips
```

e) Generate an executable program in the build directory

```
# ls mips/
app_manager obj/
```

10.8 FAQ

10.8.1 bsa_server does not support -lpm parameter

The low power mode is not supported

10.8.2 Common normal error print, do not affect the normal use

The 3D command is not supported

```
BSA_trace 7827@ 01/01 00h:27m:42s:803ms: Num HCI Cmd Packets : 1 (0x01)
BSA_trace 7828@ 01/01 00h:27m:42s:803ms: Cmd Code : 0xfc7 (VSC [Set TV2TV 3D])
BSA_trace 7829@ 01/01 00h:27m:42s:803ms: Status : Unknown HCI Command (0x01)
BSA_trace 7830@ 01/01 00h:27m:42s:803ms: Raw HCI Data Received:
BSA_trace 7831@ 01/01 00h:27m:42s:803ms: 0000: 0e 04 01 b7 fc 01
BSA_trace 7832@ 01/01 00h:27m:42s:803ms: >>
BSA_trace 7833@ 01/01 00h:27m:42s:803ms: [3DTV]bsa_sv_dm_send_3dtv_vsc_cmpl_cback
BSA_trace 7834@ 01/01 00h:27m:42s:803ms: [3DTV]ERROR bsa_sv_dm_send_3dtv_vsc_cmpl_cback FAIL
BSA_trace 7835@ 01/01 00h:27m:42s:803ms: [3DTV]bsa_sv_dm_send_3dtv_vsc_cmpl_cback RETURN:8
BSA_trace 7836@ 01/01 00h:27m:42s:803ms: bsa_sv_dm_send_set_config_complete return status:0x0
```

Check that the low-power mode does not exist

```
BSA_trace 7681@ 01/01 00h:27m:42s:795ms: HCILP_Disable
BSA_trace 7682@ 01/01 00h:27m:42s:795ms: BTM: BTM_VendorSpecificCommand: Opcode: 0xFC27, ParamLen: 12.
BSA_trace 7683@ 01/01 00h:27m:42s:795ms: UPIO_set_bluesleep_proto : 0
BSA_trace 7684@ 01/01 00h:27m:42s:795ms: Fail to open /proc/bluetooth/sleep/proto
```

11 THE WATCHDOG MODULE

11.1 Module function introductions

The watchdog timer is used to reset the processor when the processor is disturbed by faults such as noise and system errors. The watchdog timer can generate a reset signal. The watchdog uses the clock source provided by RTC, which can be divided into 1, 4, 16, 64, 256 and 1024 by software, and has a 16-bit counter register. In addition, it also supports semi-interrupt processing.

11.2 Driver source code location

Path of driver source code:

```
drivers/watchdog/ingenic_wtd.c
```

11.3 Device Tree Configurations

Location of the device tree:

```
module_drivers/dts/x1600.dtsi
```

WDT controller description:

```
watchdog: watchdog@0x10002000 {
    compatible = "ingenic,watchdog";
    reg = <0x10002000 0x40>;
    interrupt-parent = <&core_intc>;
    interrupts = <IRQ_TCU0>;
    status = "ok";
}
```

11.3.1 Default Configurations of Device Tree

The default compilation will generate WDT devices.

11.3.2 Device Tree Custom Configurations

Users can turn off the WDT device according to actual needs, and configure the node to disable:

```
&watchdog {
    status = "disable";
};
```

11.4 Kernel compilation configurations

The kernel configuration INGENIC_WDT, the configuration description is as follows:

```
Symbol: INGENIC_WDT [=y]
Type : tristate
Defined at module_drivers/drivers/watchdog/Kconfig
Prompt: Ingenic ingenic SoC hardware watchdog
```

11.4.1 Kernel default compilation configurations

The kernel is configured with WDT watchdog by default, and the configuration interface is as follows:

```
<*> Ingenic ingenic SoC hardware watchdog
< > SGM820 watchdog
```

11.4.2 Kernel custom compilation configurations

Users can remove the driver configuration according to actual needs.

11.5 Module kernel differences

no difference

11.6 Device node generation

Generate device node after successful driver registration

```
/dev/watchdog
```

11.7 Application Instructions

In the released SDK, busybox will configure the watchdog test application by default. The following is the description of the test application:

```
watchdog [-t N[ms]] [-T N[ms]] [-F] DEV

Periodically write to watchdog device DEV

Options:

    -T N Reboot after N seconds if not reset (default 60)
    -t N Reset every N seconds (default 30)
    -F Run in foreground

Use 500ms to specify period in milliseconds
```

Execute the following command to test the watchdog:

- 1) Set the feeding cycle

```
watchdog -t 3 /dev/watchdog0 /*Perform a dog feeding operation ping every 3s*/
```

- 2) Set timeout to restart

```
watchdog -t 5 -T 3 /dev/watchdog0
```

```
/*Reset after 3s (feed the dog every 5s, reset if the dog is not fed for 3s)*/
```

3) reboot restart

```
reboot (The system restarts immediately after you run the reboot command)
```

12 PDMA controller

12.1 Module function introduction

Programmable DMA Controller (PDMAC) dedicated to intelligent data transfer between on-chip peripherals (MSC, AIC, UART, etc.), external memory and memory-mapped peripherals, supporting up to 32 independent DMA channels.

12.2 Drive position

The location of the driver source code:

```
module_drivers/drivers/dma/ingenic
├── ingenic_dma.c
├── ingenic_dma.h
└── Makefile
```

12.3 Device Tree Configuration

The location of the device tree:

```
module_drivers/dts/x1600.dtsi
```

PDMA Controller Description:

```
pdma: dma@13420000 {
    compatible = "ingenic,x1600-pdma";
    reg = <0x13420000 0x10000>;
    interrupt-parent = <&core_intc>;
    interrupt-names = "pdma", "pdmad";
    interrupts = <IRQ_PDMA>, <IRQ_PDMAD>;
    #dma-channels = <32>;
    #dma-cells = <1>;
    ingenic,reserved-chs = <0x3>;
};
```

12.3.1 Device Tree Default Configuration

The default compilation will generate a pdma device.

```
&pdma {
    status = "okay";
};
```

12.3.2 Device Tree Custom Configuration

Users can turn off the PDMA device according to actual needs, and can configure the node to disable.

```
&pdma {
```

```
status = "disable";  
};
```

12.4 Kernel compilation configuration

The kernel configures INGENIC_PDMAC, the configuration instructions are as follows:

```
Symbol: INGENIC_PDMAC [=y]  
Type : bool  
Defined at module_drivers/drivers/dma/ingenic/Kconfig:1  
Prompt: Ingenic programmable dma controller (Of Driver)  
Depends on: MACH_XBURST [=y] || MACH_XBURST2 [=n]  
Location:  
    -> Ingenic device-drivers Configurations  
        -> [DMA] Drivers  
Selects: DMA_VIRTUAL_CHANNELS [=y] && DMA_ENGINE [=y]  
Selected by [y]: - SND_ASOC_PDMA [=y] && SND_ASOC_INGENIC [=y] &&  
SND_ASOC_INGENIC_AS_V1 [=y]
```

12.4.1 Kernel default compilation configuration

The kernel configures the PDMA driver by default, and the configuration interface is as follows:

```
-*- Ingenic programmable dma controller (Of Driver)  
<*> ingenic dma receiving data is externally determined  
[ ] Ingenic AIC use pdma as dma.
```

12.4.2 Kernel Custom Compilation Configuration

1. Users can remove the configuration of the driver according to actual needs.
2. Drive test configuration, compile configuration is not enabled by default, the configuration description is as follows:

```
Symbol: DMATEST [=y]  
Type : tristate  
Prompt: DMA Test client  
Location:  
    -> Device Drivers  
        -> DMA Engine support (DMADEVICES [=y])  
Defined at drivers/dma/Kconfig:563  
Depends on: DMADEVICES [=y] && DMA_ENGINE [=y]
```

The configuration interface is as follows:

```

--- DMA Engine support
[ ] DMA Engine debugging
*** DMA Devices ***
< > Freescale eDMA engine support
< > IMG MDC support
< > Intel integrated DMA 64-bit support
[*] Ingenic programmable dma controller (Of Driver)
< > Synopsys DesignWare AHB DMA platform driver
<*> ingenic dma receiving data is externally determined
*** DMA Clients ***
[ ] Async_tx: Offload support for the async_tx api
<*> DMA Test client

```

12.5 Version Differences

The difference between kernel4.4.94 and kernel5.10 is that kernel5.10 removes part of the api and kernel4.4.94 has a different api interface for dma channel application, users can refer to 12.8.2.2.

12.6 Device Node Generation

12.6.1 Controller Node

After the driver is loaded successfully, the following controller node is generated:

```

# cd /sys/devices/platform/ahb2/13420000.dma/dma/
# ls
dma0chan0 dma0chan14 dma0chan2 dma0chan25 dma0chan30 dma0chan8
dma0chan1 dma0chan15 dma0chan20 dma0chan26 dma0chan31 dma0chan9
dma0chan10 dma0chan16 dma0chan21 dma0chan27 dma0chan4
dma0chan11 dma0chan17 dma0chan22 dma0chan28 dma0chan5
dma0chan12 dma0chan18 dma0chan23 dma0chan29 dma0chan6
dma0chan13 dma0chan19 dma0chan24 dma0chan3 dma0chan7

```

12.6.2 Test program node

If the driver test is configured, the following test nodes are generated:

```

# cd /sys/module/dmatest/parameters/
# ls
channel noverify threads_per_chan xor_sources
device pq_sources timeout
iterations run verbose
max_channels test_buf_size wait

```

12.7 How to use mcu

The MCU is an xburst@I CPU integrated into the SOC running at 300MHz. The main CPU can communicate with the MCU via interrupts.

MCU SDK is divided into two main parts.

-
- 1.MCU firmware development, the program running on the MCU
 - 2.MCU firmware loader and MCU functions called by the main CPU

12.7.1 mcu firmware introduction

libmcu-bare is MCU firmware source code project, programs running on MCU can be developed in this library.

Path:

```
# cd development/libmcu-bare
```

12.7.2 Introduction of mcu loader

1.The code to load the mcu firmware is in the "development/mcu-host" path, which is compiled to generate mcu-host.

2.The MCU loader kernel driver is located in the following directory:

```
# module_drivers/drivers/remoteproc/ingenic/mcu_driver
```

This can be selected in the kernel by the following configuration:

```
Symbol: INGENIC_MCU_RPROC [=y]
Type : tristate
Prompt: ingenic mcu remoteproc support
Location:
    -> Device Drivers
        -> Remoteproc drivers
Defined at drivers/remoteproc/Kconfig:90
Depends on: HAS_DMA [=y]
Selects: REMOTEPROC [=n]
```

12.7.3 How to use mcu loader

Execute on the development board :

1 The compiled firmware via libmcu-bare is placed under the lib/firmware directory (or in the /firmware directory).

2 The application compiled by mcu-host is placed under the file system.

3./mcu-host write_firmware /lib/firmware/libmcu-bare.bin to load the firmware

4./mcu-host bootup to enable mcu function

The following will be printed if enabled successfully:

```
#!/mcu-host bootup
dmr: fffffff fffffffd 33
mcu @ Nov 21 2022 14:55:46
vendor_init is called
```

5./mcu-host write_data The host core writes data to the mcu

A successful write will result in the following print:

```
#!/mcu-host write_str test123456  
len: 11 : test123456  
[ 61.750454] received msg: 11
```

6./mcu-host read_data The host core reads data from mcu

A successful read will result in the following print:

```
#!/mcu-host read_str 11  
test123456
```

12.8 Application Instructions

12.8.1 Dmatest test program

1. Enter the node of the test program

```
# echo dma0chan31 > /sys/module/dmatest/parameters/channel  
# echo 2000 > /sys/module/dmatest/parameters/timeout  
# echo 1 > /sys/module/dmatest/parameters/iterations  
# echo 1 > /sys/module/dmatest/parameters/run
```

2. Test results

```
[ 148.309057] dmatest: Started 1 threads using dma0chan31  
[ 148.309825] dmatest: dma0chan31-copy: summary 1 tests, 0 failures 539 iops 5399 KB/s (0)
```

12.8.2 SSI driver test

The PDMA driver follows the kernel standard dmaengine driver framework and can be used as a provider to provide dma transmission channels for other drivers. Here, the SSI driver is used as an example to introduce how to configure.

12.8.2.1 Device Tree Configuration

```
spi0: spi0@0x10043000 {  
    compatible = "ingenic,spi";  
    reg = <0x10043000 0x1000>;  
    interrupt-parent = <&core_intc>;  
    interrupts = <IRQ_SSI0>;  
    dmas = <&pdma INGENIC_DMA_TYPE(INGENIC_DMA_REQ_SSI0_TX)>,  
          <&pdma INGENIC_DMA_TYPE(INGENIC_DMA_REQ_SSI0_RX)>;  
    dma-names = "tx", "rx";  
    #address-cells = <1>;  
    #size-cells = <0>;  
    status = "disabled";  
};
```

Among them, the dmas and dma-names nodes are dma-related configurations

```
dmas = <&pdma INGENIC_DMA_TYPE(INGENIC_DMA_REQ_SSI0_TX)>,  
      <&pdma INGENIC_DMA_TYPE(INGENIC_DMA_REQ_SSI0_RX)>;  
dma-names = "tx", "rx";
```

dmas Indicates that the corresponding transmission channel provided by PDMA is used
dma-names is a flag used to apply for a dma transmission channel in the SSI driver.

12.8.2.2 Driver usage flow

Since the kernel standard dmaengine driver framework is used, all consumer-side drivers use the same API, and the usage process is basically the same.

The process used in the SSI driver in kernel-4.4.94 is as follows:

1. Apply dma channel

```
ingspi->txchan = dma_request_slave_channel_reason(dev, "tx");  
ingspi->rxchan = dma_request_slave_channel(dev, "rx");
```

2. Configure the parameters of the dma channel

```
dmaengine_slave_config(txchan, &tx_config);  
dmaengine_slave_config(rxchan, &rx_config);
```

3. get transfer descriptor

```
txdesc = dmaengine_prep_slave_sg(  
    txchan,  
    t->tx_sg.sgl, t->tx_sg.nents,  
    DMA_DEV_TO_MEM, DMA_PREP_INTERRUPT | DMA_CTRL_ACK);  
rxdesc = dmaengine_prep_slave_sg(  
    rxchan,  
    t->rx_sg.sgl, t->rx_sg.nents,  
    DMA_DEV_TO_MEM, DMA_PREP_INTERRUPT | DMA_CTRL_ACK);
```

4. Commit and start transfer

```
dmaengine_submit(txdesc);  
dmaengine_submit(rxdesc);  
dma_async_issue_pending(rxchan);  
dma_async_issue_pending(txchan);
```

The process used in the SSI driver in kernel-5.10 is as follows:

1. Apply dma channel

```
ingspi->txchan = dma_request_chan(dev, "tx");  
ingspi->rxchan = dma_request_chan(dev, "rx");
```

2. Configure the parameters of the dma channel

```
dmaengine_slave_config(txchan, &tx_config);
```

```
dmaengine_slave_config(rxchan, &rx_config);
```

3. get transfer descriptor

```
txdesc = dmaengine_prep_slave_sg(  
    txchan,  
    t->tx_sg.sgl, t->tx_sg.nents,  
    DMA_DEV_TO_MEM, DMA_PREP_INTERRUPT | DMA_CTRL_ACK);  
rxdesc = dmaengine_prep_slave_sg(  
    rxchan,  
    t->rx_sg.sgl, t->rx_sg.nents,  
    DMA_DEV_TO_MEM, DMA_PREP_INTERRUPT | DMA_CTRL_ACK);
```

4. Commit and start transfer

```
dmaengine_submit(txdesc);  
dmaengine_submit(rxdesc);  
dma_async_issue_pending(rxchan);  
dma_async_issue_pending(txchan);
```

13 THE SADC CONTROLLER

13.1 Module function introductions

The A/D conversion module of the X1600 chip is a 10-bit high precision analog-to-digital converter that supports six analog inputs. The A/D is CMOS low-power dissipation 10bit touch screen SAR(successive Approximation Register) analog to digital converter. It operates with 3.3V power supply. It is developed as an embedded high resolution ADC targeting to the 40nm CMOS process and has wide application in portable electronic

devices, high-end home entertainment center, communication systems and so on.

The SAR A/D controller is dedicated to control A/D to work at two different modes: normal operation mode and repeat sampling. The sampling data can transfer to memory through CPU.

13.2 Driver location

The path of the driver source code:

```
module_drivers/drivers/mfd/ingenic_adc_v13.c
module_drivers/drivers/mfd/ingenic_adc_aux.c
```

13.3 Device Tree Configurations

The path of the device tree:

```
module_drivers/dts/x1600.dtsi
```

SADC controller description:

```
sadc: sadc@10070000 {
    compatible = "ingenic,x2000-v12-sadc";
    reg = <0x10070000 0x32>;
    interrupt-parent = <&core_intc>;
    interrupts = <IRQ_SADC>;
    interrupt-controller;
    status = "ok";
};
```

13.3.1 Device tree default configurations

The default compilation of the device tree will generate SADC devices.

13.3.2 Device Tree Custom Configurations

The user can turn off the SADC device according to actual needs, and configure the

node as disabled in the board.dts.

```
&rtc {  
    status = "disabled";  
};
```

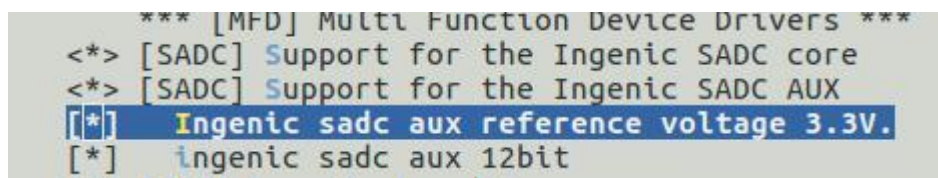
13.4 Kernel compilation configurations

The kernel configuration MFD_INGENIC_SADC_V13, the configuration description is as follows:

```
Selects: MFD_CORE [=y]  
Symbol: MFD_INGENIC_SADC_V13 [=y]  
Type : tristate  
Defined at module_drivers/drivers/mfd/Kconfig  
Prompt: [SADC] Support for the Ingenic SADC core  
  
Symbol: MFD_INGENIC_SADC_AUX [=y]  
Type : tristate  
Defined at module_drivers/drivers/mfd/Kconfig  
Prompt: [SADC] Support for the Ingenic SADC AUX
```

13.4.1 Kernel default compilation configurations

The kernel is configured with RTC driver by default, and the configuration interface is as follows:



```
*** [MFD] Multi Function Device Drivers ***  
<*> [SADC] Support for the Ingenic SADC core  
<*> [SADC] Support for the Ingenic SADC AUX  
[*] Ingenic sadc aux reference voltage 3.3V.  
[*] ingenic sadc aux 12bit
```

13.5 Module kernel differences

no difference

13.6 Device node generation

After the driver is loaded successfully, the following nodes are generated:

```
# cd /dev/  
# ls  
/dev/ingenic_adc_aux_0  
/dev/ingenic_adc_aux_1  
/dev/ingenic_adc_aux_2  
/dev/ingenic_adc_aux_3
```

13.7 Application Instructions

13.7.1 Application source location

```
packages/example/Sample/sadc
```

13.7.2 Commands and Parameters

By default, applications are stored in /testsuite/sadc_sample:

```
cd /testsuite/sadc_sample  
./sadc_sample adc_num
```

adc_num indicates the node number of the adc device in /dev. In the x1600 chip, the desirable values of adc_num are 0,1,2,3.

14 THE RTC CONTROLLER

14.1 Module function introductions

The real-time clock (RTC) unit can run when the chip's main power is on or the main power is off but the RTC power is still on. In this case, the RTC power domain consumes only a few microwatts of power. RTC includes real-time, timing alarm logic and power-off and wake-up control logic.

14.2 Driver location

The path of the driver source code:

```
module_drivers/drivers/rtc/rtc-ingenic.c
module_drivers/drivers/rtc/rtc-ingenic.h
```

14.3 Device Tree Configurations

The path of the device tree:

```
module_drivers/dts/x1600.dtsi
```

RTC controller description:

```
rtc: rtc@0x10003000 {
    compatible = "ingenic,rtc";
    reg = <0x10003000 0x4c>;
    interrupt-parent = <&core_intc>;
    interrupts = <IRQ_RTC>;
    system-power-controller;
    power-on-press-ms = <1000>;
    status = "ok";
};
```

14.3.1 Device tree default configurations

The default compilation of the device tree will generate RTC devices.

14.3.2 Device Tree Custom Configurations

The user can turn off the RTC device according to actual needs, and configure the node as disabled in the *board.dts*.

```
&rtc {
    status = "disabled";
};
```

14.4 Kernel compilation configurations

The kernel configuration RTC_DRV_INGENIC, the configuration description is as follows:

```
CONFIG_RTC_DRV_INGENIC:
Symbol: RTC_DRV_INGENIC [=y]
Type : tristate
Defined at module_drivers/drivers/rtc/Kconfig
```

14.4.1 Kernel default compilation configurations

The kernel is configured with RTC driver by default, and the configuration interface is as follows:

```
< > [AES] Support for INGENIC AES hw engine
      [WDT] watchdog Drivers --->
      [PWM] drivers --->
      [RTC] drivers --->
```

```
<*> INGENIC RTC
[*]   Ingenic RTC suspend test
(60)   Ingenic RTC suspend alarm time Unit of second
```

14.4.2 Kernel custom compilation configurations

- 1) The user can remove the driver configuration according to actual needs.
- 2) Configure SUSPEND_TEST and SUSPEND_ALARM_TIME during RTC test. The configuration instructions are as follows:

```
CONFIG_SUSPEND_TEST:

If you say yes here the Ingenic rtc will support suspend test.

Symbol: SUSPEND_TEST [=y]
Type : bool
Defined at arch/mips/xburst/Kconfig

Prompt: auto suspend test
Depends on: SOC_TYPE [=n]
Location:
    -> Machine selection
    -> SOC type (SOC_TYPE [=n])
Defined at module_drivers/drivers/rtc/Kconfig

Prompt: Ingenic RTC suspend test
Depends on: RTC_DRV_INGENIC [=y]
```

```
CONFIG_SUSPEND_ALARM_TIME:
```

If you say yes here Automatically wake up after a set time of sleep.

Symbol: SUSPEND_ALARM_TIME [=60]

Type : integer

Defined at arch/mips/xburst/Kconfig

Prompt: suspend alarm time(second)

Depends on: SOC_TYPE [=n] && SUSPEND_TEST [=y]

Location:

-> Machine selection

-> SOC type (SOC_TYPE [=n])

-> auto suspend test (SUSPEND_TEST [=y])

Defined at module_drivers/drivers/rtc/Kconfig

Prompt: Ingenic RTC suspend alarm time Unit of second

Depends on: SUSPEND_TEST [=y]

Location:

-> Ingenic device-drivers Configurations

-> [RTC] drivers

-> INGENIC RTC (RTC_DRV_INGENIC [=y])

-> Ingenic RTC suspend test (SUSPEND_TEST [=y])

The configuration interface is as follows:

```
--- INGENIC RTC
[*] Ingenic RTC suspend test
(60) Ingenic RTC suspend alarm time Unit of second
```

14.5 Module kernel differences

no difference

14.6 Device node generation

After the driver is loaded successfully, the following nodes are generated:

```
/dev/rtc0
```

14.7 Application Instructions

14.7.1 Timed wake-up test

```
echo +5 > /sys/class/rtc/rtc0/wakealarm
```

After the rtc calculates the specified time, the system automatically wakes up.

15 EFUSE INTERFACE

15.1 Module Functions

The EFUSE module provides an interface to view the value of each efuse register. You can view the cpu information saved in efuse or user-defined information.

15.2 Driver Position

Driver source location

```
module_drivers/drivers/misc/ingenic_efuse_v2.c
```

15.3 Device Tree Configuration

Device tree location:

```
module_drivers/dts/x1600.dtsi
```

Efuse Controller Description:

```
efuse: efuse@0x13540000 {  
    compatible = "ingenic,x1600-efuse";  
    reg = <0x13540000 0x10000>;  
    status = "okay";  
};
```

15.3.1 Device Tree Default Configuration

By default, EFUSE devices are not compiled in the device tree.

15.3.2 Device Tree User-defined Configuration

You can enable the EFUSE device as required and set the EFuse node to disabled in the board-level.

```
&efuse {  
    status = "disable";  
};
```

15.4 Kernel compilation configuration

The kernel configuration INGENIC_EFUSE_v2 is described as follows:

```
Symbol: INGENIC_EFUSE_V2 [=n]  
Type : boolean  
Prompt: [Efuse] Ingenic Efuse v2 Driver  
Location:  
(1) -> Ingenic device-drivers Configurations  
Defined at module_drivers/drivers/misc/Kconfig:19
```

Depends on: SOC_X1600 [=y]

15.4.1 Kernel Default compilation Configuration

The EFUSE driver is not configured in the kernel by default. The configuration page is as follows:

```
[ ] Linux pmem allocator
[ ] [Efuse] Ingenic Efuse v2 Driver
< > Bluetooth power control driver for BCM-4345C5 module
[RANDOM] drivers --->
*** [MFD] Multi Function Device Drivers ***
< > [SADC] Support for the Ingenic SADC core
< > [SADC] Support for the Ingenic SADC AUX
```

15.4.2 Kernel Custom Compilation Configuration

You can enable the configuration of the driver as required.

15.5 Kernel version differences

15.6 Device Node Generation

The following nodes are generated after the driver is successfully loaded:

```
ls /sys/devices/platform/ahb2/13540000.efuse/efuse_rw/
chipid  cutid   nku      socinfo trim1   trim3
chipkey hideblk prt      trim0   trim2   userkey
```

15.7 Application Instructions

15.7.1 Reading Data in efuse

Taking trim0 as an example, the method of reading efuse data is introduced.

Execute an order:

```
cat /sys/devices/platform/ahb2/13540000.efuse/efuse_rw/trim0
```

The read data is shown in the figure below.

```
# cat /sys/devices/platform/ahb2/13540000.efuse/efuse_rw/trim0
00001555
```

15.8 Precautions

efuse can be written only once. Repeated writing may cause unexpected errors on the chip. Therefore, exercise caution when writing EFUSE.

16 GPIO General IO interface

16.1 Module function introduction

gpio is generally fixed when the development board is designed. Some gpio can only be used as device multiplexing function pins, and some gpio are used as common input and output and interrupt detection functions. For fixed device multiplexing function pins Defined in the following file:

```
module_drivers/dts/x1600-pinctrl.dtsi
```

In module_drivers/dts/"board level".dts, the corresponding device function pins will be selected according to the driver configuration. The gpio driver of the kernel is implemented based on the gpio subsystem, which is very convenient for gpio control.

16.2 Drive position

The location of the driver source code

```
module_drivers/drivers/pinctrl
```

16.3 Device Tree Configuration

The location of the device tree:

```
module_drivers/dts/x1600-pinctrl.dtsi
```

Some GPIO configurations are as follows:

```
&pinctrl {
    uart0_pin: uart0-pin {
        uart0_pd: uart0-pd {
            ingenic,pinmux = <&gpd 7 10>;
            ingenic,pinmux-funcsel = <PINCTL_FUNCTION0>;
        };
    };
    uart1_pin: uart1-pin {
        .....
    };

    uart2_pin: uart2-pin {
        .....
    };

    uart3_pin: uart3-pin {
        .....
    }
}
```

16.3.1 Device Tree Default Configuration

16.3.2 Device Tree Custom Configuration

16.3.2.1 Configuring pincfg properties

When gpio is multiplexed as a device function, additional properties of a pin can be specified through "ingenic, pincfg",

- Pull up
- Pull down
- High resistance
- Driving ability
- SCHMITT
- SLEW_RATE

Where x1600 does not support filtering, high resistance state, drive capability, Schmidt input and conversion rate settings.

PINCTL_CFG_BIAS_DISABLE	Off polarization voltage configuration
PINCTL_CFG_BIAS_HIGH_IMPEDANCE	Set high resistance state
PINCTL_CFG_BIAS_PULL_DOWN	Set the pull-down voltage
PINCTL_CFG_BIAS_PULL_PIN_DEFAULT	Set default pull-up voltage
PINCTL_CFG_BIAS_PULL_UP	Set pull-up voltage
PINCTL_CFG_DRIVE_STRENGTH	Set drive capability
PINCTL_CFG_FILTER	Set the filter function
PINCTL_CFG_INPUT_SCHMITT_ENABLE	Enable Schmidt input
PINCTL_CFG_SLEW_RATE	Set slew rate

where pincfgs can be of the form:

```
#define PINCTL_CFG_BIAS_DISABLE      1
#define PINCTL_CFG_BIAS_HIGH_IMPEDANCE  2
#define PINCTL_CFG_BIAS_PULL_DOWN    3
#define PINCTL_CFG_BIAS_PULL_PIN_DEFAULT  4
#define PINCTL_CFG_BIAS_PULL_UP      5
#define PINCTL_CFG_DRIVE_STRENGTH    9
#define PINCTL_CFG_FILTER            10
#define PINCTL_CFG_INPUT_SCHMITT_ENABLE 11
#define PINCTL_CFG_SLEW_RATE         17
```

```

* PINCFG_PACK(PINCTL_CFG_DRIVE_STRENGTH, 7) # value: 0 ~ 7
* PINCFG_PACK(PINCTL_CFG_BIAS_DISABLE, 1)
* PINCFG_PACK(PINCTL_CFG_PULL_DOWN, 1) # enable pull down
* PINCFG_PACK(PINCTL_CFG_PULL_UP, 1) # Enable pull-up
* PINCFG_PACK(PINCTL_CFG_INPUT_SCHMITT_ENABLE, 1)
* PINCFG_PACK(PINCTL_CFG_SLEW_RATE, 1)

```

specific grammar:

```
ingenic,pincfg = <gpio_group start_pin end_pin pincfgs>
```

eg:

```

mac0_rgmii_p1_normal: mac0-rgmii-p1-normal {
    ingenic,pinmux = <&gpc 1 5>, <&gpc 10 10>, <&gpc 12 15>;
    ingenic,pinmux-funcsel = <PINCTL_FUNCTION1>;
    ingenic,pincfg = <&gpc 1 5 PINCFG_PACK(PINCTL_CFG_DRIVE_STRENGTH, 7)>, \
        <&gpc 10 10 PINCFG_PACK(PINCTL_CFG_DRIVE_STRENGTH, 7)>, \
        <&gpc 12 15 PINCFG_PACK(PINCTL_CFG_DRIVE_STRENGTH, 7)>;
};

```

16.3.2.2 Configuring gpio properties

Where x1600 does not support high resistance, driveability, conversion rate and Schmittability.

INGENIC_GPIO_NOBIAS	Turn off polarization voltage
INGENIC_GPIO_PULLEN	Enable pull-up voltage
INGENIC_GPIO_PULLUP	Pull-up voltage
INGENIC_GPIO_PULLDOWN	Pull down voltage
INGENIC_GPIO_HIZ	High resistance
INGENIC_GPIO_DS_*	Drive Capability
INGENIC_GPIO_SLEW_RATE	Slew Rate
INGENIC_GPIO_SCHMITT	Schmitt

When defining a common GPIO, you can also configure the properties of the GPIO. details as follows:

```

#define INGENIC_GPIO_BIAS_MASK      0x7
#define INGENIC_GPIO_BIAS_SFT      0
#define INGENIC_GPIO_NOBIAS        (0 << INGENIC_GPIO_BIAS_SFT)
#define INGENIC_GPIO_PULLEN        (1 << INGENIC_GPIO_BIAS_SFT)
#define INGENIC_GPIO_PULLUP        (2 << INGENIC_GPIO_BIAS_SFT)
#define INGENIC_GPIO_PULLDOWN      (3 << INGENIC_GPIO_BIAS_SFT)
#define INGENIC_GPIO_HIZ           (4 << INGENIC_GPIO_BIAS_SFT)

```

```

#define INGENIC_GPIO_DS_SFT      3
#define INGENIC_GPIO_DS_MSK      0x7
#define INGENIC_GPIO_DS_0        (0 << INGENIC_GPIO_DS_SFT)
#define INGENIC_GPIO_DS_1        (1 << INGENIC_GPIO_DS_SFT)
#define INGENIC_GPIO_DS_2        (2 << INGENIC_GPIO_DS_SFT)
#define INGENIC_GPIO_DS_3        (3 << INGENIC_GPIO_DS_SFT)
#define INGENIC_GPIO_DS_4        (4 << INGENIC_GPIO_DS_SFT)
#define INGENIC_GPIO_DS_5        (5 << INGENIC_GPIO_DS_SFT)
#define INGENIC_GPIO_DS_6        (6 << INGENIC_GPIO_DS_SFT)
#define INGENIC_GPIO_DS_7        (7 << INGENIC_GPIO_DS_SFT)

#define INGENIC_GPIO_SLEW_RATE_SFT  6
#define INGENIC_GPIO_SLEW_RATE      (1 << INGENIC_GPIO_SLEW_RATE_SFT)

#define INGENIC_GPIO_SCHMITT_SFT    7
#define INGENIC_GPIO_SCHMITT        (1 << INGENIC_GPIO_SCHMITT_SFT)

```

Specific syntax:

```
xxxx,xx-gpio = <gpio_group pin polarity gpiocfs>
```

The "gpio" and "gpios" are kernel keywords. When defining gpio, you need to add "-gpio" or "-gpios" at the end, these suffixes will be used when using the kernel api to get the gpio number.

When you need to apply multiple attributes to a GPIO, use the value or, for example, configure the GPIO as pull-up, SCHMITT, drive capability 6

```
ingenic,lcd-pwm-gpio = <&gpc 1 GPIO_ACTIVE_LOW (INGENIC_GPIO_PULLUP |
INGENIC_GPIO_SCHMITT | INGENIC_GPIO_DS_6 )>;
```

Defining a generic GPIO example:

```

bt_power {
    compatible = "ingenic,bt_power";
    ingenic,reg-on-gpio = <&gpd 20 GPIO_ACTIVE_LOW INGENIC_GPIO_NOBIAS>;
    ingenic,wake-gpio = <&gpd 21 GPIO_ACTIVE_LOW INGENIC_GPIO_NOBIAS>;
};

```

16.3.2.3 System sleep GPIO configuration

When the system is in hibernation, the GPIO configuration for each development board is different. To reduce the power consumption of the system in hibernation, it is necessary to configure the GPIO to input up and down or output high ground.

In the board .dts file, you can configure the gpio attributes of the following gpio groups to make the system in the corresponding state when sleeping. For example: configure the pin of

the PA group (0 - 10) to be in the pull-up state, (11 - 15) for output high.

```
&gpa {
    ingenic,gpio-sleep-pullup    = <0 1 2 3 4 5 6 7 8 9 10>;
    ingenic,gpio-sleep-pulldown  = <>;
    ingenic,gpio-sleep-hiz       = <>;
    ingenic,gpio-sleep-low       = <>;
    ingenic,gpio-sleep-high      = <11 12 13 14 15>;
};
```

Configure the following properties according to the actual situation:

```
/* Board Sleep GPIO configuration. */
/* <0 1 2 3 .... 31>, set one of the pin to state.*/
&gpa {
    ingenic,gpio-sleep-pullup    = <>;
    ingenic,gpio-sleep-pulldown  = <>;
    ingenic,gpio-sleep-hiz       = <>;
    ingenic,gpio-sleep-low       = <>;
    ingenic,gpio-sleep-high      = <>;
};

&gpb {
    ingenic,gpio-sleep-pullup    = <>;
    ingenic,gpio-sleep-pulldown  = <>;
    ingenic,gpio-sleep-hiz       = <>;
    ingenic,gpio-sleep-low       = <>;
    ingenic,gpio-sleep-high      = <>;
};

&gpc {
    ingenic,gpio-sleep-pullup    = <>;
    ingenic,gpio-sleep-pulldown  = <>;
    ingenic,gpio-sleep-hiz       = <>;
    ingenic,gpio-sleep-low       = <>;
    ingenic,gpio-sleep-high      = <>;
};

&gpd {
    ingenic,gpio-sleep-pullup    = <>;
    ingenic,gpio-sleep-pulldown  = <>;
    ingenic,gpio-sleep-hiz       = <>;
    ingenic,gpio-sleep-low       = <>;
    ingenic,gpio-sleep-high      = <>;
};
```

16.4 Kernel compilation configuration

The kernel configuration GPIOLIB, kernel 4.4.94 configuration instructions are as follows:

Symbol: *GPIOLIB* [=y]
Type : *boolean*
Prompt: *GPIO Support*
Location:
-> *Device Drivers*
Defined at *drivers/gpio/Kconfig:34*
Depends on: *ARCH_WANT_OPTIONAL_GPIOLIB [=n] || ARCH_REQUIRE_GPIOLIB [=y]*
Selected by: *BCM47XX [=n] && <choice> || ARCH_REQUIRE_GPIOLIB [=y] || PINCTRL_AT91 [=n] && PINCTRL [=y] && OF [=y] && ARCH_AT91 || PINCTRL_AT91PIO4 [=n] && PINCTRL [=y] && OF [=y] && ARCH_AT91*

Symbol: *ARCH_REQUIRE_GPIOLIB* [=y]
Type : *boolean*
Defined at *drivers/gpio/Kconfig:23*
Selects: *GPIOLIB [=y]*
Selected by: *GPIO_TXX9 [=n] || MIPS_ALCHEMY [=n] && <choice> || AR7 [=n] && <choice> || ATH79 [=n] && <choice> || BCM63XX [=n] && <choice> || MACH_INGENIC [=n] && <choice> || MACH_XBURST [=n] &*

Symbol: *ARCH_WANT_OPTIONAL_GPIOLIB* [=n]
Type : *boolean*
Defined at *drivers/gpio/Kconfig:13*
Selected by: *BCM47XX [=n] && <choice>*

Symbol: *GPIOLIB_IRQCHIP* [=n]
Type : *boolean*
Defined at *drivers/gpio/Kconfig:59*
Depends on: *GPIOLIB [=y]*
Selects: *IRQ_DOMAIN [=y]*
Selected by: *PINCTRL_AT91 [=n] && PINCTRL [=y] && OF [=y] && ARCH_AT91 || PINCTRL_AT91PIO4 [=n] && PINCTRL [=y] && OF [=y] && ARCH_AT91 || PINCTRL_AMD [=n] && PINCTRL [=y] && GPIOLIB [=y] || PI*

The kernel configuration GPIOLIB, kernel 5.10 configuration instructions are as follows:

Symbol: *GPIOLIB* [=y]
Type : *bool*
Defined at *drivers/gpio/Kconfig:14*
Prompt: *GPIO Support*
Location:
(1) -> *Device Drivers*

Selected by [y]:
- PINCTRL_INGENIC_V2 [=y] && OF [=y] && (MIPS [=y] || COMPILE_TEST [=n])

Selected by [n]:
- MACH_INGENIC [=n]
.....

Symbol: ARCH_REQUIRE_GPIOLIB [=ARCH_REQUIRE_GPIOLIB]

Type : unknown

Selected by [y]:
- MACH_XBURST2 [=y] && <choice>

Selected by [n]:
- MACH_XBURST [=n] && <choice>

Symbol: GPIOLIB_FASTPATH_LIMIT [=512]

Type : integer

Range : [32 512]

Defined at drivers/gpio/Kconfig:25

Prompt: Maximum number of GPIOs for fast path

Depends on: GPIOLIB [=y]

Location:

-> Device Drivers

(2) -> GPIO Support (GPIOLIB [=y])

Symbol: GPIOLIB_IRQCHIP [=y]

Type : bool

Defined at drivers/gpio/Kconfig:46

Depends on: GPIOLIB [=y]

Selects: IRQ_DOMAIN [=y]

Selected by [y]:
- PINCTRL_INGENIC_V2 [=y] && OF [=y] && (MIPS [=y] || COMPILE_TEST [=n])

Selected by [n]:
- PINCTRL_AT91 [=n] && PINCTRL [=y] && OF [=y] && ARCH_AT91
.....

16.4.1 Kernel default build configuration

The kernel 4.4.94 kernel is configured with the GPIO driver by default, and the configuration interface is as follows:


```

[*] Pinctrl driver for ingenic
[ ] disable GPIO A GROUP IRQ CONTROLLER
[ ] disable GPIO B GROUP IRQ CONTROLLER
[ ] disable GPIO C GROUP IRQ CONTROLLER
[ ] disable GPIO D GROUP IRQ CONTROLLER
[ ] disable GPIO E GROUP IRQ CONTROLLER
[ ] multiple input lineout to single output

```

The kernel 5.10 kernel is configured with the GPIO driver by default, and the configuration interface is as follows.

```

[*] Pinctrl driver for the Ingenic X2000/X1600 SoCs (NEW)
[ ] multiple input lineout to single output (NEW)

```

16.4.2 Kernel Custom Compilation Configuration

Users can customize GPIO compilation configuration according to actual needs.

16.5 Version Differences

The differences between kernel 4.4.94 and kernel 5.10 are mainly reflected in section 16.4.1. kernel 5.10 no longer provides the ability to turn off gpio group interrupts via menuconfig.

16.6 Device Node Generation

On the premise that the kernel exports the gpio node, you can operate the /sys/class/gpio node, control the gpio input and output, and the node path:

```
/sys/class/gpio/
```

16.6.1 GPIO number calculation instructions

In the /sys/class/gpio directory of the development board system, find the gpiochipXX directory as follows:

```

# ls
export      gpiochip0  gpiochip32 gpiochip64 gpiochip96 unexport

```

Go to the gpiochip0 folder to view the contents of the label file, as shown below:

```

# cd gpiochip0/
# ls
base      device    label     ngpio     power     subsystem uevent
# cat label
GPA

```

View the contents of ngpio as follows.

```

# cat ngpio
32

```

From this it can be determined that the reference value for the gpioa group of pins is 0. The same holds true for the other gpio groups.

If you want to operate the gpioc group pin 10, then the corresponding gpio number is $64+10=74$.

16.6.2 GPIO import and export instructions

"export": User space can ask the kernel to export a GPIO control to user space by writing its number to this file. For example: if the kernel code does not request GPIO 19,

```
echo 19 > export
```

will create a "gpio19" node for GPIO19.

"unexport": Inverse of exporting to user space. E.g

```
echo 19 > unexport
```

will remove the "gpio19" node exported using the "export" file.

16.6.3 GPIO property

GPIO signal path (take GPIO 42 as an example):

```
/sys/class/gpio/gpio42
```

and has read/write properties as follows:

```
/sys/class/gpio/gpio42/direction  
/sys/class/gpio/gpio42/value  
/sys/class/gpio/gpio42/edge  
/sys/class/gpio/gpio42/active_low
```

- direction:

Read to get "in" or "out". This value usually runs the write.

When "out" is written, the default output of its pin is low. To ensure trouble-free operation, a "low" or "high" level value should be written to the GPIO's configuration as the initial output value.

Note: If the kernel does not support changing the GPIO direction, or if the kernel code does not explicitly allow user space to reconfigure the GPIO direction when exporting, then this property will not exist.

- value:

A read yields 0 (low) or 1 (high). This value allows writes if the GPIO is configured as an output. Any non-zero value is treated high.

If the pin can be configured as an interrupt signal, and if an interrupt-generating mode has been configured (see the description of "edge"), you can use a polling operation (poll(2)) on this file, and the polling operation will return when the interrupt is triggered. If you use the polling operation (poll(2)), set POLLPRI and POLLERR in events. If you use the polling operation (select(2)), set the file descriptor you expect in exceptfds. After the polling operation (poll(2)) returns, the beginning of the sysfs file can be read by the lseek(2) operation, or the file can be closed and reopened to read the data.

- edge:

Reading results in "none", "rising", "falling" or "both", writing these strings to this file selects edge-triggered mode, which will cause the polling operation (select(2)) to be in the

"value" file return. This file only exists if this pin can be configured as an interrupt-generating input pin.

- active_low:

A read yields 0 (false) or 1 (true). Writing any non-zero value flips the \ (read-write\) value of this property. already exists or has later set "rising" via the "edge" property.

16.7 Application Instructions

16.7.1 Source code location

packages/example/Sample/gpio/

16.7.2 Test method

1. The pins controlled by the test program can be changed, and the macro definition path is:

```
cat packages/example/Sample/gpio/user_gpio/huser_gpio.h
#ifndef __USER_GPIO_H
#define __USER_GPIO_H

#include <stdint.h>

#include "../gpio/gpio.h"

// GPIO 引脚定义
// LED GPIO 引脚
#define GPIO_LED 35
// BEEP GPIO 引脚
#define GPIO_BEEP 29
// KEY GPIO 引脚
#define GPIO_KEY 95
...
```

2. Take the WKUP button as an example, connect the button to the gpc31 pin, and change the GPIO_KEY definition to 95 (PA group 32 bits + PB group 32 bits + PC group 31 bits).
3. Cancel the configuration of the WKUP button in the device tree

```
/*wakeup {
    label = "WAKEUP";
    linux,code = <KEY_WAKEUP>;
    gpios = <&gpc 31 GPIO_ACTIVE_LOW INGENIC_GPIO_NOBIAS>;
    gpio-key,wakeup;
};*/
```

```
// KEY GPIO 引脚
#define GPIO_KEY 95
```

2. 以 WKUP 按键为例, 按键组 31 位)

4. The test demo is compiled to the file system by default, burn and execute gpio_demo for testing:

```
# ./gpio_demo
>>Button loosen
```

5. Press the WKUP button, the program prints:

```
# ./gpio_demo
>>Button loosen
>>Button press
```

16.7.3 dump_gpio file

A quick view gpio function file is provided under the path /sys/devices/platform/apb/10010000.pinctrl. Users can cat dump_gpio file to see if the corresponding gpio function is the desired function for the purpose of express debug. As shown in the following figure:

```
# cat dump_gpio
REG      |+++GPA++|+++GPB++|+++GPC++|+++GPD++|
INL      |00000000|c5e00007|88540000|00000000|
INT      |00000000|00000000|98000000|00000000|
MSK      |ffffffff|c003f7b0|06000006|00000000|
PAT1     |ffffffff|0003f7a0|9e000006|00000000|
PAT0     |00000000|c0180003|00000000|00000000|
EDG      |00000000|00000000|98000000|00000000|
PULL     |00000000|00000007|07000007|00000000|
FLAG     |00000000|00000000|00000000|00000000|
PA00:Inpt,Low    PB00:Fun1,High    PC00:Fun0,Low    PD00:Fun0,High
PA01:Inpt,Low    PB01:Fun1,High    PC01:Inpt,Low    PD01:Fun0,Low
PA02:Inpt,Low    PB02:Fun0,High    PC02:Inpt,Low    PD02:Fun0,Low
PA03:Inpt,Low    PB03:Fun0,Low     PC03:Fun0,Low    PD03:Fun0,Low
PA04:Inpt,Low    PB04:Out0,Low     PC04:Fun0,Low    PD04:Fun0,Low
PA05:Inpt,Low    PB05:Inpt,Low     PC05:Fun0,Low    PD05:Fun0,Low
PA06:Inpt,Low    PB06:Fun0,Low     PC06:Fun0,Low    PD06:Fun0,Low
PA07:Inpt,Low    PB07:Inpt,Low     PC07:Fun0,Low    PD07:Fun0,Low
PA08:Inpt,Low    PB08:Inpt,Low     PC08:Fun0,Low    PD08:Fun0,Low
PA09:Inpt,Low    PB09:Inpt,Low     PC09:Fun0,Low    PD09:Fun0,Low
PA10:Inpt,Low    PB10:Inpt,Low     PC10:Fun0,Low    PD10:Fun0,Low
PA11:Inpt,Low    PB11:Fun0,Low     PC11:Fun0,Low    PD11:Fun0,Low
PA12:Inpt,Low    PB12:Inpt,Low     PC12:Fun0,Low    PD12:Fun0,Low
PA13:Inpt,Low    PB13:Inpt,Low     PC13:Fun0,Low    PD13:Fun0,Low
PA14:Inpt,Low    PB14:Inpt,Low     PC14:Fun0,Low    PD14:Fun0,Low
PA15:Inpt,Low    PB15:Inpt,Low     PC15:Fun0,Low    PD15:Fun0,Low
PA16:Inpt,Low    PB16:Inpt,Low     PC16:Fun0,Low    PD16:Fun0,Low
PA17:Inpt,Low    PB17:Inpt,Low     PC17:Fun0,Low    PD17:Fun0,Low
PA18:Inpt,Low    PB18:Fun0,Low     PC18:Fun0,High   PD18:Fun0,Low
PA19:Inpt,Low    PB19:Fun1,Low     PC19:Fun0,Low    PD19:Fun0,Low
PA20:Inpt,Low    PB20:Fun1,Low     PC20:Fun0,High   PD20:Fun0,Low
PA21:Inpt,Low    PB21:Fun0,High    PC21:Fun0,Low    PD21:Fun0,Low
PA22:Inpt,Low    PB22:Fun0,High    PC22:Fun0,High   PD22:Fun0,Low
PA23:Inpt,Low    PB23:Fun0,High    PC23:Fun0,Low    PD23:Fun0,Low
PA24:Inpt,Low    PB24:Fun0,High    PC24:Fun0,Low    PD24:Fun0,Low
PA25:Inpt,Low    PB25:Fun0,Low     PC25:Inpt,Low    PD25:Fun0,Low
PA26:Inpt,Low    PB26:Fun0,High    PC26:Inpt,Low    PD26:Fun0,Low
PA27:Inpt,Low    PB27:Fun0,Low     PC27:Intr,High   PD27:Fun0,Low
PA28:Inpt,Low    PB28:Fun0,Low     PC28:Intr,Low    PD28:Fun0,Low
PA29:Inpt,Low    PB29:Fun0,Low     PC29:Fun0,Low    PD29:Fun0,Low
PA30:Inpt,Low    PB30:Out1,High    PC30:Fun0,Low    PD30:Fun0,Low
PA31:Inpt,Low    PB31:Out1,High    PC31:Intr,High   PD31:Fun0,Low
```

17 THE SMB I2C INTERFACE

17.1 Module function introductions

The SMB bus is a two-wire serial interface consisting of a serial data line (SDA) and a serial clock (SCL). These wires transmit information between devices connected to the bus. Each device has a unique address and can act as a "transmitter" or "receiver" according to the function of the device. When performing data transmission, the device can also be regarded as a master or slave. The master device is the device that initiates/terminates data transmission on the bus and generates a clock signal to allow the transmission. During this period of time, any addressed device is regarded as a slave device. The SMB controller is software controlled and acts as a master or a slave. However, it does not support simultaneous operation as a master and a slave.

The i2c controller is SMB. SMB supports i2c interfaces, which support 100 Kb/s and 400 Kb/s. The I2C interface can be connected to pmu, camera and configured through the i2c interface.

17.2 Driver location

Driver source path:

```
module_drivers/drivers/i2c/busses/i2c-ingenic.c
```

17.3 Device Tree Configurations

The path of the device tree:

```
module_drivers/dts/x1600.dtsi
```

i2c controller descriptions:

```
i2c0: i2c@0x10050000 {
    compatible = "ingenic,x2000-i2c";
    reg = <0x10050000 0x1000>;
    interrupt-parent = <&core_intc>;
    interrupts = <IRQ_I2C0>;
    #address-cells = <1>;
    #size-cells = <0>;
    status = "okay";
};

i2c1: i2c@0x10051000 {
    .....
};

i2c2: i2c@0x10052000 {
    .....;
};
```

```
};

i2c3: i2c@0x10053000 {
    .....
};

i2c4: i2c@0x10054000 {
    .....
};

i2c5: i2c@0x10055000 {
    .....
};
```

17.3.1 Device tree default configurations

The default compilation of the device tree will produce i2c-adapter0 devices and i2c-adapter3 devices.

17.3.2 Device Tree Custom Configurations

Users can open or close an i2c adapter device according to actual needs, and connect the corresponding i2c-client device under the i2c-adapter node of the device tree. E.g:

```
&i2c0 {
    status = "okay";
    clock-frequency = <100000>;
    timeout = <1000>;
    pinctrl-names = "default";
    pinctrl-0 = <&i2c0_pa>;

    sc031gs_0:sc031gs@0x30 {
        status = "okay";
        compatible = "sc031gs";
        reg = <0x30>;
        pinctrl-names = "default";
        pinctrl-0 = <&cim_mipi_mclk_pc>;

        resetb-gpios = <&gpa 30 GPIO_ACTIVE_LOW INGENIC_GPIO_NOBIAS>;
        vcc-en-gpios = <&gpb 12 GPIO_ACTIVE_HIGH INGENIC_GPIO_NOBIAS>;

        port {
            sc031gs_ep0:endpoint {
                remote-endpoint = <&cim_0>;
            };
        };
    };
};
```

```
};  
  
};
```

17.4 Kernel compilation configurations

The kernel configuration I2C_INGENIC, the configuration description is as follows:

```
Symbol: I2C_INGENIC [=y]  
Type : boolean  
Prompt: Ingenic SoC based on Xburst arch's I2C controler Driver support  
Location:  
-> Device Drivers  
-> I2C support  
-> I2C support (I2C [=y])  
-> I2C Hardware Bus support  
Prompt: Ingenic SoC based on Xburst arch's I2C controler Driver support  
Location:  
-> Ingenic device-drivers Configurations  
-> [I2C] Master Drivers  
Defined at drivers/i2c/busses/Kconfig
```

17.4.1 Kernel default compilation configurations

The kernel configures the I2C controller driver by default, and the configuration interface is as follows:

```
<*> I2C support  
[*] Enable compatibility bits for old user-space  
<> I2C device interface  
<*> I2C bus multiplexing support  
Multiplexer I2C Chip support --->  
[*] Autoselect pertinent helper modules  
I2C Hardware Bus support --->  
< > I2C/SMBus Test Stub  
[ ] I2C slave support  
[ ] I2C Core debugging messages  
[ ] I2C Algorithm debugging messages  
[ ] I2C Bus debugging messages
```

```
[*] Ingenic SoC based on Xburst arch's I2C controler Driver support  
[ ] controller i2c no restart mode  
(64) INGENIC I2C Controller FIFO length  
[ ] enable or disable Ingenic Soc's I2C driver debug info
```

17.4.2 Kernel custom compilation configurations

- 1) The user can remove the driver of the I2C controller according to actual needs.
- 2) You can export the device node to /dev in the user space, configure I2C_CHARDEV, and the configuration instructions are as follows:

```
Symbol: I2C_CHARDEV [=y]  
Type : tristate
```

Prompt: I2C device interface

Location:

-> Device Drivers

-> I2C support

-> I2C support (I2C [=y])

Defined at drivers/i2c/Kconfig

Depends on: I2C [=y]

The configuration interface is as follows:

```
--- I2C support
[*] Enable compatibility bits for old user-space
[*] I2C device interface
[*] I2C bus multiplexing support
    Multiplexer I2C Chip support --->
[*] Autoselect pertinent helper modules
    I2C Hardware Bus support --->
< > I2C/SMBus Test Stub
[ ] I2C slave support
[ ] I2C Core debugging messages
[ ] I2C Algorithm debugging messages
[ ] I2C Bus debugging messages
```

17.5 Module kernel differences

no difference

17.6 GPIO simulates I2C

1. First, select two groups of unused GPIO to simulate sda and scl.
2. Customize the Settings in the device tree

```
i2c00 {
    compatible = "i2c-gpio";
    gpios = <&gpc 31 GPIO_ACTIVE_LOW INGENIC_GPIO_NOBIAS>, /* sda */
           <&gpc 30 GPIO_ACTIVE_LOW INGENIC_GPIO_NOBIAS>; /* scl */
    //i2c-gpio, sda-open-drain;
    //i2c-gpio, scl-open-drain;
    i2c-gpio, delay-us = <2>; /* ~100 kHz */
    #address-cells = <1>;
    #size-cells = <0>;
};
```

3. Select from the kernel Custom compilation configuration

```
< > Synopsys DesignWare Platform
< > EMMA Mobile series I2C adapter
[*] GPIO-based bitbanging I2C
< > Imagination Technologies I2C SCB Controller
< > OpenCores I2C Controller
< > PCA9564/PCA9665 as platform device
< > Rockchip RK3xxx I2C adapter
< > Simtec Generic I2C interface
```


4. Configuration Description:

```
Symbol: I2C_GPIO [=y]
Type : tristate
Prompt: GPIO-based bitbanging I2C
Location:
-> Device Drivers
-> I2C support
-> I2C support (I2C [=y])
-> I2C Hardware Bus support
Defined at drivers/i2c/busses/Kconfig
```

17.7 Device node generation

After opening the I2C_CHARDEV option, the corresponding node will be generated in /dev:

```
ls /dev/i2c-
i2c-0 i2c-2 i2c-3 i2c-4
```

17.8 Application Instructions

17.8.1 Application source location

```
packages/example/Sample/i2c
```

17.8.2 Commands and Parameters

By default, applications are stored in /testsuite/i2c_demo:

```
cd /testsuite/
./i2c_demo
```

Points to note:

1. i2c Test demo requires external serson. The kernel uses ov5640 as the default test case. If another camera is used or the read/write address is inconsistent with serson, the test case fails.

2. Common i2c communication kernel prompts:

Error Message	instructions
I2C_TXABRT_ABRT_7B_ADDR_NOA CK	Master is in 7-bit addressing mode and the address sent was not acknowledged by any slave.
I2C_TXABRT_ABRT_10ADDR1_NOA CK	Master is in 10-bit address mode and the first 10-bit address byte was not acknowledged by any slave.
I2C_TXABRT_ABRT_10ADDR2_NOA CK	Master is in 10-bit address mode and the second address byte of the 10-bit address was not acknowledged by any

	slave.
I2C_TXABRT_ABRT_XDATA_NOACK	This is a master-mode only bit. Master has received an acknowledgement for the address, but when it sent data byte(s) following the address, it did not receive an acknowledgement from the remote slave(s).
I2C_TXABRT_ABRT_GCALL_NOACK	I2C in master mode sent a General Call and no slave on the bus acknowledged the General Call.
I2C_TXABRT_ABRT_GCALL_READ	I2C in master mode sent a General Call but the user programmed the byte following the General Call to be a read from the bus (I2CDC[9] is set to 1).
I2C_TXABRT_ABRT_HS_ACKD	Master is in High Speed mode and the High Speed Master code was acknowledged (wrong behavior).
I2C_TXABRT_SBYTE_ACKDET	The restart is disabled (RESTART bit (I2C_CON[5]) = 0) and the user is trying to use the master to transfer data in High Speed mode.
I2C_TXABRT_ABRT_HS_NORSTRT	Restart is not available, but the user wants to send data in high speed mode
I2C_TXABRT_SBYTE_NORSTRT	The restart was not available, but the user sent a start byte
I2C_TXABRT_ABRT_10B_RD_NORS TRT	Restart is not available, but the master device sends read commands under 10bit addressing
I2C_TXABRT_ABRT_MASTER_DIS	The active device is unavailable. Users perform operations on the active device
I2C_TXABRT_ARB_LOST	Loss of arbitration
I2C_TXABRT_SLVFLUSH_TXFIFO	When it comes time to receive data from the device, there is already some data in the fifo
I2C_TXABRT_SLV_ARBLOST	When sending data, the slave device loses the bus
I2C_TXABRT_SLVRD_INTX	When the master device needed to send data, it entered the data read state

18 SSI SPI controller

18.1 Module function introduction

SSI is a full-duplex synchronous serial interface that can connect to a variety of external analog-to-digital (A/D) converters, audio and telecom codecs, and other protocols that use serial transmission of data. The X1600 supports SSI Motorola's Serial Peripheral Interface (SPI) protocol.

- GPIO Function description

Name	I/O	Description
------	-----	-------------

SSI_CLK	Output	Serialbit-rateclock
SSI_CE0	Output	Firstslaveselectenable
SSI_DT	Output	Transmitdata(serialdataout)
SSI_DR	Input	Receivedata(serialdatain)

- GPIO port

```
SSI0 PA28-31 FUNCTION0
SSI0 PB02 FUNCTION2
SSI0 PB12-14 PB16-17 FUNCTION1
```

18.2 Driver source code location

The location of the driver source code:

```
module_drivers/drivers/spi
├── ingenic_spi.c
├── ingenic_spi.h
├── ingenic_slv.c
└── ingenic_slv.h
```

18.3 Device Tree Configuration

The location of the device tree:

```
module_drivers/dts/x1600.dtsi
```

SPI Controller Description:

```
spi0: spi0@0x10043000 {
    compatible = "ingenic,x1600-spi";
    reg = <0x10043000 0x1000>;
    interrupt-parent = <&core_intc>;
    interrupts = <IRQ_SSI0>;
    dmas = <&pdma INGENIC_DMA_TYPE(INGENIC_DMA_REQ_SSI0_TX)>,
          <&pdma INGENIC_DMA_TYPE(INGENIC_DMA_REQ_SSI0_RX)>;
    dma-names = "tx", "rx";
    #address-cells = <1>;
    #size-cells = <0>;
    status = "disabled";
};
```

18.3.1 Device Tree Default Configuration

The device tree turns off the SPI controller device by default.

18.3.2 Device Tree Custom Configuration

Users can open the SPI controller device according to actual needs, such as:

```
&spi0 {
    status = "disable";
    pinctrl-names = "default";
```

```

pinctrl-0 = <&spi0_pb>;
spi-max-frequency = <54000000>;
num-cs = <2>;
/* cs-gpios = <0>, <0>; */
/* cs-gpios = <&gpb 17 GPIO_ACTIVE_HIGH INGENIC_GPIO_NOBIAS>, <&gpb 16 GPIO_ACTIVE_HIGH
INGENIC_GPIO_NOBIAS>; */
ingenic,chnl = <0>;
ingenic,allow_cs_same = <1>;
ingenic,bus_num = <0>;
ingenic,has_dma_support = <0>;
ingenic,spi-src-clk = <1>;/*0.ext; 1.ssi*/

/* Add SPI interface device */
spidev: spidev@0 {
    compatible = "rohm,dh2228fv";
    reg = <0>;
    spi-max-frequency = <10000000>;
};
};

```

Some of these attributes have the following meanings :

spi-max-frequency	Maximum operating frequency of spi controller
ingenic,chnl	spi number
ingenic,has_dma_support	Whether to use dma, 1 means use dma
ingenic,spi-src-clk	Selection of device clock source
num-cs	The controller has several slice selection pins

18.3.3 Emulate the spi protocol with gpio

In order to meet some special protocol requirements, the bitbang-based gpio can also be used to simulate the spi function..

Add the following spi-gpio node to the root node of the device tree:

The device tree in kernel 4.4.94 is added as follows:

```

/{
    spi_gpio {
        status = "okay";
        compatible = "spi-gpio";
        #address-cells = <1>;
        #size-cells = <0>;

        gpio-sck = <&gpd 2 GPIO_ACTIVE_LOW INGENIC_GPIO_NOBIAS>;
        gpio-miso = <&gpd 4 GPIO_ACTIVE_LOW INGENIC_GPIO_NOBIAS>;
        gpio-mosi = <&gpd 5 GPIO_ACTIVE_LOW INGENIC_GPIO_NOBIAS>;
    };
};

```

```

        cs-gpios = <&gpd 3 GPIO_ACTIVE_LOW INGENIC_GPIO_NOBIAS>;
        num-chipselects = <1>;
        /* clients */
        spidev1: spidev1@0 {
            status = "disable";
            compatible = "rohm,dh2228fv";
            reg = <0>;
            spi-max-frequency = <500000>;
        };
    };
}

```

The device tree in kernel 5.10 is added as follows:

```

/{
    spi_gpio {
        status = "okay";
        compatible = "spi-gpio";
        #address-cells = <1>;
        #size-cells = <0>;

        sck-gpios = <&gpd 2 GPIO_ACTIVE_HIGH INGENIC_GPIO_NOBIAS>;
        miso-gpios = <&gpd 4 GPIO_ACTIVE_HIGH INGENIC_GPIO_NOBIAS>;
        mosi-gpios = <&gpd 5 GPIO_ACTIVE_HIGH INGENIC_GPIO_NOBIAS>;
        cs-gpios = <&gpd 3 GPIO_ACTIVE_HIGH INGENIC_GPIO_NOBIAS>;
        num-chipselects = <1>;
        /* clients */
        spidev1: spidev1@0 {
            status = "disable";
            compatible = "rohm,dh2228fv";
            reg = <0>;
            spi-max-frequency = <500000>;
        };
    };
}

```

According to the "Documentation/devicetree/bindings/spi/spi-gpio.yaml" file, the original attributes "gpio-sck", "gpio-miso" and "gpio-mosi" have been deprecated in kernel 5.10. ", but the actual kernel 5.10 still retains the parsing of the old attributes, and recommends the use of the new devicetree attribute.

18.4 Kernel compilation configuration

The kernel configures INGENIC_SPI, the configuration instructions are as follows:

```

Symbol: INGENIC_SPI [=y]
Type : tristate

```

Prompt: Ingenic SPI Controller

Location:

-> Device Drivers

-> SPI support (SPI [=y])

Defined at drivers/spi/Kconfig:56

Depends on: SPI [=y] && SPI_MASTER [=y] && (MACH_XBURST [=y] || MACH_XBURST2 [=n])

Selects: SPI_BITBANG [=y]

18.4.1 Kernel default build configuration

The kernel turns off the SPI controller driver by default.

18.4.2 Kernel Custom Compilation Configuration

The user can open the SPI controller compilation according to the actual needs, the configuration interface is as follows:

```
<*> Ingenic SPI Controller
[*]   Ingenic SoC SSI controller 0 for SPI Host driver
[ ]   Use GPIO CE on Ingenic SSI controller 0
```

18.4.3 Emulate the spi protocol with gpio

In order to meet some special protocol requirements, the bitbang-based gpio can also be used to simulate the spi function.

```
Device Drivers  —>
  [*] SPI support  —>
    *- Utilities for Bitbanging SPI masters
    <*> GPIO-based bitbanging SPI Master
    <*> User mode SPI device driver support /*Provision of device equipment nodes to
users*/

-> Ingenic device-drivers Configurations
  -> [SPI] Master/Slave Drivers
    < >   Ingenic SPI Controller /*Removing the ingenic ssi controller
configuration*/
```

18.5 Device Node Generation

Driver loaded successfully log

```
[ 0.298196] INGENIC SSI Controller for SPI channel 0 driver register
```

Generate device nodes:

```
/dev/spidev0.0
```

When using gpio to emulate the spi protocol, the device node spidev%d.%d* is generated, e.g.

```
/dev/spidev32766.0
```

18.6 Application Instructions

The kernel's spi driver is written based on the spi subsystem architecture.

- Applications can be tested using spi_demo.
- The spi interface can be operated by the spi driver in the kernel space.
- The spi interface can be operated in user space through spi_ioc_transfer.

18.6.1 Test spi read and write

18.6.1.1 Source code location

The kernel 4.4.94 application is in the following location:

```
Documentation/spi/spidev_test.c
```

The kernel 5.10 application is in the following location:

```
tools/spi/spidev_test.c
```

18.6.1.2 Test method

1. Short the SSI0_DT and SSI0_DR pins

```
# ./spi_test --help
./spi_test: unrecognized option '--help'
Usage: ./spi_test [-DsbdIHOLC3]
-D --device device to use (default /dev/spidev1.1)
-s --speed max speed (Hz)
-d --delay delay (usec)
-b --bpw bits per word
-l --loop loopback
-H --cpha clock phase
-O --cpol clock polarity
-L --lsb least significant bit first
-C --cs-high chip select active high
-3 --3wire SI/SO signals shared
-v --verbose Verbose (show tx buffer)
-p Send data (e.g. "1234\xde\xad")
-N --no-cs no chip select
-R --ready slave pulls low to pause
-2 --dual dual transfer
-4 --quad quad transfer
```

2. spidev_test -D/dev/spidev0.0

18.6.1.3 Test Results

```
# ./spi_test -D /dev/spidev0.0
spi mode: 0x0
bits per word: 8
max speed: 500000 Hz (500 KHz)
RX | FF FF FF FF FF FF 40 00 00 00 00 95 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF F0 0D
```

```
| .....@.....□.
```

18.6.2 Test spi nor flash

For the nor flash of the spi interface, the kernel already has corresponding driver support. If you want to use it, you only need to configure it correctly.

18.6.2.1 kernel version 4.4.94 operation

18.6.2.1.1 Source Codec Location

```
packages/example/Sample/spi/
```

18.6.2.1.2 Device Tree Configuration

Device Tree Location:

```
module_drivers/dts/halley6_v10.dts
```

Nodes that need to add nor flash within the spi_gpio node:

```
spi_gpio {
    status = "okay";
    compatible = "spi-gpio";
    #address-cells = <1>;
    #size-cells = <0>;

    gpio-sck = <&gpd 2 GPIO_ACTIVE_LOW INGENIC_GPIO_NOBIAS>;
    gpio-miso = <&gpd 4 GPIO_ACTIVE_LOW INGENIC_GPIO_NOBIAS>;
    gpio-mosi = <&gpd 5 GPIO_ACTIVE_LOW INGENIC_GPIO_NOBIAS>;
    cs-gpios = <&gpd 3 GPIO_ACTIVE_LOW INGENIC_GPIO_NOBIAS>;
    num-chipselects = <1>;

    /* clients */
    m25p80 {
        compatible = "m25p80";
        reg = <0>;
        spi-max-frequency = <1000000>;
    };
};
```

18.6.2.1.3 Kernel compilation configuration

1. Compilation:drivers/spi/spi-gpio.c

Location:Device Drivers > SPI support

Check GPIO-based bitbanging SPI Master, as shown in the following figure:


```

--- SPI support
[ ]   Debug support for SPI drivers
      *** SPI Master Controller Drivers ***
< >   Altera SPI Controller
-*.   Utilities for Bitbanging SPI masters
< >   Cadence SPI controller
<*>   GPIO-based bitbanging SPI Master
< >   IMG SPFI controller
< >   Freescale SPI controller and Aeroflex Gaisler GRLIB SPI controller
< >   OpenCores tiny SPI
< >   Rockchip SPI controller driver
< >   NXP SC18IS602/602B/603 I2C to SPI bridge
< >   Analog Devices AD-FMCOMMS1-EBZ SPI-I2C-bridge driver
< >   Xilinx SPI controller common module
< >   Xilinx ZynqMP GQSPI controller
< >   DesignWare SPI controller core support
      *** SPI Protocol Masters ***
<*>   User mode SPI device driver support
< >   Infineon TLE62X0 (for power switching)

```

2. Compilation:drivers/mtd/spi-nor/spi-nor.c

Location: Device Drivers > Memory Technology Device (MTD) support

Check SPI-NOR device support, as shown below:

```

< >   SmartMedia/xD new translation layer
< >   Log panic/oops to an MTD buffer
[ ]   Retain master device when partitioned
      RAM/ROM/Flash chip drivers --->
      Mapping drivers for chip access --->
      Self-contained MTD device drivers --->
[ ]   NAND ECC Smart Media byte order
-*.   NAND Device Support --->
< >   OneNAND Device Support ----
      LPDDR & LPDDR2 PCM memory drivers --->
<*>   SPI-NOR device support --->
<*>   Enable UBI - Unsorted block images --->

```

3. Compilation:drivers/mtd/devices/m25p80.c

Location: Device Drivers > Memory Technology Device (MTD) support > Self-contained MTD device module_drivers/drivers

Check the Support most SPI Flash chips (AT26DF, M25P, W25X, ...) , as shown in the following figure:

```

< >   Support for AT45xxx DataFlash (NEW)
<*>   Support most SPI Flash chips (AT26DF, M25P, W25X, ...)
< >   Support SST25L (non JEDEC) SPI Flash chips (NEW)
< >   Uncached system RAM
< >   Physical system RAM
< >   Test driver using RAM
< >   MTD using block device
      *** Disk-On-Chip Device Drivers ***
< >   M-Systems Disk-On-Chip G3
<*>   Ingenic series SFC driver
      Select Ingenic series SFC driver version (Use ingenic sfc d
      the SFC external memory (nor or nand) (Support ingenic sfc-

```

18.6.2.1.4 Testing

1. Make sure the nor flash hardware is connected;
2. In drivers/mtd/spi-nor/spi-nor.c, according to the model of the nor flash, look up the following table to see if the corresponding model is supported, if not, you need to configure the corresponding parameters:

```
static const struct flash_info spi_nor_ids[] = {  
    /* Atmel -- some are (confusingly) marketed as "DataFlash" */  
    { "at25fs010", INFO(0x1f6601, 0, 32 * 1024, 4, SECT_4K) },  
    { "at25fs040", INFO(0x1f6604, 0, 64 * 1024, 8, SECT_4K) },  
    .....  
    { "xm25qh128c", INFO(0x204018, 0, 4 * 1024, 4096, SECT_4K) },  
    .....  
    /* GigaDevice */  
    { "gd25q32", INFO(0xc84016, 0, 64 * 1024, 64, SECT_4K) },  
    { "gd25q64", INFO(0xc84017, 0, 64 * 1024, 128, SECT_4K) },  
    { "gd25q128", INFO(0xc84018, 0, 64 * 1024, 256, SECT_4K) },  
    .....  
    {},  
};
```

3. Recompile the kernel and burn it;
4. If you see the following message, you have been added successfully:

```
[ 1.571673] m25p80 spi32766.0: xm25qh128c (16384 Kbytes)
```

The model of the nor flash demonstrated here is xm25qh128c, with a size of 16MB, and the specific print information is the same as the parameters configured in the struct flash_info spi_nor_ids[] structure in the spi-nor.c file;

5. After entering the root file system, you can execute the following command to further confirm, if a new mtd partition is added, it means the addition is successful, here mtd0 is the partition corresponding to the newly added nor flash

```
# cat /proc/mtd  
dev:      size  erasesize  name  
mtd0: 01000000 00001000 "spi32766.0"  
mtd1: 00100000 00020000 "uboot"  
mtd2: 00800000 00020000 "kernel"  
mtd3: 06000000 00020000 "rootfs"  
mtd4: 01700000 00020000 "userdata"
```

At the same time, the corresponding device nodes will be generated in the /dev directory. The three device nodes demonstrated here are mtd0, mtd0ro, and mtddblock0, of which mtd0 and mtd0ro are character device nodes, the difference is that mtd0 is read-write, mtd0ro is read-only, and mtddblock0 is a block device node, which can be read-write, as shown in the following figure:

```
# ls /dev/mtd  
mtd0      mtd1ro    mtd3      mtd4ro    mtddblock2  
mtd0ro    mtd2      mtd3ro    mtddblock0 mtddblock3  
mtd1      mtd2ro    mtd4      mtddblock1 mtddblock4
```

-
- Next, you can use commands such as `flash_erase`, `mtd_debug`, etc. to read and write to the character device node `mtd0` generated by nor flash, and you can also format, mount, read and write to the file system for the block device node `mtdblock0` and a series of other operations, which will not be demonstrated here.

18.6.2.2 kernel version 5.10 operation

18.6.2.2.1 Source Code Location

```
packages/example/Sample/spi/
```

18.6.2.2.2 Device Tree Configuration

Device Tree Location:

```
module_drivers/dts/halley6_v10.dts
```

Need to add nor flash node inside `spi_gpio` node:

```
spi_gpio {
    status = "okay";
    compatible = "spi-gpio";
    #address-cells = <1>;
    #size-cells = <0>;

    sck-gpios = <&gpd 2 GPIO_ACTIVE_HIGH INGENIC_GPIO_NOBIAS>;
    miso-gpios = <&gpd 4 GPIO_ACTIVE_HIGH INGENIC_GPIO_NOBIAS>;
    mosi-gpios = <&gpd 5 GPIO_ACTIVE_HIGH INGENIC_GPIO_NOBIAS>;
    cs-gpios = <&gpd 3 GPIO_ACTIVE_HIGH INGENIC_GPIO_NOBIAS>;
    num-chipselects = <1>;

    /* clients */
    m25p80 {
        compatible = "m25p80";
        reg = <0>;
        spi-max-frequency = <1000>;
        #address-cells = <1>;
        #size-cells = <1>;
    };
};
```

It is recommended to use "GPIO_ACTIVE_HIGH" for the polarity of gpio in the gpio reference, otherwise it will not recognize nor flash, which will lead to the inability to register mtd devices.

18.6.2.2.3 Kernel compilation configuration

- Compilation: `drivers/spi/spi-gpio.c`
Location: Device Drivers > SPI support

Check GPIO-based bitbanging SPI Master, as shown in the following figure:

```
--- SPI support
[*]  Debug support for SPI drivers
    *** SPI Master Controller Drivers ***
-*-  SPI memory extension
    *** SPI Master Controller Drivers ***
< >  Altera SPI Controller
< >  Analog Devices AXI SPI Engine controller
-*-  Utilities for Bitbanging SPI masters
< >  Cadence SPI controller
< >  DesignWare SPI controller core support
< >  NXP Flex SPI controller
<*>  GPIO-based bitbanging SPI Master
< >  IMG SPFI controller
< >  Freescale SPI controller and Aeroflex Gaisler GRLIB SPI controller
< >  OpenCores tiny SPI
< >  Rockchip SPI controller driver
< >  NXP SC18IS602/602B/603 I2C to SPI bridge
< >  SiFive SPI controller
< >  Macronix MX25F0A SPI controller
< >  Analog Devices AD-FMCOMMS1-EBZ SPI-I2C-bridge driver
< >  Xilinx SPI controller common module
< >  Xilinx ZynqMP GQSPI controller
< >  AMD SPI controller
    *** SPI Multiplexer support ***
< >  SPI multiplexer support
    *** SPI Protocol Masters ***
<*>  User mode SPI device driver support
< >  spi loopback test framework support
< >  Infineon TLE62X0 (for power switching)
[ ]  SPI slave protocol handlers
```

2. Compilation:drivers/mtd/spi-nor/core.c

Locations: Device Drivers > Memory Technology Device (MTD) support

Check SPI-NOR device support, as shown below:

```
Self-contained MTD device drivers --->
NAND --->
LPDDR & LPDDR2 PCM memory drivers --->
<*> SPI NOR device support --->
<*> Enable UBI - Unsorted block images --->
< > HyperBus support ----
```

Since kernel 5.10 merged the spi-nor.c and m25p80.c files in kernel 4.4.94 into core.c, you only need to select SPI-NOR device support in kernel 5.10.

18.6.2.2.4 Testing

1. Make sure the nor flash hardware is connected;
2. Under the path "drivers/mtd/spi-nor/", the chip information of most of the nor flash vendors has been defined. Users can confirm whether the corresponding nor flash vendor information contains the corresponding chip according to the actual type of nor flash used, and the following vendors are currently included:

```
atmel.c  esmt.c  fujitsu.c  intel.c
spanion.c winbond.c xmc.c  catalyst.c
eon.c  everspin.c gigadevice.c ssi.c
macronix.c micron-st.c sst.c  xilinx.c
```

3. Recompile the kernel and burn it;
4. If you see the following message, you have been added successfully:

```
sbi-nor spi1.0: found XM25QH128C, expected m25p80
sbi-nor spi1.0: XM25QH128C (16384 Kbytes)
```

The model of the nor flash demonstrated here is xm25qh128c, with a size of 16MB. The specific print information is the same as the parameters configured in the struct flash_info xmc_parts[] structure in the xmc.c file.

5. After entering the root file system, you can execute the following command to further confirm, if a new mtd partition is added, it means the addition is successful, here mtd0 is the partition corresponding to the newly added nor flash.

```
# cat /proc/mtd
dev:      size  erasesize  name
mtd0: 01000000 00001000 "spi1.0"
mtd1: 00100000 00020000 "uboot"
mtd2: 00800000 00020000 "kernel"
mtd3: 06000000 00020000 "rootfs"
mtd4: 01700000 00020000 "userdata"
```

At the same time, the corresponding device nodes will be generated in the /dev directory. The three device nodes demonstrated here are mtd0, mtd0ro, and mtdblock0, of which mtd0 and mtd0ro are character device nodes, the difference is that mtd0 is read-write, mtd0ro is read-only, and mtdblock0 is a block device node, which can be read-write, as shown in the following figure:

```
# ls mtd
mtd0      mtd1ro    mtd3      mtd4ro    mtdblock2
mtd0ro    mtd2      mtd3ro    mtdblock0 mtdblock3
mtd1      mtd2ro    mtd4      mtdblock1 mtdblock4
```

6. Next, you can use commands such as flash_erase, mtd_debug, etc. to read and write to the character device node mtd0 generated by nor flash, and you can also format, mount, read and write to the file system for the block device node mtdblock0 and a series of other operations, which will not be demonstrated here.

19 SSI SLV SPI controller

19.1 Module function introduction

- SSI-SLV is a full-duplex synchronous serial slave interface that can be connected to a variety of external analog-to-digital (A/D) converters, audio and telecom codecs, and other protocols that use serial transmission of data. Supports SSI Motorola's Serial Peripheral Interface (SPI) protocol.
- GPIO Function description

Name	I/O	Description
SSI_SLV_CLK	Output	Serialbit-rateclock
SSI_SLV_CE0	Output	Firstslaveselectenable
SSI_SLV_DT	Output	Transmitdata(serialdataout)
SSI_SLV_DR	Input	Receivedata(serialdatain)

- GPIO port

<i>SSI_SLV 0 PA28-31 FUNCTION0</i>

19.2 Driver source code location

The location of the driver source code:

<i>module_drivers/drivers/spi/ ├── ingenic_slv.c └── ingenic_slv.h</i>
--

19.3 Device Tree Configuration

The location of the device tree:

<i>module_drivers/dts/x1600.dtsi</i>

SPI Controller Description:

<pre>spi_slv0: slv@0x10045000 { compatible = "ingenic,slv"; reg = <0x10045000 0x1000>; interrupt-parent = <&core_intc>; interrupts = <IRQ_SSI_SLV>; dmas = <&pdma INGENIC_DMA_TYPE(INGENIC_DMA_REQ_SLV0_TX)>, <&pdma INGENIC_DMA_TYPE(INGENIC_DMA_REQ_SLV0_RX)>; dma-names = "tx", "rx"; #address-cells = <1>; #size-cells = <0>; status = "disabled"; };</pre>

19.3.1 Device Tree Default Configuration

The device tree turns off the SPI controller device by default.

19.3.2 Device Tree Custom Configuration

Users can open the SPI controller device according to actual needs, such as:

```
&spi_slv0 {  
    status = "disable";  
    pinctrl-names = "default";  
    pinctrl-0 = <&spi_slv_pa>;  
    ingenic,has_dma_support = <1>;  
};
```

19.4 Kernel compilation configuration

The kernel configures INGENIC_SLV, the configuration instructions are as follows:

```
Symbol: INGENIC_SLV [=n]  
Type : tristate  
Prompt: Ingenic series SPI slave driver  
Location:  
-> Ingenic device-drivers Configurations  
(1) -> [SPI] Master/Slave Drivers  
Defined at module_drivers/drivers/spi/Kconfig:18  
Depends on: MACH_XBURST [=y]  
Selects: SPI_BITBANG [=n]
```

19.4.1 Kernel default build configuration

The kernel turns off the SPI_SLV controller driver by default.

19.4.2 Kernel Custom Compilation Configuration

The user can open the SPI controller compilation according to the actual needs, the configuration interface is as follows:

```
< > Ingenic SPI Controller  
< > Ingenic series SPI slave driver
```

19.5 Device Node Generation

Driver loaded successfully log

```
[ 0.298196] IINGENIC SSI slaver Controller ok!!
```

Generate device nodes:

```
/dev/spi_SIV
```

19.6 Application Instructions

19.6.1 Test spi read and write

The ssi_slv driver of the kernel is used as a slave device driver, and the master device needs to provide ce chip select and clock during testing..

19.6.1.1 Source code location

```
packages/example/Sample/spi_slv/spislaver_test.c
```

19.6.1.2 Test method

1. Short the SSI_SLV_DT and SSI_SLV_DR pins, connect SSI_SLV_CLK to SSI0_CLK, and SSI_SLV_CE to SSI0_CE.

```
# ./slvtest -help
./slvtest: invalid option -- 'h'
Usage: ./slvtest [-DsbdlHOLC3]
-p --path device to use (default /dev/spidev1.1)
-b --bpw bits per word
-l --loop loopback
-i Send data (e.g. "1234\xdel\xad")
-d --dma use dma
```

2. To configure the SSI master device driver, refer to Chapter 18 SSI Controller.
3. Execute ./slv_test -l to test loop back mode.

```
./slv_test -l
>>>bits_prt_word : 8
>>>transfer mode : cpu
>>>len: 32
[17880.956803] slv ready
[17880.962866] write data 0
[17880.966667] write data 1
[17880.969299] write data 2
[17880.972785] write data 3
[17880.975429] write data 4
[17880.978061] write data 5
[17880.981283] write data 6
[17880.983945] write data 7
[17880.986597] write data 8
[17880.989228] write data 9
[17880.992425] write data 10
[17880.995154] write data 11
[17880.997877] write data 12
[17881.000598] write data 13
[17881.003904] write data 14
[17881.006653] write data 15
```

```
[17881.009396] write data 16
[17881.012742] write data 17
[17881.015495] write data 18
[17881.018216] write data 19
[17881.021477] write data 20
[17881.024227] write data 21
[17881.026968] write data 22
[17881.029687] write data 23
[17881.033020] write data 24
[17881.035770] write data 25
[17881.038513] write data 26
[17881.041651] write data 27
[17881.044400] write data 28
[17881.047121] write data 29
[17881.049840] write data 30
[17881.053176] write data 31
```

4. Open a new terminal and execute the ssi master test program to provide clock and chip select.

```
#adb shell
#cd
#./spi_test -D /dev/spidev0.0 -O -H
```

19.6.1.3 Test Results

```
[18309.917179] write number: 32 ; reads number:32
[18309.922251] read_c[0] = 0
[18309.925002] read_c[1] = 1
[18309.927724] read_c[2] = 2
[18309.930444] read_c[3] = 3
[18309.933745] read_c[4] = 4
[18309.936498] read_c[5] = 5
[18309.939220] read_c[6] = 6
[18309.942455] read_c[7] = 7
[18309.945186] read_c[8] = 8
[18309.947886] read_c[9] = 9
[18309.950608] read_c[10] = 10
[18309.954085] read_c[11] = 11
[18309.957016] read_c[12] = 12
[18309.959918] read_c[13] = 13
[18309.963339] read_c[14] = 14
[18309.966249] read_c[15] = 15
[18309.969168] read_c[16] = 16
```

```
[18309.972594] read_c[17] = 17
[18309.975524] read_c[18] = 18
[18309.978446] read_c[19] = 19
[18309.981774] read_c[20] = 20
[18309.984703] read_c[21] = 21
[18309.987625] read_c[22] = 22
[18309.990526] read_c[23] = 23
[18309.993961] read_c[24] = 24
[18309.996891] read_c[25] = 25
[18309.999792] read_c[26] = 26
[18310.003169] read_c[27] = 27
[18310.006099] read_c[28] = 28
[18310.009000] read_c[29] = 29
[18310.012579] read_c[30] = 30
[18310.015492] read_c[31] = 31
[18310.018394] @@@@ SLV LOOP TEST OK @@@@
```

20 THE UART MODULE

20.1 Module function introductions

Universal asynchronous receiver/transmitter (UART) serial port. There are ten UARTs: all UARTs use the same programming model. Each serial port can operate in interrupt-based mode or DMA-based mode. The Universal Asynchronous Receiver/Transmitter (UART) is compatible with the 16550 industry standard and can be used as a slow infrared asynchronous interface that complies with the Infrared Data Association (IrDA) Serial Infrared Specification 1.1.

20.2 Driver location

Driver source code path:

```
module_drivers/drivers/tty/serial/ingenic_uart.h
module_drivers/drivers/tty/serial/ingenic_uart.c
```

20.3 Device Tree Configurations

The path of the device tree:

```
module_drivers/dts/x1600.dtsi
```

UART controller description:

```
uart0: serial@0x10030000 {
    compatible = "ingenic,8250-uart";
    reg = <0x10030000 0x100>;
    reg-shift = <2>;
    interrupt-parent = <&core_intc>;
    interrupts = <IRQ_UART0>;
    clocks = <&extclk 0>;
    clock-names = "uart0";
};
uart1: serial@0x10031000 {
    .....
};
.....
uart9: serial@0x10039000 {
    .....
};
```

20.3.1 Default configurations of device tree

In the board device tree halley6_v10.dts, the default compilation will generate uar2 devices, described as follows:

```

&uart0 {
    status = "disable";
    /* dma-mode; */
    pinctrl-names = "default";
    pinctrl-0 = <&uart0_pb>;
};

&uart1 {
    status = "disable";
    /* dma-mode; */
    pinctrl-names = "default";
    pinctrl-0 = <&uart1_pb>;
};

&uart2 {
    status = "okay";
    pinctrl-names = "default";
    pinctrl-0 = <&uart2_pb>;
};

&uart3 {
    status = "disable";
};

```

20.3.2 Device Tree Custom Configurations

Users can open or close a serial device according to actual needs.

20.4 Kernel compilation configurations

The kernel configuration SERIAL_INGENIC_UART, the configuration description is as follows:

```

CONFIG_SERIAL_INGENIC_UART:

If you have a machine based on a xbrust mips soc you can
enable its onboard serial port by enabling this option.

Symbol: SERIAL_INGENIC_UART [=y]
Type : tristate
Defined at module_drivers/drivers/tty/serial/Kconfig
Prompt: ingenic serial port support
Location:
    -> Ingenic device-drivers Configurations
    -> [UART] Drivers
Selects: SERIAL_CORE [=y]

```

20.4.1 Kernel default compilation configurations

The kernel is configured with UART driver by default, and the configuration interface is as follows:

```
<*> ingenic serial port support
[*]   console on ingenic soc and compatible serial port
[*]   ingenic baudrate add support greater than 1M
[*]   ingenic uart enable Magic SysRq key
```

20.4.2 Kernel custom compilation configurations

Users can remove the driver configuration according to actual needs.

20.5 Module kernel differences

no difference

20.6 Device node generation

Generate the corresponding device node after the driver is loaded successfully.

```
# ls /dev/ttyS
ttyS1 ttyS2 ttyS3
```

20.7 Application Instructions

20.7.1 Application source location

```
packages/example/sample/serial/
```

Test Methods:

1. Test demo to control uart0 and uart1 for sending and receiving communication, connect pins of uart0 and uart1, TX and RX:
2. Start the uart0 and uart1 modules in the board.dts.
3. Test demo is compiled to the file system by default, burning uart_demo for testing:

```
# uart_demo
uart /dev/ttyS0 write data[len = 7]: 0x11 0x22 0x33 0x44 0x55 0x66 0x77

*****

uart /dev/ttyS1 read data[len = 7]: 0x11 0x22 0x33 0x44 0x55 0x66 0x77

*****

uart /dev/ttyS1 write data[len = 7]: 0x11 0x22 0x33 0x44 0x55 0x66 0x77

*****

uart /dev/ttyS0 read data[len = 7]: 0x11 0x22 0x33 0x44 0x55 0x66 0x77
```

```
*****
```

```
close uart /dev/ttyS0
```

```
*****
```

```
close uart /dev/ttyS1
```

Note: During the test, the baud rates of the two uart must be consistent. Currently, the kernel supports the following baud rates:50、75、110、134、150、200、300、600、1200、1800、2400、4800、9600、19200、38400、57600、115200、230400、460800、500000、576000、921600、1000000、1152000、2000000、2500000、3000000、3500000、4000000。

21 SD/SDIO INTERFACE

21.1 Module Function Description

The MMC is a highly configurable and programmable high-performance mobile storage master controller.

The ap6212 chip used in the x1600. AP6212 is a functional module integrating wifi and bluetooth based on BCM4340A1 scheme, which includes SDIO interface for WiFi and UART/PCM interface for Bluetooth.

21.1.1 GPIO Function Description:

21.1.1.1 SDIO

Name	I/O	Funtion
SDIO	GPIO-(PB12~PB17)	FUNCTION0
WL_REG_ON	GPIO-PC28	
WL_WAKE_HOST	GPIO-PB04	

21.1.1.2 SD

Name	I/O	Funtion
MSC0_CLK	GPIO-PD00	FUNCTION0
MSC0_CMD	GPIO-PD01	FUNCTION0
MSC0_DATA	GPIO-(PD02~PD05)	FUNCTION0

21.1.1.3 EMMC

Name	I/O	Funtion
MSC0_CLK	GPIO-PD00	FUNCTION0
MSC0_CMD	GPIO-PD01	FUNCTION0
MSC0_DATA	GPIO-(PD02~PD05)	FUNCTION0

Note: The EMMC card mode is not configured by default.

21.1.2 Mapping between MSC controller names:

controll er	Base	bootrom	kernel	pm-spec	hardware -pcd
er0	13450000	msc0	msc0	msc0	msc0
er1	13460000	msc1	msc1	sdio	sdio

21.2 Driver source location

Driver source is located at:

```
module_drivers/drivers/mmc/host/  
├──ingenic_sdio.c  
└──ingenic_mmc.c
```

21.3 Device Tree Configuration

Device tree location:

```
module_drivers/dts/x1600.dtsi
```

Description of msc0 controller:

```
msc0: msc@0x13450000 {  
    compatible = "ingenic,x1600mmc";  
    reg = <0x13450000 0x10000>;  
    status = "okay";  
    interrupt-parent = <&core_intc>;  
    interrupts = <IRQ_MSC0>;  
};
```

Description of msc1 controller:

```
msc1: msc@0x13460000 {  
    compatible = "ingenic,x1600mmc";  
    reg = <0x13460000 0x10000>;  
    status = "okay";  
    interrupt-parent = <&core_intc>;  
    interrupts = <IRQ_MSC1>;  
};
```

21.3.1 Device Tree Default Configuration

In the board device tree halley6_v10.dts, the msc1 device is generated by default, as described below:

21.3.1.1 SDIO

```
&msc0 {  
    status = "disable";  
    pinctrl-names = "default", "enable", "disable";  
    pinctrl-0 = <&msc0_4bit_f0>;  
    pinctrl-1 = <&rtc32k_enable>;  
    pinctrl-2 = <&rtc32k_disable>;  
  
    cap-sd-highspeed;
```



```

        non-removable;
        max-frequency = <50000000>;
        bus-width = <4>;
        voltage-ranges = <1800 3300>;
        non-removable;

        /* special property */
        ingenic,wp-gpios = <0>;
        ingenic,cd-gpios = <0>;
        ingenic,rst-gpios = <0>;
        ingenic,removal-manual; /*removal-dontcare, removal-nonremovable, removal-
removable, removal-manual*/

        bcmhdhd_wlan: bcmhdhd_wlan {
                compatible = "android,bcmhdhd_wlan";
                ingenic,sdio-irq = <&gpb 4 IRQ_TYPE_LEVEL_HIGH INGENIC_GPIO_NOBIAS>;
                ingenic,sdio-reset = <&gpc 28 GPIO_ACTIVE_LOW INGENIC_GPIO_NOBIAS>;
        };

```

21.3.1.2 SD

```

&msc1 {
        status = "okay";
        pinctrl-names = "default";
        pinctrl-0 = <&msc1_4bit>;
        cap-sd-highspeed;
        cd-inverted;
        max-frequency = <50000000>;
        bus-width = <4>;
        voltage-ranges = <1800 3300>;
        ingenic,sdio_clk = <0>;
        /* special property */
        ingenic,wp-gpios = <0>;
        ingenic,cd-gpios = <0>;
        ingenic,rst-gpios = <0>;
        ingenic,removal-manual; /*removal-dontcare, removal-nonremovable, removal-
removable, removal-manual*/
};

```

21.3.1.3 EMMC

```

&msc1 {
        status = "disable";
        /*mmc-hs200-1_8v;*/
        cap-mmc-highspeed;

```

```

    non-removable;
    max-frequency = <50000000>;
    bus-width = <4>;
    non-removable;
    voltage-ranges = <1800 3300>;

    /* special property */
    ingenic,wp-gpios = <0>;
    ingenic,cd-gpios = <0>;
    ingenic,rst-gpios = <0>;
};

```

Note: The EMMC card mode is not configured by default.

21.3.2 Device Tree User-defined Configuration

21.3.2.1 SDIO

Attribute Name	Instructions
● cap-sd-highspeed	Configure the SDIO to support the high speed mode
● max-frequency	This parameter can be configured to specify the maximum frequency in the current mode
● bus-width	The data bus width can be 1bit, 4bit, or 8bit
● voltage-ranges	Specified voltage range
● non-removable	Configure the property as non-removable card
● ingenic,sdio_clk	Option is used to set manual debugging of MSC tx phase 位
● keep-power-in-suspend	Hibernate to maintain power supply
● ingenic,wp-gpios	Write protection
● ingenic,cd-gpios	Check of card
● ingenic,rst-gpios	Hardware reset
● ingenic,removal-manual	removal-dontcare, removal-nonremovable, removal-removable, removal-manual
● ingenic,sdio-irq	Wifi interrupt
● ingenic,sdio-reset	Wifi reset

21.3.2.2 SD

Attribute Name	Instructions
● cap-sd-highspeed	Configure the SD to support the high speed mode
● max-frequency	This parameter can be configured to specify the maximum frequency in the current mode
● bus-width	The data bus width can be 1bit, 4bit, or 8bit
● voltage-ranges	Specified voltage range
● ingenic,wp-gpios	Configure the CD pin for SD card detection

● ingneic, cd-gpios	Check of card
● ingenic, rst-gpios	Hardware reset
● ingenic, removal-manual	removal-dontcare, removal-nonremovable, removal-removable, removal-manual

21.3.2.3 EMMC

Attribute Name	Instructions
● mmc-hs200-1_8v	Configure emmc to support hs200 transport mode
● cap-mmc-highspeed	Configure emmc to support highspeed
● non-removable	Configure the emmc property as a non-removable card
● max-frequency	This parameter can be configured to specify the maximum frequency in the current mode
● enable_cpm_rx_tuning	This option is used to set the manual tuning of MSC rx phase and the default automatic tuning of the controller
● enable_cpm_tx_tuning	Option is used to set manual debugging of MSC tx phase
● bus-width	The data bus width can be 1bit, 4bit, or 8bit
● voltage-ranges	Specified voltage range
● ingenic, wp-gpios	Write protection
● ingneic, cd-gpios	Check of card
● ingenic, rst-gpios	Hardware reset

21.4 Kernel compilation configuration

21.4.1 Kernel Block & wireless device Basic compilation Configuration

21.4.1.1 Block device Basic Compilation Configuration

```

Device Drivers —>
— MMC/SD/SDIO card support
<*> HW reset support for eMMC
<*> Simple HW reset support for MMC
<*> MMC block device driver
(16) Number of minors per block device
< > SDIO UART/GPS class support
< > MMC host test driver
*** MMC/SD/SDIO Host Controller Drivers ***
[ ] MMC host drivers debugging
-* Secure Digital Host Controller Interface support
< > SDHCI platform and OF driver helper
< > MMC/SD/SDIO over SPI
< > Synopsys DesignWare Memory Card Interface
< > Ingenic JZ47xx SD/Multimedia Card Interface support

```

< > VUB300 USB to SDIO/SD/MMC Host Controller support

21.4.1.2 Wireless device Basic Compilation Configuration

Device Drivers

— Network device support

[*] Network core driver support

[*] Wireless LAN —>

21.4.2 EMMC/SD/SDIO Controller Driver Configuration

Start by configuring INGENIC_MMC

Symbol: INGENIC_MMC [=y]

Type : tristate

Prompt: Ingenic (XBurst) MMC/SD Card Controller (MSC) support

Location:

-> Ingenic device-drivers Configurations

-> [MSC] (eMMC/SD/SDIO) and Device Drivers

21.4.2.1 Configuring INGENIC_MMC_MMC0 using Emmc/SD Drivers

Symbol: INGENIC_MMC_MMC0 [=y]

Type : boolean

Prompt: INGENIC_MMC MMC0

Location:

-> Ingenic device-drivers Configurations

-> [MSC] (eMMC/SD/SDIO) and Device Drivers

-> Ingenic (XBurst) MMC/SD Card Controller (MSC) support (INGENIC_MMC [=y])

21.4.2.2 Configuring INGENIC_MMC_MMC1 Using the sdio Driver

Symbol: INGENIC_MMC_MMC1 [=y]

Type : boolean

Prompt: INGENIC_MMC MMC1

Location:

-> Ingenic device-drivers Configurations

-> [MSC] (eMMC/SD/SDIO) and Device Drivers

-> Ingenic (XBurst) MMC/SD Card Controller (MSC) support (INGENIC_MMC [=y])

21.4.3 wifi Driver Configuration

Symbol: BCMDDHD [=y]

```

Type : tristate
Prompt: Broadcom FullMAC wireless cards support
Location:
-> Ingenic device-drivers Configurations
-> SDIO-WIFI drivers
Defined at module_drivers/drivers/net/wireless/bcmdhd/Kconfig |

```

21.4.4 Kernel Customized Compilation Configuration

Kernel configuration name	Instructions
MMC_BLOCK_MINORS	Maximum number of partitions per block device
BCMDHD_FW_PATH	The firmware path of the wifi module is configured
BCMDHD_NVRAM_PATH	The firmware path of the wifi module is configured

21.5 Kernel version differences

21.6 Device node generation

21.6.1 SDIO

Driver loading succeeded:

```

[ 1.124644] Dongle Host Driver, version 1.363.59.144.11 (r)
[ 1.125120] Register interface [wlan0] MAC: 70:4a:0e:69:5e:ba
[ 1.125195] ]dhd_module_init: Exit err=0

```

21.6.2 SD

The driver is loaded successfully:

```

[ 1.575803] mmc0: new SDHC card at address e624
[ 1.593001] mmcblk0: mmc0:e624 SU04G 3.69 GiB
[ 1.601920] Alternate GPT is invalid, using primary GPT.
[ 1.607432] mmcblk0: p1 p2 p3 p4 p5 p6 p7

```

Generating device node:

```

/dev/mmcblk0p1~p7 /*Indicates the sd partition*/

```

21.7 Application Instructions

21.7.1 EMMC/SD test method

21.7.1.1 Write Tests

```
# dd if=/dev/zero of=/dev/mmcblk0 bs=1M count=100 conv=fsync
```

21.7.1.2 Reading test

```
# sync; echo 3 > /proc/sys/vm/drop_caches
# time dd if=/dev/mmcblk0 of=/dev/null bs=1M count=100
```

21.7.2 SDIO test method

21.7.2.1 Configuring the Network

1. /etc/wpa_supplicant

```
# cat /etc/wpa_supplicant.conf
```

```
ctrl_interface=/var/run/wpa_supplicant
update_config=1
country=GB"

network={
    ssid="Guest" /*Connecting to a WiFi account*/
    psk="ingenic*123" /*Wifi Connection Password*/
    bssid=
    priority=1
}
```

2. Implement wifi_up.sh

```
# ifconfig wlan0 up
# wifi_up.sh
```

```
[ 13.285811] dhcd_open: Enter 843c4800
[ 13.290329] dhcd_conf_read_config: Ignore config file /firmware/config.txt
[ 13.297356] Final fw_path=/firmware/fw_bcm43456c5_ag.bin
[ 13.302873] Final nv_path=/firmware/nvram_ap6256.txt
[ 13.308002] Final clm_path=/firmware/clm_bcm43456c5.blob
[ 13.313152] Final conf_path=/firmware/config.txt
[ 13.317926] dhcd_set_bus_params: set use_rxchain 0
[ 13.322804] dhcd_set_bus_params: set txglomsize 36
[ 13.328047] dhcd_os_open_image: /firmware/fw_bcm43456c5_ag.bin (579388 bytes) open success
```

```
[ 13.409630] dhd_os_open_image: /firmware/nvram_ap6256.txt (2440 bytes) open success
[ 13.516230] bcmsdh_oob_intr_register: HW_OOB enabled
[ 13.521376] bcmsdh_oob_intr_register OOB irq=72 flags=0x4
[ 13.556873] Firmware up: op_mode=0x0005, MAC=c0:84:7d:31:c8:c8
[ 13.640263] dhd_txglom_enable: enable 1
[ 13.748733] dhd_open: Exit ret=0
# udhpcp: started, v1.26.2
Successfully initialized wpa_supplicant
[ 16.320669] Connecting with 8c:0c:90:98:47:cd ssid "wifi ssid", len (5) channel=153
[ 16.367807] wl_bss_connect_done succeeded with 8c:0c:90:98:47:cd
[ 16.401524] wl_bss_connect_done succeeded with 8c:0c:90:98:47:cd
udhpcp: sending select for 10.4.30.146
adding dns 219.141.136.10
adding dns 202.106.0.20
```

3. Run the ping command to connect the network

```
# ping www.baidu.com
```

```
PING www.baidu.com (220.181.38.150): 56 data bytes
64 bytes from 220.181.38.150: seq=0 ttl=53 time=14.347 ms
64 bytes from 220.181.38.150: seq=1 ttl=53 time=9.873 ms
64 bytes from 220.181.38.150: seq=2 ttl=53 time=12.889 ms
```

21.7.2.2 airkiss distribution network test

AirKissdemo is a LAN discovery and LAN communication technology for Wi-Fi devices.

1. AirKissdemo Network configuration

Note: The following operations must be configured in WIFI(external network) environment

2. Run the airkiss command

```
# airkiss
```

```
[ 113.669340] dhd_open: Enter 842f1000
[ 113.677706] Dongle Host Driver, version 1.579.77.41.26 (r-20200429-2)
[ 113.695902] ===== PULL WL_REG_ON(-1) HIGH! =====
[ 114.122604] F1 signature read @0x18000000=0x15294345
[ 114.130573] F1 signature OK, socitype:0x1 chip:0x4345 rev:0x9 pkg:0x2
[ 114.137706] DHD: dongle ram size is set to 819200(orig 819200) at 0x198000
[ 114.153611] [dhd] dhd_conf_read_config : Ignore config file /firmware/config.txt
[ 114.161413] [dhd] dhd_conf_set_path_params : Final
fw_path=/firmware/fw_bcm43456c5_ag.bin
[ 114.169851] [dhd] dhd_conf_set_path_params : Final nv_path=/firmware/nvram_ap6256.txt
[ 114.177960] [dhd] dhd_conf_set_path_params : Final
```

```

clm_path=/firmware/clm_bcm43456c5_ag_blob

[ 114.186695] [dhd] dhd_conf_set_path_params : Final conf_path=/firmware/config.txt
[ 114.194942] dhd_os_open_image: /firmware/fw_bcm43456c5_ag.bin (579388 bytes) open
success
[ 114.266737] dhd_os_open_image: /firmware/nvram_ap6256.txt (2440 bytes) open success
[ 114.283245] dhdsdio_write_vars: Download, Upload and compare of NVRAM succeeded.
[ 114.580570] [dhd-wlan0] wl_android_wifi_on : Success
Easy setup target library v4.0.0
state: 0 → 1
state: 1 → 3
state: 3 → 5
ssid: JZ_SW /*The network account configured on wechat terminal*/
password: jz_sw##1135 /*The network password configured on wechat*/
/etc/wpa_supplicant.conf create successfully!
random: 0xc1
time elapsed: 0s
# udhpcp: started, v1.31.1
Successfully initialized wpa_supplicant
[ 118.985525] [dhd-wlan0] wl_run_escan : LEGACY_SCAN sync ID: 0, bssidx: 0
wlan0: Trying to associate with c4:01:7c:78:c3:dd (SSID='JZ_SW' freq=5765 MHz)
[ 121.338272] [dhd-wlan0] wl_cfg80211_connect : Connecting with c4:01:7c:78:c3:dd ssid
"JZ_SW", len (5), sec=wpa2psk/mfpn/tkipaes, channel=153
[ 121.402046] [dhd-wlan0] wl_ext_iapsta_event : [S] Link UP with c4:01:7c:78:c3:dd
[ 121.409697] [dhd-wlan0] wl_notify_connect_status : wl_bss_connect_done succeeded with
c4:01:7c:78:c3:dd
wlan0: Associated with c4:01:7c:78:c3:dd
wlan0: CTRL-EVENT-SUBNET-STATUS-UPDATE status=0
wlan0: WPA: Key negotiation completed with c4:01:7c:78:c3:dd [PTK=CCMP GTK=TKIP]
wlan0: CTRL-EV[ 121.451231] [dhd-wlan0] wl_notify_connect_status : wl_bss_connect_done
succeeded with c4:01:7c:78:c3:dd vndr_oui: 00-90-4C 00-13-92
ENT-CONNECTED - Connection to c4:01:7c:78:c3:dd completed [id=0 id_str=]
udhpcp: sending select for 10.10.30.203
udhpcp: lease of 10.10.30.203 obtained, lease time 86400
adding dns 192.168.1.2

```

3. Run the ping command to connect the network

```
# ping www.baidu.com
```

```

PING www.baidu.com (220.181.38.150): 56 data bytes
64 bytes from 220.181.38.150: seq=0 ttl=53 time=14.347 ms
64 bytes from 220.181.38.150: seq=1 ttl=53 time=9.873 ms

```


21.7.2.3 How to use AP mode

1. Configuration file in AP mode (/etc/hostapd.conf)

```
interface=wlan0
driver=nl80211
ssid=ingenic
channel=1
hw_mode=g
macaddr_acl=0
auth_algs=1
ignore_broadcast_ssid=0
#wpa=2
#wpa_passphrase=12345678
#wpa_key_mgmt=WPA-PSK
#wpa_pairwise=TKIP
#rsn_pairwise=CCMP
```

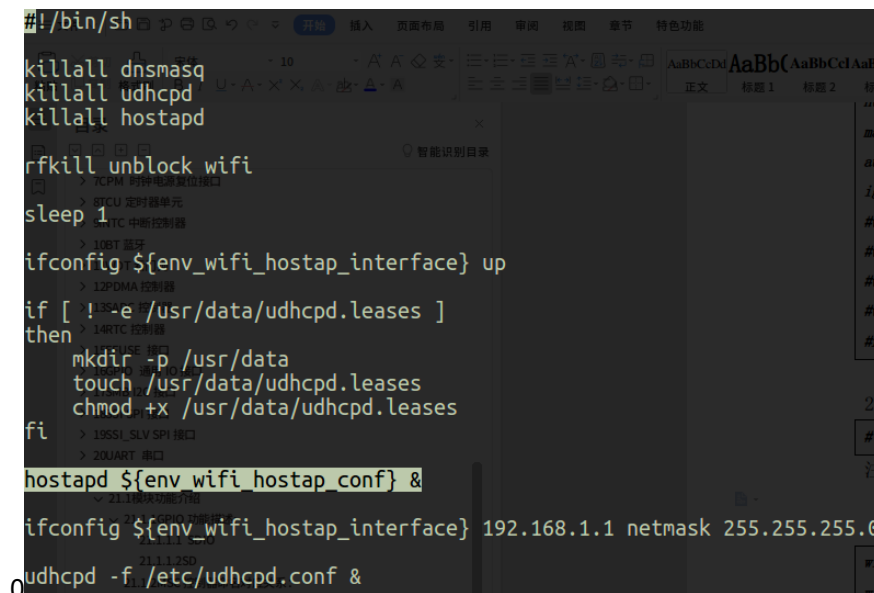
2. Enter the AP mode

```
# wifi_ap_mode_start.sh
```

The script path is

```
/usr/bin/wifi_ap_mode_start.sh
```

Note: The wifi_ap_mode_start.sh script needs to be modified before using it, as shown below:



```
#!/bin/sh
killall dnsmasq
killall udhcpd
killall hostapd
rfkill unblock wifi
sleep 1
ifconfig ${env_wifi_hostap_interface} up
if [ ! -e /usr/data/udhcpd.leases ]
then
    mkdir -p /usr/data
    touch /usr/data/udhcpd.leases
    chmod +x /usr/data/udhcpd.leases
fi
hostapd ${env_wifi_hostap_conf} &
ifconfig ${env_wifi_hostap_interface} 192.168.1.1 netmask 255.255.255.0
udhcpd -f /etc/udhcpd.conf &
```

```
wlan0: interface state UNINITIALIZED->ENABLED
wlan0: AP-ENABLED
```

3. Run ifconfig to view the connection information

```
# ifconfig
```

```

lo Link encap:Local Loopback
inet addr:127.0.0.1 Mask:255.0.0.0
UP LOOPBACK RUNNING MTU:65536 Metric:1
RX packets:0 errors:0 dropped:0 overruns:0 frame:0
TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1
RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)

wlan0 Link encap:Ethernet HWaddr C0:84:7D:6A:F1:CD
inet addr:192.168.1.1 Bcast:192.168.1.255 Mask:255.255.255.0
UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
RX packets:46 errors:0 dropped:21 overruns:0 frame:0
TX packets:3 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1000
RX bytes:6509 (6.3 KiB) TX bytes:744 (744.0 B)

```

4. Use your phone (or computer) to connect to wifi

The connection SSID is "ingenic" corresponding to the configuration file ssid=ingenic, and the password can be specified using wpa_passphrase=12345678 (no password was set in this test).

The following information is displayed if the connection is successful:

```

[ 515.831824] [dhd-wlan0] wl_ext_iapsta_event : [A] connected device 00:26:c6:58:50:54
[ 515.839828] [dhd-wlan0] wl_notify_connect_status_ap : connected device 00:26:c6:58:50:54
[ 515.851919] [dhd] CFG80211-ERROR) wl_cfg80211_change_station : WLC_SCB_AUTHORIZE
sta_flags_mask not set
udhcpd: sending OFFER to 192.168.1.2
udhcpd: sending ACK to 192.168.1.2

```

5. If the connection is successful, Ping the network

```
# ping 192.168.1.2
```

```

PING 192.168.1.2 (192.168.1.2): 56 data bytes
64 bytes from 192.168.1.2: seq=0 ttl=64 time=8.148 ms
64 bytes from 192.168.1.2: seq=1 ttl=64 time=15.411 ms
64 bytes from 192.168.1.2: seq=2 ttl=64 time=12.505 ms
64 bytes from 192.168.1.2: seq=3 ttl=64 time=7.654 ms
64 bytes from 192.168.1.2: seq=4 ttl=64 time=7.505 ms

```


22 USB OTG Controller Interface

22.1 Module Function Introduction

Universal Serial Bus (USB) is a new data communication method that gradually replaces other interface standards. It was jointly developed by Intel, Compaq, Digital, IBM, Microsoft, NEC, Northern Telecom and other computer companies and communication companies in 1995, and gradually formed an industry standard.

USB bus, as a high-speed serial bus, its extremely high transmission speed can meet the requirements of the application environment for high-speed data transmission, and it also has simple power supply (bus power supply), convenient installation and configuration (plug and play and hot plug are supported), simple expansion port (127 peripherals can be expanded at most through the hub), and diversified transmission modes (4 transmission modes), And good compatibility (backward compatibility after product upgrade).

USB OTG controller realizes serial data exchange between USB host and various portable peripherals that can be accessed at the same time.

22.2 Driver source code location

Location of driver source code:

```
drivers/usb/dwc2$ tree
├── core.c
├── core.h
├── core_intr.c
├── debugfs.c
├── debug.h
├── gadget.c
├── hcd.c
├── hcd_dma.c
├── hcd.h
├── hcd_intr.c
├── hcd_queue.c
├── hw.h
├── Kconfig
├── Makefile
├── pci.c
└── platform.c
```

22.3 Device Tree Configuration

dts file path:

```
module_drivers/dts/x1600.dtsi
```

Device Tree Description:

```
otg: otg@0x13500000 {
    compatible = "ingenic,dwc2-hsotg";
    reg = <0x13500000 0x40000>;
    interrupt-parent = <&core_intc>;
    interrupts = <IRQ_OTG>;
    ingenic,usbphy=<&otg_phy>;
    status = "disabled";
};

otg_phy: otg_phy {
    #address-cells = <1>;
    #size-cells = <1>;
    compatible = "ingenic,innophy-x1600", "syscon";
    reg = <0x10000000 0x100 0x10078000 0x110>;
};
```

22.3.1 Device Tree Default Configuration

The default compilation of the device tree will generate otg and otg_phy device, The configuration in halley6_v10.dts is as follows:

```
&otg {
    g-use-dma;
    dr_mode = "otg"; // host, peripheral, otg
    status = "okay";
};

&otg_phy {
    ingenic,drvbus-gpio = <&gpb 27 GPIO_ACTIVE_HIGH INGENIC_GPIO_NOBIAS>;
    status = "okay";
};
```

22.3.2 Device Tree Custom Configuration

The user can close the node according to actual requirements, or perform the following configurations:

Attribute Name	Explain
● ingenic,id-dete-gpio	GPIO for usb id detection
● ingenic,vbus-dete-gpio	GPIO for usb vbus detection
● Ingenic,drvbus-gpio	GPIO for usb drv vbus

22.4 Kernel compilation configuration

Different configurations need to be selected according to different USB functions. The USB composite function can support multiple device functions at the same time.

USB as host:

Support functions	Explain
● mass storage	As a host, it can identify mass storage devices
● usb camera	As a host, USB camera device can be recognized
● hid	As a host, it can identify human-computer interaction devices, such as mouse...

USB as device:

Support functions	Explain
● mass storage	USB as a mass storage device
● adb (Android Debug Bridge)	For USB debug function
● hid	USB as human-computer interaction device, such as mouse and keyboard
● uvc (usb video class)	USB as a video device
● uac1.0 (usb audio class1.0)	USB as audio device
● printer	USB as a printer device
● rndis	USB as network card device
● serial	USB as serial port device

22.4.1 Kernel default compilation configuration

USB host default support mass storage、usb camera、hid.

USB device composite default support mass storage、adb、uvc、uac1.0、printer、rndis can be used at the same time.

22.4.2 Kernel custom compile configuration

22.4.2.1 USB controller mode configuration

The USB controller can be configured as host only, device only or otg mode through the kernel configuration or device tree. The configuration method is as follows:

Configure the USB controller to host only mode through the kernel configuration :

```
Symbol: USB_DWC2_HOST [=n]
Type : boolean
Prompt: Host only mode
Location:
-> Device Drivers
-> USB support (USB_SUPPORT [=y])
-> DesignWare USB2 DRD Core Support (USB_DWC2 [=y])
-> DWC2 Mode Selection (<choice> [=y])
```

Defined at drivers/usb/dwc2/Kconfig:24

Depends on: <choice> && (USB [=y]=y || USB_DWC2 [=y]=m && USB [=y])

Configure the USB controller to device only mode through the kernel configuration:

Symbol: USB_DWC2_PERIPHERAL [=n]

Type : boolean

Prompt: Gadget only mode

Location:

-> Device Drivers

-> USB support (USB_SUPPORT [=y])

-> DesignWare USB2 DRD Core Support (USB_DWC2 [=y])

-> DWC2 Mode Selection (<choice> [=y])

Defined at drivers/usb/dwc2/Kconfig:34

Depends on: <choice> && (USB_GADGET [=y]=y || USB_GADGET [=y]=USB_DWC2 [=y])

Configure USB controller to otg mode through the kernel configuration:

Symbol: USB_DWC2_DUAL_ROLE [=y]

Type : boolean

Prompt: Dual Role mode

Location:

-> Device Drivers

-> USB support (USB_SUPPORT [=y])

-> DesignWare USB2 DRD Core Support (USB_DWC2 [=y])

-> DWC2 Mode Selection (<choice> [=y])

Defined at drivers/usb/dwc2/Kconfig:43

Depends on: <choice> && (USB [=y]=y && USB_GADGET [=y]=y || USB_DWC2 [=y]=m && USB [=y] && USB_GADGET [=y])

Or configuring USB controller mode in halley6_v10.dts:

```
&otg {
    g-use-dma;
    dr_mode = "otg"; //host, peripheral, otg
    status = "okay";
};
```

22.4.2.2 USB host kernel configuration

22.4.2.2.1 USB host Mass Storage

(1)VFAT (Windows-95) fs support

Symbol: VFAT_FS [=y]

```

Type : tristate
Prompt: VFAT (Windows-95) fs support
Location:
-> File systems
-> DOS/FAT/NT Filesystems
Defined at fs/fat/Kconfig:60
Depends on: BLOCK [=y]
Selects: FAT_FS [=y]

(2)Codepage 437 (United States, Canada)
Symbol: NLS_CODEPAGE_437 [=y]
Type : tristate
Prompt: Codepage 437 (United States, Canada)
Location:
-> File systems
-> Native language support (NLS [=y])
Defined at fs/nls/Kconfig:39
Depends on: NLS [=y]

(3)NLS ISO 8859-1 (Latin 1; Western European Languages)
Symbol: NLS_ISO8859_1 [=y]
Type : tristate
Prompt: NLS ISO 8859-1 (Latin 1; Western European Languages)
Location:
-> File systems
-> Native language support (NLS [=y])
Defined at fs/nls/Kconfig:318
Depends on: NLS [=y]

(4)SCSI disk support
Symbol: BLK_DEV_SD [=y]
Type : tristate
Prompt: SCSI disk support
Location:
-> Device Drivers
-> SCSI device support
Defined at drivers/scsi/Kconfig:73
Depends on: SCSI [=y]

(5)Symbol: USB_STORAGE [=y]
Type : tristate
Prompt: USB Mass Storage support

```


Location:
 -> Device Drivers
 -> USB support (USB_SUPPORT [=y])
 -> Support for Host-side USB (USB [=y])
 Defined at drivers/usb/storage/Kconfig:8
 Depends on: USB_SUPPORT [=y] && USB [=y] && SCSI [=y]

22.4.2.2.2 USB host camera

Symbol: USB_VIDEO_CLASS [=y]
Type : tristate
Prompt: USB Video Class (UVC)
Location:
 -> Device Drivers
 -> Multimedia support (MEDIA_SUPPORT [=y])
 -> Media USB Adapters (MEDIA_USB_SUPPORT [=y])
 Defined at drivers/media/usb/uvc/Kconfig:1
 Depends on: USB [=y] && MEDIA_SUPPORT [=y] && MEDIA_USB_SUPPORT [=y] && MEDIA_CAMERA_SUPPORT [=y] && VIDEO_V4L2 [=y]
 Selects: VIDEOBUF2_VMALLOC [=n]

22.4.2.2.3 USB host hid mouse

(1)USB HIDBP Mouse (simple Boot) support
Symbol: USB_MOUSE [=y]
Type : tristate
Prompt: USB HIDBP Mouse (simple Boot) support
Location:
 -> Device Drivers
 -> HID support
 -> USB HID support
 -> USB HID Boot Protocol module_drivers/drivers
 Defined at drivers/hid/usbhid/Kconfig:66
 Depends on: USB_HID [=n]!y && EXPERT [=y] && USB [=y] && INPUT [=y]

(2)Mouse interface
Symbol: INPUT_MOUSEDEV [=y]
Type : tristate
Prompt: Mouse interface
Location:
 -> Device Drivers

```
-> Input device support
-> Generic input layer (needed for keyboard, mouse, ...) (INPUT [=y])
Defined at drivers/input/Kconfig:95
Depends on: !UML && INPUT [=y]
```

22.4.2.2.4 USB host rndis

```
Symbol: USB_USBNET [=y]
Type : tristate
Prompt: Multi-purpose USB Networking Framework
Location:
-> Device Drivers
-> Network device support (NETDEVICES [=y])
-> USB Network Adapters (USB_NET_DRIVERS [=y])
Defined at drivers/net/usb/Kconfig:121
Depends on: NETDEVICES [=y] && USB_NET_DRIVERS [=y]
Selects: MII [=y]
Selected by: USB_NET_RNDIS_WLAN [=n] && NETDEVICES [=y] && WLAN [=y] && USB [=y] && CFG80211 [=y]
```

22.4.2.3 USB gadget kernel configuration

22.4.2.3.1 USB device mass storage

```
Symbol: USB_CONFIGFS_MASS_STORAGE [=y]
Type : boolean
Prompt: Mass storage
Location:
-> Device Drivers
-> USB support (USB_SUPPORT [=y])
-> USB Gadget Support (USB_GADGET [=y])
-> USB Gadget Drivers (<choice> [=y])
-> USB functions configurable through configfs (USB_CONFIGFS [=y])
Defined at drivers/usb/gadget/Kconfig:338
Depends on: <choice> && USB_CONFIGFS [=y] && BLOCK [=y]
Selects: USB_F_MASS_STORAGE [=y]
```

22.4.2.3.2 USB device adb

```

Symbol: USB_CONFIGFS_F_FS [=y]
Type : boolean
Prompt: Function filesystem (FunctionFS)
Location:
-> Device Drivers
-> USB support (USB_SUPPORT [=y])
-> USB Gadget Support (USB_GADGET [=y])
-> USB Gadget Drivers (<choice> [=y])
-> USB functions configurable through configfs (USB_CONFIGFS [=y])
Defined at drivers/usb/gadget/Kconfig:362
Depends on: <choice> && USB_CONFIGFS [=y]
Selects: USB_F_FS [=y]

```

22.4.2.3.3 USB device hid

```

Symbol: USB_CONFIGFS_F_HID [=y]
Type : boolean
Prompt: HID function
Location:
-> Device Drivers
-> USB support (USB_SUPPORT [=y])
-> USB Gadget Support (USB_GADGET [=y])
-> USB Gadget Drivers (<choice> [=y])
-> USB functions configurable through configfs (USB_CONFIGFS [=y])
Defined at drivers/usb/gadget/Kconfig:419
Depends on: <choice> && USB_CONFIGFS [=y]
Selects: USB_F_HID [=n]

```

22.4.2.3.4 USB device uvc

```

(1)USB Video Class (UVC)
Symbol: USB_VIDEO_CLASS [=y]
Type : tristate
Prompt: USB Video Class (UVC)
Location:
-> Device Drivers
-> Multimedia support (MEDIA_SUPPORT [=y])
-> Media USB Adapters (MEDIA_USB_SUPPORT [=y])
Defined at drivers/media/usb/uvc/Kconfig:1
Depends on: USB [=y] && MEDIA_SUPPORT [=y] && MEDIA_USB_SUPPORT [=y] && MEDIA_CAMERA_SUPPORT [=y] && VIDEO_V4L2 [=y]

```

Selects: VIDEOBUF2_VMALLOC [=n]

(2)USB Webcam function

Symbol: USB_CONFIGFS_F_UVC [=y]

Type : boolean

Prompt: USB Webcam function

Location:

-> Device Drivers

-> USB support (USB_SUPPORT [=y])

-> USB Gadget Support (USB_GADGET [=y])

-> USB Gadget Drivers (<choice> [=y])

-> USB functions configurable through configs (USB_CONFIGFS [=y])

Defined at drivers/usb/gadget/Kconfig:429

Depends on: <choice> && USB_CONFIGFS [=y] && VIDEO_DEV [=y]

Selects: VIDEOBUF2_VMALLOC [=y] && USB_F_UVC [=n]

22.4.2.3.5 USB device remote NDIS

Symbol: USB_CONFIGFS_RNDIS [=y]

Type : boolean

Prompt: RNDIS

Location:

-> Device Drivers

-> USB support (USB_SUPPORT [=y])

-> USB Gadget Support (USB_GADGET [=y])

-> USB Gadget Drivers (<choice> [=y])

-> USB functions configurable through configs (USB_CONFIGFS [=y])

Defined at drivers/usb/gadget/Kconfig:297

Depends on: <choice> && USB_CONFIGFS [=y] && NET [=y]

Selects: USB_U_ETHER [=n] && USB_F_RNDIS [=n]

22.4.2.3.6 USB device serial

Symbol: USB_CONFIGFS_SERIAL [=y]

Type : boolean

Prompt: Generic serial bulk in/out

Location:

-> Device Drivers

-> USB support (USB_SUPPORT [=y])

-> USB Gadget Support (USB_GADGET [=y])

```

-> USB Gadget Drivers (<choice> [=y])
-> USB functions configurable through configs (USB_CONFIGFS [=y])
Defined at drivers/usb/gadget/Kconfig:241
Depends on: <choice> && USB_CONFIGFS [=y] && TTY [=y]
Selects: USB_U_SERIAL [=n] && USB_F_SERIAL [=n]

Symbol: USB_CONFIGFS_ACM [=y]
Type : boolean
Prompt: Abstract Control Model (CDC ACM)
Location:
-> Device Drivers
-> USB support (USB_SUPPORT [=y])
-> USB Gadget Support (USB_GADGET [=y])
-> USB Gadget Drivers (<choice> [=y])
-> USB functions configurable through configs (USB_CONFIGFS [=y])
Defined at drivers/usb/gadget/Kconfig:250
Depends on: <choice> && USB_CONFIGFS [=y] && TTY [=y]
Selects: USB_U_SERIAL [=n] && USB_F_ACM [=n]

Symbol: USB_CONFIGFS_OBEX [=y]
Type : boolean
Prompt: Object Exchange Model (CDC OBEX)
Location:
-> Device Drivers
-> USB support (USB_SUPPORT [=y])
-> USB Gadget Support (USB_GADGET [=y])
-> USB Gadget Drivers (<choice> [=y])
-> USB functions configurable through configs (USB_CONFIGFS [=y])
Defined at drivers/usb/gadget/Kconfig:260
Depends on: <choice> && USB_CONFIGFS [=y] && TTY [=y]
Selects: USB_U_SERIAL [=n] && USB_F_OBEX [=n]

```

22.4.2.3.7 USB device printer

```

Symbol: USB_CONFIGFS_F_PRINTER [=y]
Type : boolean
Prompt: Printer function
Location:
-> Device Drivers
-> USB support (USB_SUPPORT [=y])
-> USB Gadget Support (USB_GADGET [=y])

```

```
-> USB Gadget Drivers (<choice> [=y])
-> USB functions configurable through configs (USB_CONFIGFS [=y])
Defined at drivers/usb/gadget/Kconfig:469
Depends on: <choice> && USB_CONFIGFS [=y]
Selects: USB_F_PRINTER [=y]
```

22.4.2.3.8 USB device uac1.0

```
Symbol: USB_CONFIGFS_F_UAC1 [=y]
Type : boolean
Prompt: Audio Class 1.0
Location:
-> Device Drivers
-> USB support (USB_SUPPORT [=y])
-> USB Gadget Support (USB_GADGET [=y])
-> USB Gadget Drivers (<choice> [=y])
-> USB functions configurable through configs (USB_CONFIGFS [=y])
Defined at drivers/usb/gadget/Kconfig:402
Depends on: <choice> && USB_CONFIGFS [=y] && SND [=y]
Selects: USB_LIBCOMPOSITE [=y] && SND_PCM [=y] && USB_F_UAC1 [=y]
```

22.4.2.4 USB Legacy Kernel Configuration

22.4.2.4.1 USB device serial

```
Symbol: USB_G_SERIAL [=y]
Type : tristate
Prompt: Serial Gadget (with CDC ACM and CDC OBEX support)
Location:
-> Device Drivers
-> USB support (USB_SUPPORT [=y])
-> USB Gadget Support (USB_GADGET [=y])
-> USB Gadget Drivers (<choice> [=y])
Defined at drivers/usb/gadget/legacy/Kconfig:260
Depends on: <choice> && TTY [=y]
Selects: USB_U_SERIAL [=y] && USB_F_ACM [=y] && USB_F_SERIAL [=y] && USB_F_OBEX [=y] &&
USB_LIBCOMPOSITE [=y]
```

22.4.2.4.2 USB device uvc

```

Symbol: USB_G_WEBCAM [=y]
Type : tristate
Prompt: USB Webcam Gadget
Location:
  -> Device Drivers
    -> USB support (USB_SUPPORT [=y])
      -> USB Gadget Support (USB_GADGET [=y])
        -> USB Gadget Drivers (<choice> [=y])
Defined at drivers/usb/gadget/legacy/Kconfig:471
Depends on: <choice> && VIDEO_DEV [=y]
Selects: USB_LIBCOMPOSITE [=y] && VIDEOBUF2_VMALLOC [=y] && USB_F_UVC [=y] && USB_DWC2_HIGHWIDTH_FIFO [=y]

```

22.4.2.5 USB gadget descriptor configuration

hid、serial composite devices are not supported, and the configuration needs to be modified manually.

The configuration path of usb composite and related function descriptors is as follows:

```

buildroot/package/ingenic/system_config/usb-device
├── S90usb
├── usb
│   ├── adb
│   ├── hid
│   ├── mass_storage
│   ├── printer
│   ├── rndis
│   ├── serial
│   ├── uac1
│   ├── udc_daemon
│   └── uvc

```

22.4.2.6 USB Power saving mode

Power saving mode refers to:

The USB controller is used as the otg mode. When the PC is inserted, the VBUS pin level changes from low to high. The vbus-dete-gpio connected to the VBUS pin triggers the interrupt signal, turns on the USB controller and USB PHY clock, and initializes the USB device to communicate with the PC. On the contrary, when the PC is unplugged, the VBUS level changes from high to low, and the USB controller and USB PHY clock are turned off. When a USB peripheral is inserted through the OTG cable, the ID pin is pulled to GND, the id-dete-gpio connected to the ID pin triggers the interrupt signal, turns on the USB controller and USB PHY clock, and initializes the USB host to establish a connection with the peripheral. On the contrary, when the OTG is pulled out, the ID pin level changes

from low to high, turns off the USB controller and USB PHY clock, and stops the external power supply to achieve the power saving function.

22.4.2.6.1 Kernel configuration

halley6_v10 4.4.94 default don't configuration uses power-saving mode.

Compilation options need to be configured when using power-saving mode: USB_DWC2_DETECT_CONNECT, Configuration description is as follows:

```
Symbol: USB_DWC2_DETECT_CONNECT [=y]
Type : boolean
Prompt: These is detect function.
Location:
  -> Device Drivers
    -> USB support (USB_SUPPORT [=y])
      -> DesignWare USB2 DRD Core Support (USB_DWC2 [=y])
        -> DWC2 DETECT_MODE <choice> [=y]
Defined at drivers/usb/dwc2/Kconfig:71
Depends on: <choice>
Selects: USB_DWC2_EXT_VBUS_DETECT [=y]
```

You need to unconfigure compilation options when not using power-saving mode: USB_DWC2_FORCE_CONNECT, Configuration description is as follows:

```
Symbol: USB_DWC2_FORCE_CONNECT [=y]
Type : boolean
Prompt: These is no detect function.
Location:
  -> Device Drivers
    -> USB support (USB_SUPPORT [=y])
      -> DesignWare USB2 DRD Core Support (USB_DWC2 [=y])
        -> DWC2 DETECT_MODE <choice> [=y]
Defined at drivers/usb/dwc2/Kconfig:68
Depends on: <choice>
```

22.4.2.6.2 Device Tree Configuration

In the power-saving mode, you need to configure id-dete-gpio and vbus-dete-gpio in the board level device tree at the same time. But id-dete-gpio and vbus-dete-gpio not introduced from halley6_v10 board, If you need to use the power saving function, you need to export id-dete-gpio and vbus-dete-gpio, and Configure pin attributes in the dts device tree, Refer to the configuration method halley5_v30.dts.

22.4.2.6.3 Kernel version difference

Kernel-4.4.94 supports the use of USB power saving mode.
Kernel-5.10 does not support USB power saving mode temporarily.

22.5 Application Instructions

22.5.1 USB host

22.5.1.1 USB host Mass Storage

1. When the USB disk is inserted, the print information pops up

```
[ 46.070034] usb 1-1: new high-speed USB device number 2 using dwc2
[ 46.296250] usb 1-1: New USB device found, idVendor=0951, idProduct=1666
[ 46.303202] usb 1-1: New USB device strings: Mfr=1, Product=2, SerialNumber=3
[ 46.310592] usb 1-1: Product: DataTraveler 3.0
[ 46.315185] usb 1-1: Manufacturer: Kingston
[ 46.319508] usb 1-1: SerialNumber: 60A44C413C4EB211A98A00A0
[ 46.325821] usb-storage 1-1:1.0: USB Mass Storage device detected
[ 46.342363] scsi host0: usb-storage 1-1:1.0
[ 47.726378] scsi 0:0:0:0: Direct-Access Kingston DataTraveler 3.0 PMAP PQ: 0 ANSI: 6
[ 47.736215] sd 0:0:0:0: [sda] 30277632 512-byte logical blocks: (15.5 GB/14.4 GiB)
[ 47.752538] sd 0:0:0:0: [sda] Write Protect is off
[ 47.759990] sd 0:0:0:0: [sda] No Caching mode page found
[ 47.765648] sd 0:0:0:0: [sda] Assuming drive cache: write through
[ 47.780215] sda: sda1
[ 47.789136] sd 0:0:0:0: [sda] Attached SCSI removable disk
```

2. Mount the USB disk to the file system

```
# mount -t vfat /dev/sda1 mnt/
```

3. Enter /mnt for data interaction.

22.5.1.2 USB host camera

1. The recognition is successful after inserting the USB camera device

```
[ 56.560037] usb 1-1: new high-speed USB device number 2 using dwc2
[ 56.910300] usb 1-1: New USB device found, idVendor=058f, idProduct=5608
[ 56.917231] usb 1-1: New USB device strings: Mfr=3, Product=1, SerialNumber=0
[ 56.924635] usb 1-1: Product: USB 2.0 Camera
[ 56.929114] usb 1-1: Manufacturer: Alcor Micro, Corp.
[ 56.939481] uvcvideo: Found UVC 1.00 device USB 2.0 Camera (058f:5608)
[ 56.949251] input: USB 2.0 Camera as /devices/platform/ahb2/13500000.otg/usb1/1-1/1-1:1.0/input/input0
```

2. Photographic test

```
usage: uvcview [-d <device>] [-c <count>]
--help -H print this message
--print_formats -F print video device info
--device -d , video device, default is /dev/video0
--width -w grab width
--height -h grab height
--count -c set the count to grab
--rate -r frame sample rate(fps)
--yuv -y use yuyv input format
--timeout -t select timeout
--match -m do picture match test
--perf -p do performance test
```

Note: */dev/vidio5 is a standard video node generated by USB camera*

```
# ./grab -w 640 -h 480 -d /dev/video5 -y -c 3
```

3. Generate Image

```
p-0.jpg p-1.jpg p-2.jpg
```

22.5.1.3 USB host hid mouse

1. Insert mouse device to controller

```
[ 117.100038] usb 1-1: new low-speed USB device number 2 using dwc2
[ 117.313977] usb 1-1: New USB device found, idVendor=046d, idProduct=c077
[ 117.320929] usb 1-1: New USB device strings: Mfr=1, Product=2, SerialNumber=0
[ 117.328305] usb 1-1: Product: USB Optical Mouse
[ 117.333098] usb 1-1: Manufacturer: Logitech
[ 117.338568] input: Logitech USB Optical Mouse as /devices/platform/ahb2/13500000.otg/usb1/1-1/1-1:1.0/input/input0
```

2. Event triggered by moving the mouse after executing the program capturing the event

```
# cd /testsuite/usb_test/usb_host/getevent_test/
# ./getevent_test 0
```

```
/dev/input/mouse0
evdev version: 0.0.0
name:
features: relative reserved unknown unknown unknown unknown unknown unknown
/dev/input/mouse0: open, fd = 3
```

```
Sun Mar 1 16:38:18 2020.000000 type 0x0011; code 0x000e; value 0x00000000; Led
[ 125.626285] random: nonblocking pool is initialized
Wed Dec 25 06:26:16 2019.000000 type 0x0011; code 0x000e; value 0x00000000; Led
Thu Dec 26 00:59:52 2019.000000 type 0x0011; code 0x000e; value 0x00000000; Led
Mon Dec 23 18:27:20 2019.000000 type 0x0011; code 0x000e; value 0x00000000; Led
Thu Dec 26 00:55:36 2019.000000 type 0x0011; code 0x000e; value 0x00000000; Led
Wed Dec 25 06:47:36 2019.000000 type 0x0011; code 0x000e; value 0x00000000; Led
Tue Dec 24 12:31:04 2019.000000 type 0x0011; code 0x000e; value 0x00000000; Led
Wed Dec 25 06:47:36 2019.000000 type 0x0011; code 0x000e; value 0x00000000; Led
Mon Dec 23 18:10:16 2019.000000 type 0x0011; code 0x000e; value 0x00000000; Led
....
....
....
```

22.5.1.4 USB host rndis

1. Insert USB network card device to controller

```
[ 68.720047] usb 1-1: new high-speed USB device number 2 using dwc2
[ 68.935798] usb 1-1: New USB device found, idVendor=0b95, idProduct=1790
[ 68.942770] usb 1-1: New USB device strings: Mfr=1, Product=2, SerialNumber=3
[ 68.950422] usb 1-1: Product: AX88179
[ 68.954211] usb 1-1: Manufacturer: ASIX Elec. Corp.
[ 68.959249] usb 1-1: SerialNumber: 0000000000002E
[ 69.294505] ax88179_178a 1-1:1.0 eth1: register 'ax88179_178a' at usb-13500000.otg-1, ASIX AX88179
USB 3.0 Gigabit Ethernet, 00:0e:c6:fa:54:fa
```

2. View USB network card information on the development board

```
#ifconfig -a
....
....
eth1      Link encap:Ethernet HWaddr 00:0E:C6:FA:54:FA
          BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
....
....
```

3. Configure USB network card ip at the development board

```
#ifconfig eth1 192.168.4.100
```

4. Test the network card on the development board

```
#ping 192.168.4.100
```

```
PING 192.168.4.100 (192.168.4.100): 56 data bytes
64 bytes from 192.168.4.100: seq=0 ttl=64 time=0.370 ms
64 bytes from 192.168.4.100: seq=1 ttl=64 time=0.148 ms
64 bytes from 192.168.4.100: seq=2 ttl=64 time=0.154 ms
64 bytes from 192.168.4.100: seq=3 ttl=64 time=0.150 ms
....
....
```

22.5.2 USB device

22.5.2.1 USB device mass storage

1. Make drive letter

```
# dd if=/dev/zero of=fat32.img bs=1k count=2048
# mkfs.vfat fat32.img
```

2. Add drive letter

```
# echo fat32.img > /sys/kernel/config/usb_gadget/demo/functions/mass_storage.0/lun.0/file
```

3. After completion, the drive letter will pop up on the PC and support hot plug

22.5.2.2 USB device adb

1. Start adb function directly without any configuration after startup
2. The adb does not support hot plugging. After plugging, the device will be cleared, and it needs to be rebound when it is plugged in again

```
# echo 13500000.otg > /sys/kernel/config/usb_gadget/demo/UDC
```

22.5.2.3 USB device hid

1. PC terminal identification device

```
$ dmesg
```

```
[2180880.531376] usb 2-1.5: new high-speed USB device number 115 using ehci-pci
[2180880.628285] usb 2-1.5: New USB device found, idVendor=18d1, idProduct=d002
[2180880.628292] usb 2-1.5: New USB device strings: Mfr=1, Product=2, SerialNumber=3
[2180880.628304] usb 2-1.5: Product: composite-demo
[2180880.628307] usb 2-1.5: Manufacturer: ingenic
[2180880.628309] usb 2-1.5: SerialNumber: 0123456789ABCDEF
[2180880.635882] input: ingenic composite-demo as /devices/pci0000:00/0000:00:1d.0/usb2/2-1/2-1.5/2-1.5:1.0/0003:18D1:D002.006C/input/input126
[2180880.691853] hid-generic 0003:18D1:D002.006C: input,hidraw3: USB HID v1.01 Keyboard [ingenic
```

```
composite-demo] on usb-0000:00:1d.0-1.5/input0
```

2. Development board execution test program

```
# cd /testsuite/usb_test/usb_gadget/hid_gadget_test
```

Analog USB keyboard

```
# ./hid_gadget_test /dev/hidg0 keyboard
```

Analog USB mouse

```
# ./hid_gadget_test /dev/hidg1 mouse
```

22.5.2.4 USB device uvc

1. PC terminal identification device

```
$ dmesg
```

```
[2187560.274157] usb 2-1.5: new high-speed USB device number 19 using ehci-pci
[2187560.371029] usb 2-1.5: New USB device found, idVendor=18d1, idProduct=d002
[2187560.371033] usb 2-1.5: New USB device strings: Mfr=1, Product=2, SerialNumber=3
[2187560.371035] usb 2-1.5: Product: composite-demo
[2187560.371037] usb 2-1.5: Manufacturer: ingenic
[2187560.371039] usb 2-1.5: SerialNumber: 0123456789ABCDEF
[2187560.394857] uvcvideo: Found UVC 1.00 device composite-demo (18d1:d002)
[2187560.403005] input: composite-demo as /devices/pci0000:00/0000:00:1d.0/usb2/2-1/2-1.5/2-1.5:1.0/input/input141
```

2. Development board execution test program

webcam_gadget usage method:

```
Usage: webcam_gadget [options]
Available options are
-b          Use bulk mode
-d          Do not use any real V4L2 capture device
-h          Print this help screen and exit
-i          images dir for [uvc-WxH.jpg uvc-WxH.yuv]
-m          Streaming mult for ISOC (b/w 0 and 2)
-n          Number of Video buffers (b/w 2 and 32)
-o <IO method> Select UVC IO method:
              0 = MMIO
              1 = USER_PTR
-s <speed>   Select USB bus speed (b/w 0 and 2)
              0 = Full Speed (FS)
              1 = High Speed (HS)
              2 = Super Speed (SS)
-t          Streaming burst (b/w 0 and 15)
-u device    UVC Video Output device
```

```
-v device      V4L2 Video Capture device
-e device      HELIX Video Capture device
```

Camera dynamic test:

format:

```
# ./webcam_gadget -u /dev/video<uvc video node #> -v /dev/video<camera video node #> -e /dev/video<helix video node #>
```

example:

```
# ./webcam_gadget -u /dev/video7 -v /dev/video4 -e /dev/video0
```

Static picture test:

format:

Note: It is necessary to put the test picture in the path of the -i command in advance. Naming rules: uvc-(weight)x(height).yuv and uvc-(weight)x(height).jpg

example:

```
# ./webcam_gadget -u /dev/video<uvc video node #> -v /dev/video<camera video node #> -e /dev/video<helix video node #> -d -i /mnt
```

pc test tool:

```
linux os:
    Video tools: xawtv
    Video tools: cheese webcam booth

windows os:
    Video tools: amcap

Mobile terminal:
    APP: usb camera
```

Note: Please use the legacy configuration for Huawei mobile phones.

22.5.2.5 USB device remote NDIS

1. PC terminal identification device

```
$ dmesg
```

Device information:

```
[2188325.480982] usb 2-1.5: new high-speed USB device number 21 using ehci-pci
[2188325.577912] usb 2-1.5: New USB device found, idVendor=18d1, idProduct=d002
[2188325.577919] usb 2-1.5: New USB device strings: Mfr=1, Product=2, SerialNumber=3
[2188325.577930] usb 2-1.5: Product: composite-demo
[2188325.577932] usb 2-1.5: Manufacturer: ingenic
[2188325.577934] usb 2-1.5: SerialNumber: 0123456789ABCDEF
[2188325.586180] rndis_host 2-1.5:1.0 usb0: register 'rndis_host' at usb-0000:00:1d.0-1.5, RNDIS device, 52:ea:19:19:5f:69
[2188325.650205] rndis_host 2-1.5:1.0 enp0s29u1u5: renamed from usb0
[2188325.670351] IPv6: ADDRCONF(NETDEV_UP): enp0s29u1u5: link is not ready
```

PC identify the USB network card device:

```
$ ifconfig -a
```

```
enp0s29ulu5 Link encap:Ethernet HWaddr 52:ea:19:19:5f:69
    UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
    RX packets:0 errors:0 dropped:0 overruns:0 frame:0
    TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
    collisions:0 txqueuelen:1000
    RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)
```

2. PC terminal configure the USB network card device IP:

```
$ sudo ifconfig enp0s29ulu5 192.168.4.250 up
```

```
enp0s29ulu5 Link encap:Ethernet HWaddr 52:ea:19:19:5f:69
    inet addr:192.168.4.250 Bcast:192.168.4.255 Mask:255.255.255.0
    inet6 addr: fe80::50ea:19ff:fe19:5f69/64 Scope:Link
    UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
    RX packets:0 errors:0 dropped:0 overruns:0 frame:0
    TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
    collisions:0 txqueuelen:1000
    RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)
```

3. PC terminal test network card:

```
$ ping 192.168.4.250
```

```
PING 192.168.4.250 (192.168.4.250) 56(84) bytes of data.
64 bytes from 192.168.4.250: icmp_seq=1 ttl=64 time=0.034 ms
64 bytes from 192.168.4.250: icmp_seq=2 ttl=64 time=0.034 ms
64 bytes from 192.168.4.250: icmp_seq=3 ttl=64 time=0.031 ms
```

22.5.2.6 USB device serial

4. PC terminal identification device

```
$ dmesg
```

Device information:

```
[2189021.818872] usb 2-1.5: new high-speed USB device number 25 using ehci-pci
[2189021.915645] usb 2-1.5: New USB device found, idVendor=0525, idProduct=a4a7
[2189021.915651] usb 2-1.5: New USB device strings: Mfr=1, Product=2, SerialNumber=0
[2189021.915655] usb 2-1.5: Product: Gadget Serial v2.4
[2189021.915659] usb 2-1.5: Manufacturer: Linux 4.4.94+ with 13500000.otg
[2189021.922892] cdc_acm 2-1.5:2.0: ttyACM0: USB ACM device
```


1. PC configure the USB serial's VID and PID

```
$ echo 0x18d1 0xd002 > /sys/bus/usb-serial/module_drivers/drivers/generic/new_id
```

2. Interaction test between development board and PC terminal

PC terminal test:

Note: ttyUSB1 corresponds to USB serial device

```
$ echo 111111111 > /dev/ttyUSB1
```

```
$ cat /dev/ttyUSB1
```

Development board test:

```
# cat /dev/ttyGS0
```

```
# echo 222 > /dev/ttyGS0
```

22.5.2.7 USB device printer

1. PC terminal identification device

```
$ dmesg
```

Device information:

```
usb1p 2-1.7:1.4: usb1p0: USB Bidirectional printer dev 38 if 4 alt 0 proto 2 vid 0x18D1 pid 0xD002
```

2. Interaction test between development board and PC terminal

Development board test:

```
# cd /testsuite/usb_test/usb_gadget/prn_example/
```

```
# ./prn_example -read_data
```

```
# cat data_file | ./prn_example -write_data
```

PC terminal test:

```
$ echo 111 > /dev/usb/lp0
```

```
$ cat /dev/usb/lp0
```

22.5.2.8 USB device uac1.0

1. PC terminal identification device

```
$ aplay -l
```

Sound card information:

```
card 1: compositdemo [composite-demo], device 0: USB Audio [USB Audio]
```

```
Subdevices: 1/1
```

```
Subdevice #0: subdevice #0
```

2. Select the USB sound card device on the PC to play the sound.

22.5.3 USB legacy

22.5.3.1 USB device serial

1. PC terminal identification device

```
$ dmesg
```

Device information:

```
[2189021.818872] usb 2-1.5: new high-speed USB device number 25 using ehci-pci
```



```
[2189021.915645] usb 2-1.5: New USB device found, idVendor=0525, idProduct=a4a7
[2189021.915651] usb 2-1.5: New USB device strings: Mfr=1, Product=2, SerialNumber=0
[2189021.915655] usb 2-1.5: Product: Gadget Serial v2.4
[2189021.915659] usb 2-1.5: Manufacturer: Linux 4.4.94+ with 13500000.otg
[2189021.922892] cdc_acm 2-1.5:2.0: ttyACM0: USB ACM device
```

2. Exchange information

PC terminal test:

```
$echo 111111111 > /dev/ttyACM0
$cat /dev/ttyACM0
```

Development board test:

```
#cat ttyGS0
#echo 2222 > ttyGS0
```

22.5.3.2 USB device uvc

1. PC terminal identification device

```
$ dmesg
```

Device information:

```
[1278801.857403] usb 2-1.7: new high-speed USB device number 25 using ehci-pci
[1278801.954312] usb 2-1.7: New USB device found, idVendor=1d6b, idProduct=0102
[1278801.954318] usb 2-1.7: New USB device strings: Mfr=1, Product=2, SerialNumber=0
[1278801.954322] usb 2-1.7: Product: Webcam gadget
[1278801.954326] usb 2-1.7: Manufacturer: Linux Foundation
[1278801.976022] uvcvideo: Found UVC 1.00 device Webcam gadget (1d6b:0102)
[1278801.983318] input: Webcam gadget as /devices/pci0000:00/0000:00:1d.0/usb2/2-1/2-1.7/2-1.7:1.0/input/input119
```

2. Mobile phone terminal test method

- a) The mobile phone is connected to the development board through the adapter cable
- b) Phone opens APP: USB camera

Note: If the mobile phone recognizes the device, please restart the development board

3. Detailed test method

the same as above: [\[device-uvc-gadget\]](#)

22.5.4 USB software switch

In addition to selecting the USB controller mode through the kernel configuration or device tree configuration, you can also actively switch between master and slave modes through the command line. When using the command line to switch, the USB controller must be configured to the otg mode (refer to the USB controller mode configuration section for the method). Examples of using the

command line are as follows:

```
# echo host > /sys/devices/platform/apb/10000000.otg_phy/sw_switch_hstg  
[ 81.955542] Force ID Level Low!  
  
# echo device > /sys/devices/platform/apb/10000000.otg_phy/sw_switch_hstg  
[ 148.459009] Force ID level High!
```

22.5.5 USB eye map adjustment

The X1600 USB IP follows the USB2.0 specification and supports multiple test modes, including Test_ J mode, Test_ K mode, Test_ SE0_ NAK mode, Test_ Packet Mode and Test_ Force_ Enable mode.

1、As a device for eye map testing, you need to use HSETT, a testing software tool on the computer side, to make it enter the test_Packet test mode.

2、As a device for eye map testing, you need to configure HPRT [16,13] bits of the register to enter test_Packet test mode.

The IP manufacturer's pre reserved table of x1600 USB Phy can be used to adjust the eye diagram to obtain the best signal quality. The USB Phy register is mapped to the bus base address 0x10078000.

Addr	Bit region	Default	Description
0x30	[2:0]	3' b001	Enables pre-emphasis under certain circumstances in order to compensate for excessive capacitive loading on D+ / D-. This tends to increase the slew rate. 3' b001: enable pre-emphasis during SOF and EOP 3' b010: enable pre-emphasis during chirp 3' b100: enable pre-emphasis in non-chirp state 3' b111: always enable pre-emphasis All other combinations are reserved
0x40	[5:3]	3' b001	Tx HS pre_emphasize strength configure, 3' b111 represents the strongest, 3' b000 the weakest
0x11C	[4:0]	5' b10101	45ohm HS ODT value tuning & FS/LS driver strength tuning 5' b11111: smallest HS ODT value & largest FS/LS driver strength and fastest FS/LS slew rate. 5' b10000: biggest HS ODT value & smallest FS/LS driver strength and slowest FS/LS slew rate.
0x124	[4:2]	3' b000	HS eye height tuning 3' b000 : 400mv(default) 3' b001 : 475mv 3' b010 : 350mv 3' b011 : 500mv 3' b100 : 412.5mv 3' b101 : 425mv 3' b110 : 437.5mv 3' b111 : 450mv
Addr	Bit region	Default	Description
0x60	[4:2]	3' b000	This is used to adjust the 45Ω HS termination by adjusting the reference voltage that tunes it. Useful to compensate for package and board level parasitic. Setting the compensation voltage lower, sets the ODT impedance lower. 3' b000: 268mV (default)

			3' b001: 262mV 3' b010: 250mV 3' b011: 237.5mV 3' b100: 275mV 3' b101: 281mV 3' b110: 293mV 3' b111: 300mV
	[7:5]	3' b001	Hs driver current compensation voltage reference. This is used to adjust the HS current by adjusting the reference voltage that tunes it. Useful to compensate for package and board level parasitic. Setting the compensation voltage lower, sets the current lower, the opposite, sets the current larger 3' b000: 200mV (default) 3' b001: 225mV 3' b010: 250mV 3' b011: 175mV 3' b100: 400mV 3' b101: 425mV 3' b110: 450mV 3' b111: 375mV

22.6 matters needing attention

22.6.1 Modify the USB gadget configuration

Modify the default supported gadget devices. The configuration file path is as follows:

```
buildroot/package/ingenic/system_config/usb-device/S90usb
```

When using a function, other functions need to be annotated with '#':

Note: hid and serial can only be used separately, and composite equipment is not supported

```

/etc/init.d/usb/uvic $1
/etc/init.d/usb/adb $1
/etc/init.d/usb/mass_storage $1
#/etc/init.d/usb/hid $1
/etc/init.d/usb/printer $1
/etc/init.d/usb/rndis $1
/etc/init.d/usb/uac1 $1
#/etc/init.d/usb/serial $1

```

22.6.2 When Huawei mobile phones use UVC, the OTG cannot identify the problem Legacy configuration can solve the problem.

23GMAC GIGABIT ETHERNET CONTROLLER INTERFACE

23.1 Module function introductions

RMII Reduced Media Independent Interface is another implementation in the IEEE 802.3u standard besides MII interface. RMII supports bus interface speeds of 10M and 100M.

23.2 Driver source code location

Kernel driver code path:

```
module_drivers/drivers/net/ethernet/ingenic
```

```
|— ethtool.c
|— ingenic_mac.c
|— ingenic_mac.h
|— Kconfig
|— Makefile
|— readme
|— synopGMAC_Dev.c
|— synopGMAC_Dev.h
|— synopGMAC_plat.c
|— synopGMAC_plat.h
```

23.3 Device Tree Configurations

The path of the device tree:

```
module_dirvers/dts/x1600.dtsi
```

23.3.1 Device tree default configurations

RMII configuration reference is as follows:

```
&mac0 {
    pinctrl-names = "default", "reset";
    status = "okay";
    pinctrl-0 = <&mac0_rmii_p0_normal>, <&mac0_rmii_p1_normal>, <&mac0_phy_clk>;
    pinctrl-1 = <&mac0_rmii_p0_rst>, <&mac0_rmii_p1_normal>, <&mac0_phy_clk>;
    ingenic,rst-gpio = <&gpb 31 GPIO_ACTIVE_LOW INGENIC_GPIO_NOBIAS>;
    ingenic,rst-ms = <10>;
    ingenic,mac-mode = <RMII>;
    ingenic,mode-reg = <0xb0000054>;
    ingenic,phy-clk-freq = <50000000>;
}
```

Halley6_V1.x series development boards support single network card MAC0.

The device tree attributes are described as follows:

Attribute names	Description
● ingenic,rst-gpio	Define reset GPIO
● ingenic,rst-ms	Define reset time
● ingenic,rst-delay-ms	Delay time after reset, no operation on phy during the delay period
● ingenic,mac-mode	RMII
● ingenic,mode-reg	
● ingenic,phy-clk-freq	If you use the chip to supply the working clock of the phy, you need to configure the working clock frequency of the phy chip

note:

1. Change the reset gpio of the mac device corresponding to the phy according to the design of the development board, reset the effective level, and the hold time required for the reset.

23.4 Kernel compilation configurations

The kernel configuration `INGENIC_MAC`, the configuration description is as follows:

```
Symbol: INGENIC_MAC [=y]
Type : tristate
Defined at module_drivers/drivers/net/ethernet/ingenic/Kconfig
```

23.4.1 Kernel default compilation configurations

The kernel is configured with GMAC driver by default, and the configuration interface is as follows:

```
— Network device support
[*] Network core driver support
[*] Ethernet driver support —>
— Ethernet driver support
<*> ingenic on-chip MAC support
[*] Ingenic mac dma interfaces
    Ingenic mac dma bus interfaces (MAC_AHB_BUS) —>
<*> PHY Device support and infrastructure —>
```

23.4.2 Kernel custom compilation configurations

Configurable options	Configuration instructions
Ingenic gmac receive descriptor number[80..10240]	The number of buffers for the gmac controller to receive data packets. The larger the buffer, the packet loss rate at full load can be reduced. After testing, setting 8192 can avoid packet loss at the controller layer. In this case, the dynamic memory of a network card will use 16M-32M during operation.

23.5 Module kernel differences

no difference

23.6 Device node generation

After the driver is loaded successfully, eth0 or eth1 will appear after executing ifconfig -a.

23.7 Application Instructions

Execute the ifconfig -a command to view the supported network devices. If eth0 or eth1 appears, the network card is loaded successfully.

```
# ifconfig -a

eth0   Link encap:Ethernet HWaddr 1E:01:7F:E6:D3:26
       BROADCAST MULTICAST MTU:1500 Metric:1
       RX packets:0 errors:0 dropped:0 overruns:0 frame:0
       TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
       collisions:0 txqueuelen:1000
       RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)

eth1   Link encap:Ethernet HWaddr 3E:4C:9D:F4:74:4B
       BROADCAST MULTICAST MTU:1500 Metric:1
       RX packets:0 errors:0 dropped:0 overruns:0 frame:0
       TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
       collisions:0 txqueuelen:1000
       RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)

lo     Link encap:Local Loopback
       LOOPBACK MTU:65536 Metric:1
       RX packets:0 errors:0 dropped:0 overruns:0 frame:0
       TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
       collisions:0 txqueuelen:1
       RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)
```

23.7.1 Network performance test

Configure network IP

```
ifconfig eth0 IP /*Configure eth0 or eth1 according to the actual selected network port*/
```

Use the ping command to view the network connection

```
ping IP -I eth0 /*The -I option can specify the network port used by the ping command*/
```

Find a computer with Gigabit Ethernet function, use the network cable to connect directly, use the iperf3 command to test, the computer as the server, the development board as the client.

- Execute commands on the computer

```
iperf3 -s
```

- Test one-way sending performance

```
iperf3 -c 192.168.4.105 -u -b 100M -l 65507 -t 10
```

- Test one-way reception performance

```
iperf3 -c 192.168.4.105 -u -b 100M -l 65507 -t 10 -R
```

- Application layer methods to improve network performance

The following methods are used according to specific application scenarios. Unreasonable settings will affect the test results

```
sysctl -w net.core.rmem_default=10485760
```

Increase the buffer received by the network core layer, this setting will use 10M memory, dual network port bridge does not use

```
sysctl -w net.core.rmem_max=10485760
```

Increase the buffer received by the network core layer

Specify the CPU core used by the kernel to process the receiving network protocol stack. After specifying it, you can avoid the unstable performance caused by the imbalance of dual-core usage caused by the dynamic switching process of the kernel:

```
echo 0 > /proc/irq/61/smp_affinity_list
```

Specify cpu0 to process the data received by gmac1.

```
echo 1 > /proc/irq/63/smp_affinity_list
```

Specify cpu1 to process the data received by gmac0, gmac0 interrupt number 63 gmac1 interrupt number 61.

```
sysctl -w net.core.netdev_max_backlog=4000
```

Increasing the data packet buffer length during the transfer process can reduce data jitter but consume more memory.

```
echo 2 > /sys/class/net/eth0/queues/rx-0/rps_cpus
```

Specify cpu1 to process eth0 received data packets, enter 1 to specify cpu0, 2 to specify cpu1, 3 to specify cpu0-1.

24 AES Encryption and decryption driver interface

24.1 Module Function Introduction

- Advanced Encryption Standard
- Fully referring to Rijdael_fips_197 standard
- AES Engine: AES-128/192/256
- 128/192/256 bits key for AES Encryption and Decryption supported
- ECB and CBC mode supported

24.2 Driver source code location

Path of the kernel driver :

```
module_drivers/drivers/crypto/ingenic-aes.c
```

24.3 Device Tree Configuration

dts file path :

```
module_driver/dts/x1600.dtsi
```

AES Controller definition :

```
aes: aes@0x13430000 {  
    compatible = "ingenic,aes";  
    reg = <0x13430000 0x1000>;  
    interrupt-parent = <&core_intc>;  
    interrupts = <IRQ_AES>;  
    status = "ok";  
};
```

24.3.1 Device Tree Default Configuration

The default compilation of the device tree will generate AES devices.

24.3.2 Device Tree Custom Configuration

The user can shut down the AES device according to the actual needs, and configure the node as disabled in the board.dts.

```
&aes {  
    status = "disabled";  
};
```

24.4 Kernel compile configuration

Compilation options : CRYPTO_DEV_INGENIC_AES, Configuration description is as follows:

```
Symbol: CRYPTO_USER_API_SKCIPHER [=n]
Type : tristate
Prompt: User-space interface for symmetric key cipher algorithms
Location:
(1) -> Cryptographic API (CRYPTO [=y])
Defined at crypto/Kconfig:1616
Depends on: CRYPTO [=y] && NET [=y]
Selects: CRYPTO_BLKIPHER [=y] && CRYPTO_USER_API [=y]
```

```
Symbol: CRYPTO_DEV_INGENIC_AES [=n]
Type : tristate
Prompt: Support for INGENIC AES hw engine
Location:
-> Cryptographic API (CRYPTO [=y])
(1) -> Hardware crypto devices (CRYPTO_HW [=n])
Prompt: [AES] Support for INGENIC AES hw engine
Location:
(2) -> Ingenic device-drivers Configurations
Defined at drivers/crypto/Kconfig:311
Depends on: CRYPTO [=y] && CRYPTO_HW [=n] && (MACH_XBURST [=y] || MACH_XBURST2 [=n])
Selects: CRYPTO_AES [=y] && CRYPTO_BLKIPHER2 [=y] && CRYPTO_AES [=y] && CRYPTO_BLKIPHER2 [=y]
```

24.4.1 Kernel default compile configuration

The kernel is not configured with AES driver by default.

24.4.2 Kernel custom compile configuration

The user can configure the AES drive according to the actual needs, and the configuration interface is as follows:

```
<*> [SHA] Support for Ingenic SHA hw accelerator
<*> [AES] Support for INGENIC AES hw engine
[WDT] watchdog Drivers --->
[PWM] drivers --->
<M> [AUDIO] ASoC support for Ingenic --->
```

24.5 Device node generation

The following nodes are generated after successful drive loading

```
/sys/bus/platform/drivers/aes
```

24.6 Application Instructions

Test application path

```
packages/example/Sample/security_utils/aes/
```

```
├── aes.c
├── CMakeLists.txt
├── gen_key_base.c
├── include
│   ├── aes.h
│   ├── bignum.h
│   ├── gen_key_base.h
│   └── keys.h
└── main.c
```

test method :

```
aes_test in_file out_file key_file op(en:1 de:0)
```

- *in_file* : Encrypt/decrypt source data
- *out_file* : Result data
- *key_file* : secret key
- *op* : 1 encryption、0 decrypt

Relevant test data:

```
packages/example/Sample/security_utils/test_data/aes_test/
├── aes_test_data.txt
├── aes_test_en.txt
└── key.txt
```

- *key.txt* : secret key
- *aes_en.txt* : result data
- *aes_test_data.txt* : Data to be tested

Example of use :

```
# aes_test aes_test_data.txt encrypt.txt key.txt 1
# aes_test encrypt.txt decrypt.txt key.txt 0
# cat aes_test_data.txt
123456789011111
# cat decrypt.txt
123456789011111
```

25 Hash Module drive interface

25.1 Module Function Introduction

HASH engine: SHA and MD5

SHA1 fully referring to RFC-3174 standard

MD5 fully referring to RFC-1321 standard

SHA core: support SHA 160/224/256/384/512bit digest

25.2 Driver source code location

Location of driver source code :

```
module_drivers/drivers/crypto/ingenic-hash.c
```

25.3 Device Tree Configuration

dts file path :

```
module_driver/dts/x1600.dtsi
```

Hash Controller definition

```
hash: hash@0x13470000 {  
    compatible = "ingenic,hash";  
    reg = <0x13470000 0x10000>;  
    interrupt-parent = <&core_intc>;  
    interrupts = <IRQ_HASH>;  
    status = "ok";  
};
```

25.3.1 Device Tree Default Configuration

The default compilation of the device tree will generate hash devices.

25.3.2 Device Tree Custom Configuration

The user can turn off the hash device according to actual needs and configure the node as disabled.

25.4 Kernel compile configuration

Kernel driver compilation option : CRYPTO_DEV_INGENIC_SHA

```
Symbol: CRYPTO_USER_API_HASH [=y]  
Type : tristate
```

```
Prompt: User-space interface for hash algorithms
Location:
(1) -> Cryptographic API (CRYPTO [=y])
Defined at crypto/Kconfig:1607
Depends on: CRYPTO [=y] && NET [=y]
Selects: CRYPTO_HASH [=y] && CRYPTO_USER_API [=n]

Symbol: CRYPTO_DEV_INGENIC_SHA [=y]
Type : tristate
Prompt: Support for Ingenic SHA hw accelerator
Location:
-> Cryptographic API (CRYPTO [=y])
-> Hardware crypto devices (CRYPTO_HW [=n])
Prompt: [SHA] Support for Ingenic SHA hw accelerator
Location:
-> Ingenic device-drivers Configurations
Defined at drivers/crypto/Kconfig:493
Depends on: CRYPTO [=y] && CRYPTO_HW [=n] && (MACH_XBURST [=y] || MACH_XBURST2 [=n])
Selects: CRYPTO_ALGAPI [=y] && CRYPTO_ALGAPI [=y]
```

25.4.1 Kernel default compile configuration

The default compilation of the kernel will generate a Hash controller device. The configuration interface is as follows:

```
< > Pseudo Random Number Generation for Cryptographic modules
-* NIST SP800-90A DRBG --->
-* Jitterentropy Non-Deterministic Random Number Generator
<*> User-space interface for hash algorithms
<*> User-space interface for symmetric key cipher algorithms
< > User-space interface for random number generator algorithms
< > User-space interface for AEAD cipher algorithms

--- Hardware crypto devices
<*> Support for INGENIC AES hw engine
< > Imagination Technologies hardware hash accelerator
<*> Support for Ingenic SHA hw accelerator
```

25.4.2 Kernel custom compile configuration

The user can shut down the equipment according to actual needs.

25.5 Device node generation

```
/sys/bus/platform/drivers/hash
```

25.6 Application Instructions

application test program path :

```
packages/example/Sample/security_utils/hash/  
├── CMakeLists.txt  
├── hash.c  
├── hash_openssl.c  
├── include  
│   ├── bignum.h  
│   ├── hash.h  
│   └── hash_openssl.h  
└── main.c
```

test method :

```
hash_test in_file out_file
```

in_file is the name of the data file to be encrypted, The data of successful encryption will be saved in out_ File file.

Relevant test data :

```
packages/example/Sample/security_utils/test_data/hash_test/  
├── hash_result.txt  
└── hash_test_data.txt
```

- hash_result.txt : Result data
- hash_test_data.txt : Data to be tested

```
# hash_test hash_test_data.txt hash_result.txt
```

Print after encryption is successful :

```
hash crypt success!!
```

26 PWM Module driver interface

26.1 Module function introduction

The Pulse width modulation (pwm) module is designed to be divided into 8 independent channels, which can work independently without affecting each other. At the same time, it is divided into two working modes, cpu mode and DMA mode. The cpu mode generates a square wave by using the input clock source, and the square wave generation in the DMA mode determines the period and duty cycle by the data in the fifo. In addition, the DMA mode must be aligned according to 4word.

26.2 Driver source code location

The location of the driver source code:

```
module_drivers/drivers/pwm/pwm-ingenic-v3.c
```

26.3 Device Tree Configuration

The location of the device tree:

```
module_drivers/dts/x1600-pinctrl.dtsi
```

1. Define the relevant PWM IO Function

```
pwm_pin: pwm-pin{
    pwm0_pc: pwm0_pc {
        ingenic,pinmux = <&gpc 0 0>;
        ingenic,pinmux-funcsel = <PINCTL_FUNCTION0>;
    };
    ....
    ....
    ....

    pwm7_pb_21: pwm7_pb_21 {
        ingenic,pinmux = <&gpc 21 21>;
        ingenic,pinmux-funcsel = <PINCTL_FUNCTION2>;
    };
};
```

2. Defining the PWM Controller Function

```
pwm: pwm@0x134c0000 {
    compatible = "ingenic,x1600-pwm";
    #pwm-cells = <2>;
    reg = <0x134c0000 0x10000>;
    interrupt-parent = <&core_intc>;
    interrupts = <IRQ_PWM>;
    dmas = <&pdma INGENIC_DMA_TYPE(INGENIC_DMA_REQ_PWM0_TX)>;
};
```

```
<&pdma INGENIC_DMA_TYPE(INGENIC_DMA_REQ_PWM1_TX)>,
<&pdma INGENIC_DMA_TYPE(INGENIC_DMA_REQ_PWM2_TX)>,
<&pdma INGENIC_DMA_TYPE(INGENIC_DMA_REQ_PWM3_TX)>,
<&pdma INGENIC_DMA_TYPE(INGENIC_DMA_REQ_PWM4_TX)>,
<&pdma INGENIC_DMA_TYPE(INGENIC_DMA_REQ_PWM5_TX)>,
<&pdma INGENIC_DMA_TYPE(INGENIC_DMA_REQ_PWM6_TX)>,
<&pdma INGENIC_DMA_TYPE(INGENIC_DMA_REQ_PWM7_TX)>;

dma-names = "0", "1", "2", "3", "4", "5", "6", "7";

};
```

26.3.1 Device Tree Default Configuration

The kernel halley6_v10.dts board-level configuration file uses pwm0_pc as the PWM output function by default.

The kernel enables pwm devices by default.

```
&pwm {
    pinctrl-names = "default";
    pinctrl-0 = <&pwm0_pc>;
    status = "disable";
};
```

26.3.2 Device Tree Custom Configuration

If you need to use another channel's GPIO as PWM output, you can modify the halley6_v10.dts file, such as adding pwm2_pc

```
&pwm {
    pinctrl-names = "default";
    pinctrl-0 = <&pwm0_pc>, <&pwm2_pc>;
    status = "okay";
};
```

26.4 Kernel compilation configuration

The kernel driver compilation options are controlled by PWM_INGENIC_V3, and the configuration instructions are as follows:

```
Symbol: PWM_INGENIC_V3 [=y]
Type : boolean
Prompt: Ingenic V3 PWM support
Location:
    -> Ingenic device-drivers Configurations
(1) -> [PWM] drivers
Defined at module_drivers/drivers/pwm/Kconfig:26
```


Depends on: SOC_X1600 [=y]

Selects: PWM [=y]

26.4.1 Kernel default compilation configuration

Kernel compiles PWM drivers by default

26.4.2 Kernel Custom Compilation Configuration

If the product needs to enable the PWM function, you can select the PWM compilation option. The configuration interface is as follows:

```
[*] Ingenic V3 PWM support
[*] pwm gpio trigger support
```

26.5 Device Node Generation

You can view the specific conditions of the pwm device through the standard /sys/class/pwm, and export and use the pwm user space through the standard sysfs interface.

26.5.1 Export of the kernel pwm standard framework

When using the kernel pwm standard framework, you can operate under the path "sys/class/pwm/pwmchip0/", first you can cat the npwm file to see the number of supported pwm channels.

```
# ls
device export npwm power subsystem uevent unexport
# cat npwm
8
```

The x1600 supports 8 pwm channels.

The user can write the desired channel number to the export file, and a pwm* folder will be generated, under which the file corresponding to the pwm channel will be operated.

```
# echo 0 > export
# ls
device npwm pwm0 uevent
export power subsystem unexport
# cd pwm0/
# ls
duty_cycle enable period polarity power uevent
```

period: This file is used to set the period in ns.

duty_cycle: this file is used to set the duty cycle (positive duty cycle), the unit is ns.

polarity: this file is used to set the polarity, i.e. low level for on state, or high level for on state, usually normal, i.e. high level for on state (inversed, normal).

enable: This file is used to enable or disable the pwm output.

When setting the period and duty cycle, the period must be set before the duty cycle,

otherwise an error will be reported.

When the pwm channel is not needed, the user can import the pwm channel number to the unexport file to turn off the pwm related configuration file.

```
# ls
device npwm pwm0 uevent
export power subsystem unexport
# echo 0 > unexport
# ls
device export npwm power subsystem uevent unexport
```

26.6 pwm playback

26.6.1 Kernel compilation configuration

Using the pwm playback function requires an optional kernel.

```
Symbol: INGENIC_PWM_AUDIO [=y]
Type : boolean
Prompt: [PWM-AUDIO] Ingenic Pwm Audio Driver
Location:
    -> Ingenic device-drivers Configurations
Defined at module_drivers/drivers/char/Kconfig:8
Depends on: PWM_INGENIC_V3 [=y]
Selects: PWM [=y]
```

26.6.2 Device Tree Configuration

The device tree needs to be opened.

```
pwm_audio {
    status = "okay";
    compatible = "ingenic,x1600-pwm-audio";
    pwm-audio-mode = <2>;
    pwm-audio-channel = <0>;
};
```

Users can set it to "disable" to turn it off if they don't need it.

26.6.3 Application Compilation

The pwm playback function is written using a character device driver and the user can find the application at the following path:

```
packages/example/Sample/pwm_audio
```

When using the application for pwm playback testing, the audio file needs to be prepared

and then tested using the following method:

```
./pwm_audio_test FM876-summer.wav
```

26.7 Application Instructions

26.7.1 Test method

1. Go to the sys/devices/platform/ahb2/134c0000.pwm/ directory:

```
# ls
channels    finish_level  of_node      sg_pwm_num
driver      free          period_ns    subsystem
driver_override  init_level    power        trigger
duty_ns     modalias      pwm          uevent
enable      mode          request
```

2. By writing 0 to 7 to the request, the corresponding pwm can be exported for use. The following takes pwm0 as an example:

```
# echo 0 > request
```

Then you can see that channel 0 under channels has been applied for, and other channels are similar:

```
# cat channels
ch: 00 requested
ch: 01 unrequested
ch: 02 unrequested
ch: 03 unrequested
ch: 04 unrequested
ch: 05 unrequested
ch: 06 unrequested
ch: 07 unrequested
```

3. At present, you can set the duty cycle duty_ns, period period_ns, and mode mode. You can use the cat mode command to view the mode:

```
# cat mode
0 --> CPU mode, Continuous waveform.
1 --> DMA mode, Specified number of waveforms.
2 --> DMA mode, Continuous waveform.
current mode: 0
```

4. If you choose mode 1, you need to set the number of cycles, other modes do not need to set the number of cycles:

```
# echo 1 > mode
```

```
# echo 1000000 > sg_pwm_num
```

5. Set the period to 1000000000 in nanoseconds:

```
# echo 100000 > period_ns
```

6. Set the duty cycle to 500000000 in nanoseconds:

```
# echo 50000 > duty_ns
```

7. Enable pwm0:

```
# echo 1 > enable
```

If the setting is correct, there will be corresponding waveform output at this time.

8. If the echo channel number is no longer used > free can turn off the waveform output:

```
# echo 0 > free
```

26.7.2 Source code location

```
packages/example/Sample/pwm/
```

The test demo integrates the above process, select channel 0 by default, 50% duty cycle, cpu mode, compile to the file system by default, burn and execute pwm_demo test channel 0 waveform output.

27 DTRNG Module drive interface

27.1 Module Function Introduction

DTRNG(Digital True Random Number Generator) is used to provide true random number encryption seed for system.

27.2 Driver source code location

Location of driver source code :

```
module_drivers/drivers/char/hw_random/ingenic-rng.c
```

27.3 Device Tree Configuration

dts file path :

```
module_driver/dts/x1600.dtsi
```

Device Tree Description :

```
dtrng: dtrng@0x10072000 {
    compatible = "ingenic, dtrng";
    reg = <0x10072000 0x100>;
    interrupt-parent = <&core_intc>;
    interrupts = <IRQ_DTRNG>;
    status = "disabled";
};
```

27.3.1 Device Tree Default Configuration

Default compilation will generate dtrng devices.

```
&dtrng {
    status = "okay";
};
```

27.3.2 Device Tree Custom Configuration

If the users need to shut down the dtrng device, the above nodes can be configured as disabled.

```
&dtrng {
    status = "disabled";
};
```

27.4 Kernel compile configuration

Kernel Configure : HW_RANDOM_INGENIC, Configuration description is as follows :

```
Symbol: HW_RANDOM_INGENIC [=y]
Type : tristate
Prompt: Ingenic HW Random Number Generator support
Location:
  -> Device Drivers
    -> Character devices
(1)    -> Hardware Random Number Generator Core support (HW_RANDOM [=y])
Prompt: [RANDOM] Ingenic HW Random Number Generator support
Location:
  -> Ingenic device-drivers Configurations
(2)    -> [RANDOM] drivers
Defined at drivers/char/hw_random/Kconfig:384
Depends on: HW_RANDOM [=y] && MIPS [=y]
```

27.4.1 Kernel default compilation configuration

Kernel default configure dtrng driver, Configuration description is as follows :

```
<+> [RANDOM] Ingenic HW Random Number Generator support
```

27.4.2 Kernel custom compile configuration

The users can remove the driver configuration according to the actual situation.

27.5 Device node generation

When the drive is loaded successfully, /dev/hwrng will generated, The user can judge whether the driver is loaded successfully according to this node.

27.6 Application Instructions

The following applications can be used to test random numbers.

```
#include<stdio.h>
#include<stdlib.h>
#include<linux/watchdog.h>
#include<sys/ioctl.h>
#include<sys/types.h>
#include<fcntl.h>
#include<unistd.h>
#include<errno.h>

int main(int argc, char *argv[])
{
    int fd ,retval;
```

```
char buf[64] = {0};
int cmd = 0;
if (!argv[1])
{
    printf("argv[1] is NULL \n");
    exit(errno);
}
fd = open(argv[1], O_RDONLY);
if(fd < 0)
{
    printf("open %s false\n", argv[1]);
    exit(errno);
}
if(read(fd, buf, 32) < 0)
    printf(" read false\n");
else
    printf("random_num register value %d\n", *(unsigned int *)buf);

close(fd);
return retval;
}
```

test result

```
# ./hwrng /dev/hwrng
random_num register value 9363246
# ./hwrng /dev/hwrng
random_num register value 1329736129
# ./hwrng /dev/hwrng
random_num register value 567871915
# ./hwrng /dev/hwrng
random_num register value 966768232
```

28 THE CAN MODULE

28.1 Module function introductions

X1600 chip integrates two groups of independent CAN controller; Each group of CAN controllers supports the following functions:

1. Support CAN 2.0A Standard 11-bit and CAN 2.0B Extended 29-bit ID
2. Arbitration Bit Rate up to 1Mbps
3. Built-in 4-byte SRAM as Receive FIFO for maximal 4 received frames
4. Listen-Only mode
5. Self-Loop Test Mode

28.2 Driver source code location

Path of driver source code:

```
module_drivers/drivers/net/can/ingenic/ingenic_can.c
```

28.3 Device Tree Configurations

Location of the device tree:

```
module_drivers/dts/x1600.dtsi
```

CAN controller description:

```
can0: can@0x13560000 {
    compatible = "ingenic,x1600-can";
    reg = <0x13560000 0x10000>;
    status = "okay";
    interrupt-parent = <&core_intc>;
    interrupts = <IRQ_CAN0>;
    dmas = <&pdma INGENIC_DMA_TYPE(INGENIC_DMA_REQ_CAN0_TX)>,
          <&pdma INGENIC_DMA_TYPE(INGENIC_DMA_REQ_CAN0_RX)>;
    dma-names = "tx", "rx";
    #address-cells = <1>;
    #size-cells = <0>;
};

can1: can@0x13570000 {
    compatible = "ingenic,x1600-can";
    reg = <0x13570000 0x10000>;
    status = "okay";
    interrupt-parent = <&core_intc>;
    interrupts = <IRQ_CAN1>;
    dmas = <&pdma INGENIC_DMA_TYPE(INGENIC_DMA_REQ_CAN1_TX)>,
          <&pdma INGENIC_DMA_TYPE(INGENIC_DMA_REQ_CAN1_RX)>;
};
```



```

        <&pdma INGENIC_DMA_TYPE(INGENIC_DMA_REQ_CAN1_RX)>;

        dma-names = "tx", "rx";
        #address-cells = <1>;
        #size-cells = <0>;

    };

```

28.3.1 Default Configurations of Device Tree

The default compilation will not generate CAN devices.

28.3.2 Device Tree Custom Configurations

Users can turn off the CAN device according to actual needs, and configure the node to disable:

```

&can0 {
    status = "disable";
    pinctrl-names = "default";
    pinctrl-0 = <&can0_pd>;
    ingenic,clk-freq = <24000000>;
};

&can1 {
    status = "disable";
    pinctrl-names = "default";
    pinctrl-0 = <&can1_pd>;
    ingenic,clk-freq = <24000000>;
};

```

28.4 Kernel compilation configurations

The kernel configuration INGENIC_CAN, the configuration description is as follows:

```

Symbol: INGENIC_CAN [=y]
Type : tristate
Prompt: ingenic on-chip CAN support
Location:
    -> Ingenic device-drivers Configurations
        -> [CAN] (ethernet) Drivers
            -> INGENIC CAN support
Defined at module_drivers/drivers/net/can/ingenic/Kconfig

```

28.4.1 Kernel default compilation configurations

The kernel is configured with CAN by default, and the configuration interface is as follows:

```
<*> ingenic on-chip CAN support
[*] ingenic can use dma transmission
```

CAN_CLOCK should be configured during kernel baud rate configuration, and CAN_CLOCK should be consistent with CAN bus clock in the device tree:

```
Symbol: CAN_CLOCK [=24]
Type : integer
Prompt: can clock in MHz
Location:
-> Machine selection
(1) -> SOC type (SOC_TYPE [=y])
Defined at arch/mips/xburst/Kconfig:131
Depends on: SOC_TYPE [=y] && MACH_XBURST [=y]
```

28.4.2 Kernel custom compilation configurations

Users can remove the driver configuration according to actual needs.

28.5 Module kernel differences

no difference

28.6 Device node generation

After the driver is successfully loaded, can0 or can1 will appear after executing ifconfig -

a.

```
# ifconfig -a

can0    Link encap:UNSPEC HWaddr 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00
        NOARP MTU:16 Metric:1
        RX packets:0 errors:0 dropped:0 overruns:0 frame:0
        TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
        collisions:0 txqueuelen:10
        RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)
        Interrupt:48

can1    Link encap:UNSPEC HWaddr 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00
        NOARP MTU:16 Metric:1
        RX packets:0 errors:0 dropped:0 overruns:0 frame:0
        TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
        collisions:0 txqueuelen:10
        RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)
        Interrupt:49
```

```
lo          Link encap:Local Loopback
            inet addr:127.0.0.1  Mask:255.0.0.0
            UP LOOPBACK RUNNING  MTU:65536  Metric:1
            RX packets:0 errors:0 dropped:0 overruns:0 frame:0
            TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
            collisions:0 txqueuelen:1
            RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
```

28.7 Application Instructions

1. can connection

Chip integrated CAN controller, but can not communicate. The CAN controller needs to connect to the transponder, and then send to the transponder of another CAN device.

When the transponder is connected. CAN_H connects to CAN_H and CAN_L connects to CAN_L. When there is a problem with the connection, a can bus error is reported.

2. Test the normal mode

2.1 Set the bit rate

The bit rate must be set before the CAN device is enabled.

Example Set the 1Mbps bit rate:

```
ip link set can0 type can bitrate 1000000
ip link set can1 type can bitrate 1000000
```

In this method, the kernel automatically calculates the bit rate to be set. Currently, the baud rate supported by the kernel driver is 1M, 500K, 250K, 100K, and 50K. Users can also add their own baud rate, details refer to the x1600 PM manual.

Note: If the baud rate is 20K or 10K, select an external 24M clock as the clock source.

2.2 Enable the can device

The can0 and can1 devices were enabled

```
ifconfig can0 up
ifconfig can0 up
```

2.3 Set can1 to receive data

```
candump can1 &
```

2.4 Set can0 to send data

Send a standard frame, an extended frame and a remote frame

```
cansend can0 -i 0x123 0x11 0x22 0x33 0x44 0x55 0x66 0x77 0x88
cansend can0 -i 0x12345 -e 0x11 0x22 0x33 0x44 0x55 0x66 0x77 0x88
cansend can0 -i 0x123 -r 0x11 0x22 0x33 0x44 0x55 0x66 0x77 0x88
```

2.5 Shut down the can

Shut down can0 and can1 devices

```
ifconfig can0 down
ifconfig can1 down
```

3. Check the can status

Example Query the parameter Settings of can0 device

```
ip -details link show can0
```

Query the working status of can0 during device operation

```
ip -details -statistics link show can0
```

4. Test in self-loop mode

This test shortens tx and rx inside the controller, sending tx data directly to rx.

You do not need to connect to a can device for testing.

4.1 Set the can rate and enable the loop mode

```
ip link set can0 type can bitrate 1000000 loopback on
```

4.2 Start can

```
ifconfig can0 up
```

4.3 Set can reception

```
candump can0 &
```

4.4 can send data

```
cansend can0 -i 0x123 0x11 0x22 0x33 0x44 0x55 0x66 0x77 0x88
```

4.5 Shut down the can

```
ifconfig can0 down
```



29 THE WATCHDOG MODULE

29.1 Module function introductions

CDBUS controller supports asynchronous serial communication protocol and supports the following functions:

1. Arbitration mechanism is introduced to automatically avoid conflicts such as CAN bus.
2. Support dual baud rate, maximum rate can greater than 10Mbps.
3. Supports unicast, multicast, and broadcast.
4. The maximum user data len is 253 bytes.
5. Hardware packaging, unpacking, validation, and filtering.
6. Backward compatible with traditional RS485 hardware (still retain arbitration function)

29.2 Driver source code location

Path of driver source code:

```
module_drivers/drivers/char/ingenic_cdbus.c
```

29.3 Device Tree Configurations

Location of the device tree:

```
module_drivers/dts/x1600.dtsi
```

CDBUS controller description:

```
cdbus: cdbus@0x13430000 {  
    compatible = "ingenic,x1600-cdbus";  
    reg = <0x13550000 0x10000>;  
    interrupt-parent = <&core_intc>;  
    interrupts = <IRQ_CDBUS>;  
    status = "ok";  
};
```

29.3.1 Default Configurations of Device Tree

The default compilation will generate CDBUS devices.

29.3.2 Device Tree Custom Configurations

Users can open the CDBUS device according to actual needs, and configure the node to oka:

```
&cdbus {  
    status = "okay";  
    pinctrl-names = "default";  
    pinctrl-0 = <&cdbus_pd>, <&cdbus_tx_en>;  
};
```

29.4 Kernel compilation configurations

The kernel configuration INGENIC_WDT, the configuration description is as follows:

```
Symbol: INGENIC_CDBUS [=y]
Type : boolean
Prompt: Ingenic Cdbus Driver
Location:
    -> Device Drivers
    -> Character devices
Defined at drivers/char/Kconfig
```

29.4.1 Kernel default compilation configurations

The kernel is configured with cdbus by default, and the configuration interface is as follows:

```
[RANDOM] drivers --->
[*] [CDBUS] Ingenic Cdbus Driver
[] [PWM-AUDIO] Ingenic Pwm Audio Driver
```

29.4.2 Kernel custom compilation configurations

Users can remove the driver configuration according to actual needs.

29.5 Module kernel differences

no difference

29.6 Device node generation

Generate device node after successful driver registration

```
# ls /dev/cdbus
```

29.7 Application Instructions

5. can connection

Chip integrated CAN controller, but can not communicate. The CAN controller needs to connect to the transponder, and then send to the transponder of another CAN device.

When the transponder is connected. CAN_H connects to CAN_H and CAN_L connects to CAN_L. When there is a problem with the connection, a can bus error is reported.

6. cdbus test requires two development boards for sending and receiving test.

7. The execution of the development board at the receiving end is as follows:

(1) Loading mcu firmware:

```
echo 1 > /sys/devices/platform/ahb2/13420000.mcu/load_fw
[ 22.179418] fw: c01a7000: 10000007 00000000 00000000 f4000000 .....
```



```
[ 22.186999] fw: c01a7010: f4002f50 f4002f28 f4002f10 f4002f70 P/.. (/.../..p/..
[ 22.195087] fw: c01a7020: 40806000 3c1af400 275a2f50 3c1bf400 .`.@...<P/Z'...<
[ 22.202752] fw: c01a7030: 277b68e0 135b0005 00000000 af400000 .h{'. [. . . . . @.
[ 22.210627] fw: c01a7040: 275a0004 175bffff 00000000 3c1df400 ..Z'.. [. . . . . <
[ 22.218203] fw: c01a7050: 27bd78e0 3c1af400 275a1f10 0340f809 .x.'...<..Z'..@.
[ 22.226065] fw: c01a7060: 00000000 00000000 00000000 00000000 .....
[ 22.233675] fw: c01a7070: 00000000 00000000 00000000 00000000 .....
```

(2) Execute the receiving procedure

```
# ./cdbus_test_recv

[ 32.319573] jz_cdbus_hw_init:373 >>> mode CDBUS-A mode
[ 32.324898] jz_cdbus_hw_init:384 >>> clock CDBUS-A mode
[ 32.330836] jz_cdbus_hw_init:399 >>> interrupt CDBUS-A mode
[ 32.336668] cdbus debug

=====recv=====

main():152 >>> recv header = 0x1fd420c
main():153 >>> recv src_addr = 0xc
main():154 >>> recv dst_addr = 0x42
main():155 >>> recv data_len = 253
main():156 >>> recv data:

 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34
35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 6
0 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90
91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 11
2 113 114 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135
136 137 138 139 140 141 142 143 144 145 146 147 148 149 150 151 152 153 154
155 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177
178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196
197 198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219
220 221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 2
39 240 241 242 243 244 245 246 247 248 249 250 251 252 253

=====
```

8. Run the following command on the development board of the sending end:

(1) Loading mcu firmware:

```
echo 1 > /sys/devices/platform/ahb2/13420000.mcu/load_fw

[ 22.179418] fw: c01a7000: 10000007 00000000 00000000 f4000000 .....
[ 22.186999] fw: c01a7010: f4002f50 f4002f28 f4002f10 f4002f70 P/.. (/.../..p/..
[ 22.195087] fw: c01a7020: 40806000 3c1af400 275a2f50 3c1bf400 .`.@...<P/Z'...<
[ 22.202752] fw: c01a7030: 277b68e0 135b0005 00000000 af400000 .h{'. [. . . . . @.
[ 22.210627] fw: c01a7040: 275a0004 175bffff 00000000 3c1df400 ..Z'.. [. . . . . <
[ 22.218203] fw: c01a7050: 27bd78e0 3c1af400 275a1f10 0340f809 .x.'...<..Z'..@.
[ 22.226065] fw: c01a7060: 00000000 00000000 00000000 00000000 .....
```



```
[ 22.233675] fw: c01a7070: 00000000 00000000 00000000 00000000 .....
```

(3) Execute the sending program

```
# ./cdbus_test_send
[ 42.600848] jz_cdbus_hw_init:373 >>> mode CDBUS-A mode
[ 42.606155] jz_cdbus_hw_init:384 >>> clock CDBUS-A mode
[ 42.612076] jz_cdbus_hw_init:399 >>> interrupt CDBUS-A mode

=====send=====

send header = 0x1fd420c
send src_addr = 12
send dst_addr = 66
send data_len = 253
send data:
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34
35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 6
0 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90
91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 11
2 113 114 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135
136 137 138 139 140 141 142 143 144 145 146 147 148 149 150 151 152 153 154
155 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177
178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196
197 198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219
220 221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 2
39 240 241 242 243 244 245 246 247 248 249 250 251 252 253

=====
```

9. Receiver test program code:

```
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ioctl.h>
#include <getopt.h>
#include <stdint.h>

struct cdbus_header {
    unsigned char src_addr;
    unsigned char dst_addr;
    unsigned char data_len;
};
```

```

struct cdbus_frame {
    struct cdbus_header header;
    unsigned char data[253];
};

enum cdbus_ioctl_cmd {
    CDBUS_A_MODE,
    CDBUS_BS_MODE,
    CDBUS_HAIF_FULL_DUPLEX_MODE,
    CDBUS_FULL_DUPLEX_MODE,
    CDBUS_SET_LOW_RATE,
    CDBUS_SET_HIGH_RATE,
    CDBUS_WRITE_FILTER,
    CDBUS_WRITE_FILTERM,
    CDBUS_ENABLE_LOOPBACK,
};

static const char *device = "/dev/cdbus";
static uint32_t self_addr = 66;
static int verbose = 1;

int main(int argc, char *argv[])
{
    int i;
    int fd, size;
    struct cdbus_frame recv_frame;

    fd = open(device, O_RDWR);
    if (fd <= 0) {
        printf("%s():%d open failed!\n", __func__, __LINE__);
        return -1;
    }

    ioctl(fd, CDBUS_WRITE_FILTER, self_addr);
    ioctl(fd, CDBUS_WRITE_FILTERM, 0x0);

    do {
        size = read(fd, &recv_frame, sizeof(struct cdbus_frame));
    } while (size == -1);

    if (verbose == 1) {
        printf("=====recv=====\\n");
    }
}

```

```

printf("%s():%d >>> recv header = 0x%x\n", __func__, __LINE__, recv_frame.header);
printf("%s():%d >>> recv src_addr = 0x%x\n", __func__, __LINE__, recv_frame.header.src_addr);
printf("%s():%d >>> recv dst_addr = 0x%x\n", __func__, __LINE__, recv_frame.header.dst_addr);
printf("%s():%d >>> recv data_len = %d\n", __func__, __LINE__, recv_frame.header.data_len);
printf("%s():%d >>> recv data:\n", __func__, __LINE__);

for (i = 0; i < recv_frame.header.data_len; i++) {
    printf("%d ", recv_frame.data[i]);
}

printf("\n");
printf("===== \n\n");
}

close(fd);
return 0;
}

```

10. Sent test program code:

```

#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ioctl.h>
#include <getopt.h>
#include <stdint.h>

struct cdbus_header {
    unsigned char src_addr;
    unsigned char dst_addr;
    unsigned char data_len;
};

struct cdbus_frame {
    struct cdbus_header header;
    unsigned char data[253];
};

enum cdbus_ioctl_cmd {
    CDBUS_A_MODE,

```

```

CDBUS_BS_MODE,
CDBUS_HAIF_FULL_DUPLEX_MODE,
CDBUS_FULL_DUPLEX_MODE,
CDBUS_SET_LOW_RATE,
CDBUS_SET_HIGH_RATE,
CDBUS_WRITE_FILTER,
CDBUS_WRITE_FILTERM,
CDBUS_ENABLE_LOOPBACK,
};

static const char *device = "/dev/cdbus";

static uint32_t src_addr = 1;
static uint32_t dst_addr = 66;
static uint32_t self_addr = 88;

static int verbose = 1;

int main(int argc, char *argv[])
{
    int i;
    unsigned int j = 0;
    int fd, size;
    int send_data_len;
    struct cdbus_frame send_frame;

    send_data_len = 253;

    fd = open(device, O_RDWR);
    if (fd <= 0) {
        printf("%s():%d open failed!\n", __func__, __LINE__);
        return -1;
    }

    ioctl(fd, CDBUS_WRITE_FILTER, self_addr);
    ioctl(fd, CDBUS_WRITE_FILTERM, 0x0);

    // src addr and dst addr
    send_frame.header.src_addr = src_addr;
    send_frame.header.dst_addr = dst_addr;

    // data_len

```

```

send_frame.header.data_len = send_data_len;

// data
for (i = 0; i < send_frame.header.data_len; i++)
    send_frame.data[i] = i + 1;

if (verbose == 1) {
    printf("=====send=====\n");
    printf("send header = 0x%x\n", send_frame.header);
    printf("send src_addr = %d\n", send_frame.header.src_addr);
    printf("send dst_addr = %d\n", send_frame.header.dst_addr);
    printf("send data_len = %d\n", send_frame.header.data_len);
    printf("send data:\n");
    for (i = 0; i < send_frame.header.data_len; i++) {
        printf("%d ", send_frame.data[i]);
    }
    printf("\n");
    printf("=====n\n");
}

write(fd, &send_frame, send_data_len + sizeof(struct cdbus_header));

close(fd);
return 0;
}

```